

src/idms_modernizer\core\config.py

```
from pathlib import Path

from pydantic_settings import BaseSettings


class Settings(BaseSettings):

    APP_NAME: str = (
        "IDMS DB2 Modernizer"
    )

    BASE_DIR: Path = Path.cwd()

    INPUT_DIR: str = "data/input"

    OUTPUT_DIR: str = "data/output"

    TEMP_DIR: str = "data/temp"

    DEBUG: bool = True

    @property
    def input_dir(self) -> str:
        return self.INPUT_DIR

    @property
    def output_dir(self) -> str:
        return self.OUTPUT_DIR

    @property
    def temp_dir(self) -> str:
        return self.TEMP_DIR

    class Config:
        env_file = ".env"

settings = Settings()
```


src/idms_modernizer\core\constants.py

```
SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)
```

```
)  
  
REPORT_FILE = (  
    "migration_report.pdf"  
)  
  
CANONICAL_MODEL_FILE = (  
    "canonical_model.json"  
)  
  
PHASE2_METADATA_FILE = (  
    "phase2_metadata.json"  
)
```

src/idms_modernizer\core/[exceptions.py](#)

```
class ModernizerException(  
    Exception  
) :  
    pass  
  
class PdfExtractionException(  
    ModernizerException  
) :  
    pass  
  
class SchemaParsingException(  
    ModernizerException  
) :  
    pass  
  
class ValidationException(  
    ModernizerException  
) :  
    pass
```

src/idms_modernizer\core/logger.py

```
import logging

def get_logger(
    logger_name: str
) -> logging.Logger:

    logger = logging.getLogger(
        logger_name
    )

    logger.setLevel(
        logging.INFO
    )

    if not logger.handlers:

        handler = (
            logging.StreamHandler()
        )

        formatter = (
            logging.Formatter(
                "%(asctime)s | %(levelname)s | %(message)s"
            )
        )

        handler.setFormatter(
            formatter
        )

        logger.addHandler(
            handler
        )

    return logger
```

src/idms_modernizer\core\utils.py

```
from pathlib import Path
```

```
def ensure_directory(  
    directory: str  
) -> None:
```

```
    Path(  
        directory  
    ).mkdir(  
        parents=True,  
        exist_ok=True  
    )
```

```
def normalize_text(  
    text: str  
) -> str:
```

```
    return " ".join(  
        text.split()  
    )
```

src/idms_modernizer\domain\canonical_models.py

```
from pydantic import BaseModel, Field

class CanonicalField(BaseModel):

    name: str

    datatype: str | None = None

    length: int | None = None

    scale: int | None = None

    occurs: bool = False

    occurs_max: int | None = None

class CanonicalRecord(BaseModel):

    name: str

    primary_key: str | None = None

    record_id: str | None = None

    location_mode: str | None = None

    fields: list["CanonicalField"] = Field(
        default_factory=list
    )

class CanonicalSet(BaseModel):

    name: str

    owner_record: str | None = None
```



```
member_record: str | None = None

order_mode: str | None = None


class CanonicalRelationship(BaseModel):

    set_name: str

    parent_record: str

    child_record: str

    cardinality: str = "1:N"


class CanonicalSchema(BaseModel):

    records: list[CanonicalRecord] = Field(
        default_factory=list
    )

    sets: list[CanonicalSet] = Field(
        default_factory=list
    )

    relationships: list[
        CanonicalRelationship
    ] = Field(
        default_factory=list
    )
```

src/idms_modernizer\domain\db2_models.py

```
from dataclasses import (
    dataclass,
    field
)

@dataclass
class DB2Column:

    name: str

    datatype: str

    nullable: bool = True

    primary_key: bool = False

@dataclass
class DB2ForeignKey:

    column_name: str

    reference_table: str

    reference_column: str

@dataclass
class DB2Table:

    name: str

    columns: list[DB2Column] = field(
        default_factory=list
    )

    foreign_keys: list[
```

```
        DB2ForeignKey
    ] = field(
        default_factory=list
    )

    primary_key: str | None = None

@dataclass
class DB2Model:

    tables: list[DB2Table] = field(
        default_factory=list
    )
```

src/idms_modernizer\domain\document_models.py

```
from pydantic import BaseModel, Field

class DocumentLine(BaseModel):
    line_number: int
    text: str

class DocumentPage(BaseModel):
    page_number: int
    lines: list[DocumentLine] = Field(
        default_factory=list
    )

class DocumentModel(BaseModel):
    pages: list[DocumentPage] = Field(
        default_factory=list
    )
```

src/idms_modernizer\domain\models.py

```
from pydantic import BaseModel

from idms_modernizer.domain.db2_models import Table
from idms_modernizer.domain.relationship_models import (
    Relationship
)
from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)

class ConversionContext(
    BaseModel
):

    schema_metadata: (
        SchemaMetadata | None
    ) = None

    relationships: (
        list[Relationship]
    ) = []

    db2_tables: (
        list[Table]
    ) = []
```

src/idms_modernizer\domain\relationship_models.py

```
from pydantic import BaseModel

class SetMembership(BaseModel):

    set_name: str

    role: str

class Relationship(BaseModel):

    set_name: str

    owner_record: str

    member_record: str

    cardinality: str = "1:N"
```

src/idms_modernizer\domain\schema_models.py

```
from dataclasses import dataclass, field

from pydantic import BaseModel, Field

from idms_modernizer.domain.relationship_models import (
    Relationship,
    SetMembership
)


class DataField(BaseModel):

    name: str

    datatype: str | None = None

    length: int | None = None

    scale: int | None = None

    picture: str | None = None


@dataclass
class Record:

    name: str

    fields: list[DataField] = field(
        default_factory=list
    )

    set_memberships: list[
        SetMembership
    ] = field(
        default_factory=list
    )
```

```
primary_key: str | None = None

class SetDefinition(BaseModel):

    name: str

    owner_record: str | None = None

    member_record: str | None = None

class SchemaMetadata(BaseModel):

    records: list[Record] = Field(
        default_factory=list
    )

    sets: list[SetDefinition] = Field(
        default_factory=list
    )

    relationships: list[Relationship] = Field(
        default_factory=list
    )
```


src/idms_modernizer\domain\section_models.py

```
from dataclasses import dataclass, field

@dataclass
class RecordSection:
    record_name: str
    lines: list[str] = field(
        default_factory=list
    )
```

src/idms_modernizer\extractors\field_extractor.py

```
import re
from typing import List, Set

from idms_modernizer.domain.schema_models import DataField

class FieldExtractor:
    """
    Simple and stable field discovery.

    Purpose:
    - Restore the earlier behavior where all fields were found.
    - Do not try to be clever about PICTURE parsing here.
    - PICTURE, length, scale are fixed later by SchemaPictureEnricher.

    This extractor finds field names from rows like:
    - 02 DEPT-ID-0410 DISPLAY 9(4) 1 4
    - 03 EMP-FIRST-NAME-0415 DISPLAY X(10) 5 10
    - 04 OFFICE-ZIP-FIRST-FIVE-0450 DISPLAY X(5) 41 5
    """

    FIELD_NAME_PATTERN = re.compile(
        r"^\s*(?P<level>0[1-9]|[1-4][0-9]|88)\s+"
        r"(?P<name>[A-Z][A-Z0-9-]*|FILLER)\b"
        r"(?P<rest>.*)$",
        re.IGNORECASE
    )

    USAGE_PATTERN = re.compile(
        r"\b(DISPLAY|COMP-3|COMP)\b",
        re.IGNORECASE
    )

    def extract(
        self,
        lines: List[str]
    ) -> List[DataField]:
```

```
print(
    "USING FIELD_EXTRACTOR VERSION SIMPLE-DISCOVERY-2026-06-30"
)

fields: List[DataField] = []
seen: Set[str] = set()

for raw_line in lines:
    line = self._clean_line(
        raw_line
    )

    if not line:
        continue

    match = self.FIELD_NAME_PATTERN.match(
        line
    )

    if not match:
        continue

    level = match.group(
        "level"
    )

    name = match.group(
        "name"
    ).upper()

    rest = match.group(
        "rest"
    )

    if level == "88":
        continue

    if name.startswith(
        "FILLER"
    ):
```

```

        continue

    if name in seen:
        continue

    usage = self._extract_usage(
        rest
    )

    fields.append(
        DataField(
            name=name,
            datatype=usage or "DISPLAY",
            length=None,
            scale=None,
            picture=None
        )
    )

    seen.add(
        name
    )

    return fields

def _clean_line(
    self,
    line: str | None
) -> str:

    if not line:
        return ""

    cleaned = str(line).strip()

    cleaned = re.sub(
        r"<[^>]+>",
        " ",
        cleaned
    )

```

```

        cleaned = cleaned.replace(
            "&nbsp;",
            " "
        )

        cleaned = cleaned.replace(
            "\t",
            " "
        )

        cleaned = cleaned.replace(
            "|",
            " "
        )

        cleaned = re.sub(
            r"\s+",
            " ",
            cleaned
        )

        return cleaned.strip()

def _extract_usage(
    self,
    text: str
) -> str | None:

    match = self.USAGE_PATTERN.search(
        text
    )

    if not match:
        return None

    return match.group(
        1
    ).upper()

```

src/idms_modernizer\extractors\owner_member_extractor.py

```
import re

from idms_modernizer.domain.relationship_models import (
    SetMembership
)

class OwnerMemberExtractor:

    SAME_LINE_PATTERN = re.compile(
        r"^\s*([A-Z0-9\-]+)\s+(?:INDEX\s+)?(OWNER|MEMBER)\b",
        re.IGNORECASE
    )

    SET_NAME_PATTERN = re.compile(
        r"^\s*([A-Z][A-Z0-9\-]+)\s*$",
        re.IGNORECASE
    )

    ROLE_PATTERN = re.compile(
        r"^\s*(OWNER|MEMBER)\b",
        re.IGNORECASE
    )

    IGNORED_SET_NAMES = {
        "CALC"
    }

    HEADER_WORDS = {
        "SET",
        "TYPE",
        "NEXT",
        "PRIOR",
        "OWNER",
        "MEMBER"
    }

    def extract(
```

```

        self,
        record_name: str,
        lines: list[str]
    ) -> list[SetMembership]:

        memberships = []
        seen = set()

        in_dbkey_block = False
        pending_set_name = None

        for raw_line in lines:

            line = raw_line.strip().upper()

            if not line:
                continue

            if "DBKEY POSITIONS" in line:
                in_dbkey_block = True
                pending_set_name = None
                continue

            if in_dbkey_block and line.startswith("DATA ITEM"):
                break

            if not in_dbkey_block:
                continue

            same_line_match = self.SAME_LINE_PATTERN.search(
                line
            )

            if same_line_match:

                set_name = same_line_match.group(1).upper()
                role = same_line_match.group(2).upper()

                pending_set_name = None

```

```
        if set_name in self.IGNORED_SET_NAMES:
            continue

        key = (
            set_name,
            role
        )

        if key not in seen:
            seen.add(key)

            memberships.append(
                SetMembership(
                    set_name=set_name,
                    role=role
                )
            )

            continue

    role_match = self.ROLE_PATTERN.search(
        line
    )

    if role_match and pending_set_name:

        set_name = pending_set_name
        role = role_match.group(1).upper()

        pending_set_name = None

        if set_name in self.IGNORED_SET_NAMES:
            continue

        key = (
            set_name,
            role
        )

        if key not in seen:
```



```

        seen.add(key)

        memberships.append(
            SetMembership(
                set_name=set_name,
                role=role
            )
        )

        continue

    set_name_match = self.SET_NAME_PATTERN.search(
        line
    )

    if set_name_match:

        candidate = set_name_match.group(1).upper()

        if candidate in self.HEADER_WORDS:
            continue

        if candidate in self.IGNORED_SET_NAMES:
            pending_set_name = None
            continue

        pending_set_name = candidate

    print(
        f"{record_name}: "
        f"{len(memberships)} memberships"
    )

    for membership in memberships:
        print(
            f"{record_name} -> "
            f"{membership.set_name} "
            f"{membership.role}"
        )

```

```
return memberships
```

src/idms_modernizer\extractors\primary_key_extractor.py

```
import re

from idms_modernizer.services.name_normalizer import NameNormalizer

class PrimaryKeyExtractor:

    CALC_PATTERN = re.compile(
        r"CALC\s+USING\s+([A-Z0-9-]+)",
        re.IGNORECASE
    )

    def extract(
        self,
        lines: list[str]
    ) -> str | None:

        for line in lines:
            match = self.CALC_PATTERN.search(line)

            if match:
                return NameNormalizer.normalize(
                    match.group(1)
                )

        return None
```

src/idms_modernizer\generators\ddl_generator.py

```
class DDLGenerator:

    def generate(
        self,
        db2_model
    ) -> str:

        ddl: list[str] = []

        for table in db2_model.tables:
            ddl.append(
                f"CREATE TABLE {table.name}"
            )
            ddl.append("(")

            column_definitions: list[str] = []
            added_columns: set[str] = set()

            for column in table.columns:
                if column.name in added_columns:
                    continue

                definition = (
                    f"    {column.name} "
                    f"{column.datatype}"
                )

                if not column.nullable:
                    definition += " NOT NULL"

                column_definitions.append(definition)
                added_columns.add(column.name)

            if table.primary_key:
                column_definitions.append(
                    f"    PRIMARY KEY "
```

```

        f"({table.primary_key})"
    )

    added_foreign_keys: set[tuple[str, str, str]] = set()

    for fk in table.foreign_keys:
        fk_key = (
            fk.column_name,
            fk.reference_table,
            fk.reference_column
        )

        if fk_key in added_foreign_keys:
            continue

        column_definitions.append(
            f"    FOREIGN KEY "
            f"({fk.column_name}) "
            f"REFERENCES "
            f"{fk.reference_table}"
            f"({fk.reference_column}) "
        )

        added_foreign_keys.add(fk_key)

    ddl.append(
        ",\n".join(column_definitions)
    )
    ddl.append(";")
    ddl.append("")

    return "\n".join(ddl)

```



```
relationships = []

all_sets = (
    set(owner_map.keys())
    | set(member_map.keys())
)

for set_name in all_sets:

    owners = owner_map.get(
        set_name,
        []
    )

    members = member_map.get(
        set_name,
        []
    )

    for owner in owners:

        for member in members:

            relationships.append(
                Relationship(
                    set_name=set_name,
                    owner_record=owner,
                    member_record=member
                )
            )

return relationships
```

src/idms_modernizer\graph\schema_graph.py

```
import networkx as nx

class SchemaGraph:

    def __init__(self):

        self.graph = nx.DiGraph()

    def add_record(
        self,
        record_name: str
    ):

        self.graph.add_node(
            record_name,
            node_type="record"
        )

    def add_relationship(
        self,
        owner: str,
        member: str,
        set_name: str
    ):

        self.graph.add_edge(
            owner,
            member,
            set_name=set_name
        )
```


src/idms_modernizer\parsers\cobol_pdf_extractor.py

```
import re
import pdfplumber

class CobolPdfExtractor:
    """
    Extracts COBOL text from PDF.

    Supports:
    - COBOL pasted as text in PDF
    - line-preserving extraction
    - fallback to empty string if extraction fails
    """

    def extract_text(
        self,
        pdf_path: str
    ) -> str:

        lines: list[str] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                page_text = (
                    page.extract_text()
                    or ""
                )

                for raw_line in page_text.splitlines():
                    cleaned_line = self._clean_line(
                        raw_line
                    )

                    lines.append(
                        cleaned_line
                    )

        return "\n".join(lines).strip()
```

```
def _clean_line(
    self,
    line: str | None
) -> str:

    if line is None:
        return ""

    cleaned = str(line).rstrip()

    cleaned = cleaned.replace(
        "\t",
        " "
    )

    cleaned = re.sub(
        r"[\u00A0]+",
        " ",
        cleaned
    )

    return cleaned
```

src/idms_modernizer\parsers\text_extractor.py

```
import re
import pdfplumber

from idms_modernizer.domain.document_models import (
    DocumentLine,
    DocumentModel,
    DocumentPage
)

class TextExtractor:
    """
    Plain text PDF extractor.

    Important:
    - Uses only page.extract_text().
    - Does not use extract_tables().
    - Does not use extract_words().
    - This keeps the useful rows seen in your debug, such as:
      03 EMP-FIRST-NAME-0415 DISPLAY X(10) 5 10
    """

    def extract_document(
        self,
        pdf_path: str
    ) -> DocumentModel:

        pages: list[DocumentPage] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page_index, page in enumerate(pdf.pages):
                text = (
                    page.extract_text()
                    or ""
                )

                lines: list[DocumentLine] = []
```

```

        for line_index, raw_line in enumerate(
            text.splitlines()
        ):
            cleaned_line = self._clean_line(
                raw_line
            )

            if not cleaned_line:
                continue

            lines.append(
                DocumentLine(
                    line_number=line_index,
                    text=cleaned_line
                )
            )

        pages.append(
            DocumentPage(
                page_number=page_index,
                lines=lines
            )
        )

    return DocumentModel(
        pages=pages
    )

def _clean_line(
    self,
    line: str | None
) -> str:

    if not line:
        return ""

    cleaned = str(line).strip()

    cleaned = re.sub(
        r"<[^>]+>",

```

```
        " ",
        cleaned
    )

    cleaned = cleaned.replace(
        "&nbsp;",
        " "
    )

    cleaned = cleaned.replace(
        "\t",
        " "
    )

    cleaned = cleaned.replace(
        "|",
        " "
    )

    cleaned = re.sub(
        r"\s+",
        " ",
        cleaned
    )

    return cleaned.strip()
```

src/idms_modernizer\services\business_datatype_inference.py

```
class BusinessDatatypeInference:

    IDENTIFIER_WORDS = {
        "PHONE",
        "ZIP",
        "POSTAL",
        "SSN",
        "SS_NUMBER",
        "ACCOUNT",
        "POLICY",
        "LICENSE",
        "CODE"
    }

    @staticmethod
    def infer(
        field_name: str,
        current_type: str
    ) -> str:

        upper_name = field_name.upper()
        current_type = (
            current_type
            or "VARCHAR"
        ).upper()

        for word in BusinessDatatypeInference.IDENTIFIER_WORDS:
            if word in upper_name:
                if current_type in {
                    "INTEGER",
                    "DECIMAL",
                    "COMP",
                    "COMP-3"
                }:
                    return "VARCHAR"

        return current_type
```

```
return current_type
```

src/idms_modernizer\services\canonical_reconciler.py

```
from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalField
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer

class CanonicalReconciler:
    """
    Reconciles canonical schema with IDMS schema metadata.

    IDMS schema is authoritative for:
    - datatype
    - length
    - scale
    - picture

    Canonical schema is authoritative for:
    - normalized record names
    - primary keys
    - relationships
    """

    def reconcile(
        self,
        canonical_schema: CanonicalSchema,
        metadata: SchemaMetadata
    ) -> CanonicalSchema:

        idms_field_lookup = self._build_idms_field_lookup(
            metadata
        )

        for record in canonical_schema.records:
            normalized_record_name = NameNormalizer.normalize(
                record.name
            )
```



```

reconciled_fields: list[CanonicalField] = []

for field in record.fields:
    normalized_field_name = NameNormalizer.normalize(
        field.name
    )

    idms_field = idms_field_lookup.get(
        (
            normalized_record_name,
            normalized_field_name
        )
    )

    if idms_field is None:
        idms_field = idms_field_lookup.get(
            (
                normalized_record_name,
                self._remove_record_suffix(
                    normalized_field_name
                )
            )
        )

    if idms_field is None:
        reconciled_fields.append(field)
        continue

    reconciled_fields.append(
        CanonicalField(
            name=field.name,
            datatype=idms_field.datatype,
            length=idms_field.length,
            scale=idms_field.scale,
            occurs=field.occurs,
            occurs_max=field.occurs_max
        )
    )

```

```

        record.fields = reconciled_fields

    return canonical_schema

def _build_idms_field_lookup(
    self,
    metadata: SchemaMetadata
) -> dict[tuple[str, str], object]:

    lookup: dict[tuple[str, str], object] = {}

    for record in metadata.records:
        record_name = NameNormalizer.normalize(
            record.name
        )

        for field in record.fields:
            field_name = NameNormalizer.normalize(
                field.name
            )

            lookup[
                (
                    record_name,
                    field_name
                )
            ] = field

            lookup[
                (
                    record_name,
                    self._remove_record_suffix(field_name)
                )
            ] = field

    return lookup

def _remove_record_suffix(
    self,
    field_name: str

```

```
) -> str:

    parts = field_name.split("_")

    if (
        len(parts) > 1
        and parts[-1].isdigit()
        and len(parts[-1]) == 4
    ):
        return "_".join(parts[:-1])

    return field_name
```

src/idms_modernizer\services\canonical_schema_builder.py

```
from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalRecord,
    CanonicalField,
    CanonicalSet,
    CanonicalRelationship
)

from idms_modernizer.services.name_normalizer import (
    NameNormalizer
)

from idms_modernizer.services.date_field_consolidator import (
    DateFieldConsolidator
)

class CanonicalSchemaBuilder:

    def build(
        self,
        metadata
    ) -> CanonicalSchema:

        schema = CanonicalSchema()

        #
        # Records
        #
        for record in metadata.records:

            print(
                f"CanonicalBuilder Input: "
                f"{record.name} "
                f"PK={record.primary_key}"
            )

            canonical_record = CanonicalRecord(
```

```

        name=NameNormalizer.normalize(
            record.name
        ),
        primary_key=(
            NameNormalizer.normalize(
                record.primary_key
            )
            if record.primary_key
            else None
        )
    )

print(
    f"CanonicalRecord Created: "
    f"{canonical_record.name} "
    f"PK={canonical_record.primary_key}"
)

normalized_fields = (
    DateFieldConsolidator consolidate(
        record.fields
    )
)

added_fields = set()

for field in normalized_fields:

    field_name = (
        NameNormalizer.normalize(
            field.name
        )
    )

    if field_name in added_fields:
        continue

    added_fields.add(
        field_name
    )

```

```

        canonical_record.fields.append(
            CanonicalField(
                name=field_name,
                datatype=field.datatype,
                length=field.length,
                scale=field.scale
            )
        )

    schema.records.append(
        canonical_record
    )

#
# Sets
#
added_relationships = set()
for rel in metadata.relationships:

    key = (
        rel.owner_record,
        rel.member_record,
        rel.set_name
    )

    if key in added_relationships:
        continue

    added_relationships.add(
        key
    )

    schema.sets.append(
        CanonicalSet(
            name=rel.set_name,
            owner_record=NameNormalizer.normalize(
                rel.owner_record
            ),
            member_record=NameNormalizer.normalize(
                rel.member_record
            )
        )
    )

```

```
        )
    )
)

schema.relationships.append(
    CanonicalRelationship(
        set_name=rel.set_name,
        parent_record=NameNormalizer.normalize(
            rel.owner_record
        ),
        child_record=NameNormalizer.normalize(
            rel.member_record
        ),
        cardinality="1:N"
    )
)

return schema
```

src/idms_modernizer\services\date_field_consolidator.py

```
import re

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer


class DateFieldConsolidator:

    DATE_PREFIXES = [
        "START",
        "FINISH",
        "BIRTH",
        "CLAIM",
        "SERVICE",
        "ADMIT",
        "DISCHARGE",
        "TERMINATION",
        "SELECTION",
        "EXPERTISE",
        "STRUCTURE",
        "PATIENT_BIRTH"
    ]

    @staticmethod
    def consolidate(
        fields: list[DataField]
    ) -> list[DataField]:

        result: list[DataField] = []
        consumed: set[str] = set()
        added_dates: set[str] = set()

        field_lookup = {
            NameNormalizer.normalize(field.name): field
            for field in fields
        }

        for field in fields:
            normalized_name = NameNormalizer.normalize(
```



```

        field.name
    )

    if field.datatype == "DATE":
        if normalized_name not in added_dates:
            result.append(field)
            added_dates.add(normalized_name)

for prefix in DateFieldConsolidator.DATE_PREFIXES:
    pattern = re.compile(
        rf"^{prefix}_YEAR_(\d+)$",
        re.IGNORECASE
    )

    for field_name in list(field_lookup.keys()):
        match = pattern.match(field_name)

        if not match:
            continue

        suffix = match.group(1)

        year_name = f"{prefix}_YEAR_{suffix}"
        month_name = f"{prefix}_MONTH_{suffix}"
        day_name = f"{prefix}_DAY_{suffix}"
        date_name = f"{prefix}_DATE_{suffix}"

        if (
            year_name in field_lookup
            and month_name in field_lookup
            and day_name in field_lookup
        ):
            if date_name not in added_dates:
                result.append(
                    DataField(
                        name=date_name,
                        datatype="DATE",
                        length=None,
                        scale=None,
                        picture=None

```

```
        )
    )

    added_dates.add(date_name)

    consumed.add(year_name)
    consumed.add(month_name)
    consumed.add(day_name)

for field in fields:
    normalized_name = NameNormalizer.normalize(
        field.name
    )

    if normalized_name in consumed:
        continue

    if normalized_name in added_dates:
        continue

    result.append(field)

return result
```

src/idms_modernizer\services\db2_datatype_mapper.py

```
class DB2DatatypeMapper:

    TEXT_IDENTIFIER_WORDS = {
        "PHONE",
        "ZIP",
        "POSTAL",
        "SSN",
        "SS_NUMBER",
        "LICENSE",
        "CODE"
    }

    @staticmethod
    def map(field) -> str:
        datatype = (
            field.datatype
            or "VARCHAR"
        ).upper()

        field_name = (
            field.name
            or ""
        ).upper()

        length = field.length
        scale = field.scale

        if datatype == "DATE":
            return "DATE"

        if datatype in {
            "TIMESTAMP",
            "DATETIME"
        }:
            return "TIMESTAMP"

        if DB2DatatypeMapper._should_force_text_identifier(
            field_name=field_name,
            datatype=datatype
        ):
```

```

):
    return DB2DatatypeMapper._map_text_identifier(
        length=length
    )

if datatype == "DISPLAY":
    actual_length = (
        length
        if length
        else 255
    )

    if actual_length == 1:
        return "CHAR(1)"

    return f"VARCHAR({actual_length})"

if datatype == "CHAR":
    actual_length = (
        length
        if length
        else 1
    )

    return f"CHAR({actual_length})"

if datatype == "VARCHAR":
    actual_length = (
        length
        if length
        else 255
    )

    if actual_length == 1:
        return "CHAR(1)"

    return f"VARCHAR({actual_length})"

if datatype == "INTEGER":
    actual_length = (

```

```

        length
        if length
        else 10
    )

    if actual_length <= 4:
        return "SMALLINT"

    if actual_length <= 9:
        return "INTEGER"

    return "BIGINT"

if datatype == "DECIMAL":
    precision = (
        length
        if length
        else 18
    )

    actual_scale = (
        scale
        if scale is not None
        else 0
    )

    return f"DECIMAL({precision},{actual_scale})"

if datatype == "COMP":
    return "INTEGER"

if datatype == "COMP-3":
    precision = (
        length
        if length
        else 18
    )

    actual_scale = (
        scale

```

```

        if scale is not None
        else 0
    )

    return f"DECIMAL({precision},{actual_scale})"

return "VARCHAR(255)"

@staticmethod
def _should_force_text_identifier(
    field_name: str,
    datatype: str
) -> bool:

    if datatype not in {
        "INTEGER",
        "DECIMAL",
        "COMP",
        "COMP-3",
        "DISPLAY"
    }:
        return False

    for word in DB2DatatypeMapper.TEXT_IDENTIFIER_WORDS:
        if word in field_name:
            return True

    return False

@staticmethod
def _map_text_identifier(
    length: int | None
) -> str:

    actual_length = (
        length
        if length
        else 255
    )

```

```
if actual_length == 1:  
    return "CHAR(1) "  
  
return f"VARCHAR({actual_length}) "
```

src/idms_modernizer\services\db2_generation_service.py

```
from idms_modernizer.generators.ddl_generator import (
    DDLGenerator
)

from idms_modernizer.services.db2_mapping_service import (
    DB2MappingService
)

class DB2GenerationService:

    def __init__(self):

        self.mapper = (
            DB2MappingService()
        )

        self.ddl_generator = (
            DDLGenerator()
        )

    def generate(
        self,
        canonical_schema
    ):

        db2_model = (
            self.mapper.build_db2_model(
                canonical_schema
            )
        )

        ddl = (
            self.ddl_generator.generate(
                db2_model
            )
        )
```



```
return db2_model, ddl
```

src/idms_modernizer\services\db2_mapping_service.py

```
from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2ForeignKey,
    DB2Model,
    DB2Table
)
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
from idms_modernizer.services.db2_name_normalizer import DB2NameNormalizer

class DB2MappingService:

    def build_db2_model(
        self,
        schema: CanonicalSchema
    ) -> DB2Model:

        tables: list[DB2Table] = []

        for record in schema.records:
            table = DB2Table(
                name=DB2NameNormalizer.normalize_record_name(
                    record.name
                ),
                columns=[],
                foreign_keys=[],
                primary_key=(
                    DB2NameNormalizer.normalize_column_name(
                        record.primary_key
                    )
                    if record.primary_key
                    else None
                )
            )

            added_columns: set[str] = set()
```

```

        for field in record.fields:
            column_name = DB2NameNormalizer.normalize_column_name(
                field.name
            )

            if not column_name:
                continue

            if column_name in added_columns:
                continue

            is_primary_key = (
                table.primary_key is not None
                and column_name == table.primary_key
            )

            table.columns.append(
                DB2Column(
                    name=column_name,
                    datatype=DB2DatatypeMapper.map(field),
                    nullable=not is_primary_key,
                    primary_key=is_primary_key
                )
            )

            added_columns.add(column_name)

        tables.append(table)

    self._add_foreign_keys(
        schema=schema,
        tables=tables
    )

    return DB2Model(
        tables=tables
    )

def _add_foreign_keys(
    self,

```

```

        schema: CanonicalSchema,
        tables: list[DB2Table]
    ) -> None:

        added_foreign_keys: set[tuple[str, str, str, str]] = set()

        for set_def in schema.sets:
            owner_table = self._find_table(
                tables=tables,
                table_name=set_def.owner_record
            )

            member_table = self._find_table(
                tables=tables,
                table_name=set_def.member_record
            )

            if owner_table is None:
                continue

            if member_table is None:
                continue

            if not owner_table.primary_key:
                continue

            parent_pk_column = self._find_column(
                table=owner_table,
                column_name=owner_table.primary_key
            )

            if parent_pk_column is None:
                continue

            fk_column_name = owner_table.primary_key

            fk_key = (
                member_table.name,
                fk_column_name,
                owner_table.name,

```

```

        owner_table.primary_key
    )

    if fk_key in added_foreign_keys:
        continue

    member_fk_column = self._find_column(
        table=member_table,
        column_name=fk_column_name
    )

    if member_fk_column is None:
        member_table.columns.append(
            DB2Column(
                name=fk_column_name,
                datatype=parent_pk_column.datatype,
                nullable=True,
                primary_key=False
            )
        )

        member_table.foreign_keys.append(
            DB2ForeignKey(
                column_name=fk_column_name,
                reference_table=owner_table.name,
                reference_column=owner_table.primary_key
            )
        )

        added_foreign_keys.add(fk_key)

def _find_table(
    self,
    tables: list[DB2Table],
    table_name: str | None
) -> DB2Table | None:

    normalized_name = DB2NameNormalizer.normalize_record_name(
        table_name
    )

```

```
        if not normalized_name:
            return None

        for table in tables:
            if table.name == normalized_name:
                return table

        return None

def _find_column(
    self,
    table: DB2Table,
    column_name: str | None
) -> DB2Column | None:

    normalized_name = DB2NameNormalizer.normalize_column_name(
        column_name
    )

    if not normalized_name:
        return None

    for column in table.columns:
        if column.name == normalized_name:
            return column

    return None
```

src/idms_modernizer\services\db2_model_service.py

```
from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model
from idms_modernizer.services.db2_mapping_service import DB2MappingService

class DB2ModelBuilder:

    def __init__(self):
        self.mapping_service = DB2MappingService()

    def build(
        self,
        schema: CanonicalSchema
    ) -> DB2Model:

        return self.mapping_service.build_db2_model(
            schema
        )
```

src/idms_modernizer\services\db2_model_service.py

```
from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Model,
    DB2Table
)

class DB2ModelService:

    def build(
        self,
        metadata
    ) -> DB2Model:

        tables = []

        for record in metadata.records:

            columns = []

            for field in record.fields:

                columns.append(
                    DB2Column(
                        name=field.name,
                        datatype="VARCHAR(255) "
                    )
                )

            tables.append(
                DB2Table(
                    name=record.name,
                    columns=columns,
                    foreign_keys=[]
                )
            )

        return DB2Model(
            tables=tables
        )
```


)

src/idms_modernizer\services\db2_name_normalizer.py

```
from idms_modernizer.services.name_normalizer import NameNormalizer

class DB2NameNormalizer:

    REMOVE_RECORD_SUFFIX = False

    @staticmethod
    def normalize_record_name(
        name: str | None
    ) -> str:

        return NameNormalizer.normalize(name)

    @staticmethod
    def normalize_column_name(
        name: str | None
    ) -> str:

        normalized = NameNormalizer.normalize(name)

        if not DB2NameNormalizer.REMOVE_RECORD_SUFFIX:
            return normalized

        parts = normalized.split("_")

        if (
            len(parts) > 1
            and parts[-1].isdigit()
            and len(parts[-1]) == 4
        ):
            return "_".join(parts[:-1])

        return normalized
```

src/idms_modernizer\services\document_segmenter.py

```
import re

from idms_modernizer.domain.section_models import (
    RecordSection
)

class DocumentSegmenter:

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z0-9\-]+)",
        re.IGNORECASE
    )

    def segment(
        self,
        document
    ) -> list[RecordSection]:

        sections = []

        current_section = None

        for page in document.pages:

            for line in page.lines:

                text = line.text.strip()

                record_match = (
                    self.RECORD_PATTERN.search(
                        text
                    )
                )

                if record_match:

                    if current_section:
```

```
        sections.append(
            current_section
        )

        current_section = (
            RecordSection(
                record_name=record_match.group(1),
                lines=[]
            )
        )

    elif current_section:

        current_section.lines.append(
            text
        )

    if current_section:

        sections.append(
            current_section
        )

    return sections
```

src/idms_modernizer\services\er_diagram_generator.py

```
import os
from pathlib import Path
from graphviz import Digraph

from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)

def generate_er_diagram(
    metadata: SchemaMetadata,
    output_path: str
) -> str:
    """Generate a simple ER diagram PNG from the canonical metadata.

    Nodes are records and edges are relationships (labeled with set name
    and cardinality if present).
    Returns the path to the generated PNG file.
    """

    out = Path(output_path)
    out.parent.mkdir(parents=True, exist_ok=True)
    os.environ["PATH"] += (
        os.pathsep +
        r"C:\Program Files\Graphviz\bin"
    )

    dot = Digraph("er", format="png")
    dot.attr(rankdir="LR")
    dot.attr("node", shape="box")

    # Add record nodes
    for record in metadata.records:
        # simple label: record name
        dot.node(record.name, label=record.name)

    # Add relationships
    for rel in metadata.relationships:
```

```

    owner = rel.owner_record
    member = rel.member_record
    label = rel.set_name
    if getattr(rel, "cardinality", None):
        label = f"{label}\n{rel.cardinality}"

    dot.edge(owner, member, label=label)

# Render to the requested path (without extension)
filename_no_ext = str(out.with_suffix(""))

try:

    dot.render(
        filename=filename_no_ext,
        cleanup=True
    )

except Exception as ex:

    raise RuntimeError(
        "Graphviz not installed. "
        "Install Graphviz and add "
        "'C:\\Program Files\\Graphviz\\bin' "
        "to PATH."
    ) from ex

# Graphviz writes the .png next to filename_no_ext
png_path = filename_no_ext + ".png"

return png_path

```

src/idms_modernizer\services\metadata_builder.py

```
import re

from idms_modernizer.domain.document_models import (
    DocumentModel
)

from idms_modernizer.domain.schema_models import (
    DataField,
    Record,
    SchemaMetadata,
)

class MetadataBuilder:

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z][A-Z0-9\-\-]+)",
        re.IGNORECASE
    )

    FIELD_PATTERN = re.compile(
        r"^\d{2}\s+([A-Z][A-Z0-9\-\-]+)",
        re.IGNORECASE
    )

    def build(
        self,
        document: DocumentModel
    ) -> SchemaMetadata:

        metadata = SchemaMetadata()

        current_record = None

        seen_records = set()

        for page in document.pages:
```

```
for line in page.lines:

    text = line.text.strip()

    if not text:
        continue

    #
    # RECORD DETECTION
    #

    record_match = (
        self.RECORD_PATTERN.search(
            text
        )
    )

    if record_match:

        record_name = (
            record_match.group(1)
            .strip()
            .upper()
        )

        if (
            record_name
            not in seen_records
        ):

            current_record = Record(
                name=record_name
            )

            metadata.records.append(
                current_record
            )

            seen_records.add(
                record_name
            )
```



```

        )

        continue

    #
    # FIELD DETECTION
    #

    if current_record:

        field_match = (
            self.FIELD_PATTERN.match(
                text
            )
        )

        if field_match:

            field_name = (
                field_match.group(1)
                .strip()
                .upper()
            )

            current_record.fields.append(
                DataField(
                    name=field_name
                )
            )

    #
    # DEBUG OUTPUT
    #

    print(
        "\n===== RECORDS ====="
    )

    for record in metadata.records:

```

```
        print(
            f"{record.name} -> "
            f"{len(record.fields)} fields"
        )

    print(
        "=====\n"
    )

    return metadata
```

src/idms_modernizer/services\metadata_service.py

```
from idms_modernizer.parsers.text_extractor import (
    TextExtractor
)

from idms_modernizer.services.document_segmenter import (
    DocumentSegmenter
)

from idms_modernizer.extractors.owner_member_extractor import (
    OwnerMemberExtractor
)

from idms_modernizer.extractors.field_extractor import (
    FieldExtractor
)

from idms_modernizer.services.schema_picture_enricher import (
    SchemaPictureEnricher
)

from idms_modernizer.services.relationship_resolver import (
    RelationshipResolver
)

from idms_modernizer.domain.schema_models import (
    SchemaMetadata,
    Record
)

from idms_modernizer.extractors.primary_key_extractor import (
    PrimaryKeyExtractor
)

class MetadataService:

    def __init__(self):
```

```

self.extractor = TextExtractor()
self.segmenter = DocumentSegmenter()

self.field_extractor = FieldExtractor()
self.picture_enricher = SchemaPictureEnricher()

self.membership_extractor = OwnerMemberExtractor()

self.relationship_resolver = RelationshipResolver()

self.primary_key_extractor = PrimaryKeyExtractor()

def build_metadata(
    self,
    pdf_path: str
) -> SchemaMetadata:

    document = self.extractor.extract_document(pdf_path)

    sections = self.segmenter.segment(document)

    metadata = SchemaMetadata()

    for section in sections:
        record = Record(
            name=section.record_name
        )

        discovered_fields = self.field_extractor.extract(
            section.lines
        )

        record.fields = self.picture_enricher.enrich(
            fields=discovered_fields,
            lines=section.lines
        )

    print("=" * 80)
    print(f"FIELDS EXTRACTED FOR {record.name}")
    print("=" * 80)

```

```
        for field in record.fields:
            print(
                field.name,
                field.datatype,
                field.length,
                field.scale,
                field.picture
            )

        record.set_memberships = self.membership_extractor.extract(
            section.record_name,
            section.lines
        )

        record.primary_key = self.primary_key_extractor.extract(
            section.lines
        )

        print(
            f"MetadataService: "
            f"{record.name} PK="
            f"{record.primary_key}"
        )

        metadata.records.append(record)

    metadata.relationships = self.relationship_resolver.resolve(
        metadata
    )

    print(
        f"Relationships Found: "
        f"{len(metadata.relationships)}"
    )

    return metadata
```

src/idms_modernizer/services/name_normalizer.py

```
import re

class NameNormalizer:

    @staticmethod
    def normalize(
        name: str | None
    ) -> str:

        if not name:
            return ""

        normalized = name.strip().upper()
        normalized = normalized.replace("-", "_")
        normalized = re.sub(
            r"^[A-Z0-9_]",
            "_",
            normalized
        )
        normalized = re.sub(
            r"_+",
            "_",
            normalized
        )

        return normalized.strip("_")
```

src/idms_modernizer/services\phase2_conversion_bridge.py

```
class Phase2ConversionBridge:
    """
    Bridge from Phase 1 outputs to the Phase 2 converter.

    Import is intentionally lazy so Phase 1 UI can still load even if
    Phase 2 package has a missing or broken module.
    """

    def convert(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str,
        ddl_text: str,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
        use_ddl: bool = True,
        use_idms_schema: bool = False,
        use_overrides: bool = False
    ) -> tuple[str, list[str]]:

        if not cobol_text or not cobol_text.strip():
            raise ValueError(
                "COBOL code is required for Phase 2 conversion."
            )

        try:
            from idms_db2_converter.services.conversion_service import (
                ConversionService
            )

        except Exception as exc:
            raise RuntimeError(
                "Phase 2 converter package could not be loaded. "
                "Check src/idms_db2_converter files. "
                f"Details: {exc}"
            ) from exc
```

```

if not canonical_json or not canonical_json.strip():
    canonical_json = '{"records": [], "sets": [], "relationships":
[]}'

if not phase2_metadata_json:
    phase2_metadata_json = ""

if not ddl_text:
    ddl_text = ""

if not idms_schema_text:
    idms_schema_text = ""

if not relationship_overrides_json:
    relationship_overrides_json = ""

service = ConversionService()

converted, validation_messages = service.convert_retrieval(
    cobol_text=cobol_text,
    canonical_json=canonical_json,
    phase2_metadata_json=phase2_metadata_json,
    ddl_text=ddl_text if use_ddl else None,
    idms_schema_text=(
        idms_schema_text
        if use_idms_schema
        else None
    ),
    relationship_overrides_json=(
        relationship_overrides_json
        if use_overrides and relationship_overrides_json.strip()
        else None
    ),
    target_program=(
        target_program.strip()
        if target_program and target_program.strip()
        else None
    )
)

```



```
return converted, validation_messages
```

src/idms_modernizer/services/phase2_metadata_generator.py

```
import json
import re
from datetime import datetime
from typing import Any

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model, DB2Table,
DB2Column
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer


class Phase2MetadataGenerator:
    """
    Generates Phase 2 metadata JSON from Phase 1 outputs.

    Inputs:
    - CanonicalSchema
    - DB2Model
    - SchemaMetadata

    Output:
    - Phase 2 metadata JSON compatible with the Phase 2 converter.

    Purpose:
    - Map IDMS record names to DB2 table names.
    - Map IDMS field names to DB2 host variables.
    - Provide set relationship metadata.
    - Provide date-part metadata for YEAR/MONTH/DAY legacy fields.
    - Provide nullable FK hints.
    - Provide CALC key mappings.
    - Provide enough schema information for Phase 2 when DDL is absent.
    """

    VERSION = "1.0"

    DATE_PARTS = {
        "YEAR": {
            "substring_start": 3,
```

```

        "substring_length": 2
    },
    "MONTH": {
        "substring_start": 6,
        "substring_length": 2
    },
    "DAY": {
        "substring_start": 9,
        "substring_length": 2
    }
}

def generate(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata
) -> dict[str, Any]:

    db2_table_lookup = self._build_db2_table_lookup(
        db2_model
    )

    canonical_record_lookup = self._build_canonical_record_lookup(
        canonical_schema
    )

    record_table_map = self._build_record_table_map(
        canonical_schema=canonical_schema,
        metadata=metadata
    )

    records_payload = self._build_records_payload(
        db2_model=db2_model
    )

    field_map = self._build_field_map(
        metadata=metadata,
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup,

```

```
        record_table_map=record_table_map
    )

    calc_key_map = self._build_calc_key_map(
        canonical_schema=canonical_schema,
        metadata=metadata,
        record_table_map=record_table_map
    )

    relationships_payload = self._build_relationships_payload(
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup
    )

    set_ordering_map = self._build_set_ordering_map(
        relationships_payload
    )

    nullable_fk_map = self._build_nullable_fk_map(
        db2_model=db2_model
    )

    date_part_map = self._build_date_part_map(
        metadata=metadata,
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup,
        record_table_map=record_table_map
    )

    navigation_intent = self._build_navigation_intent(
        relationships_payload
    )

    output_semantics = self._build_output_semantics()

    validation_messages = self._build_validation_messages(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata,
        field_map=field_map
    )
```

```

    )

    payload = {
        "version": self.VERSION,
        "generated_at": datetime.utcnow().isoformat() + "Z",
        "source": "PHASE1",
        "schema_source": "PHASE1_CANONICAL_DB2_METADATA",
        "record_table_map": record_table_map,
        "records": records_payload,
        "relationships": relationships_payload,
        "field_map": field_map,
        "calc_key_map": calc_key_map,
        "set_ordering_map": set_ordering_map,
        "navigation_intent": navigation_intent,
        "nullable_fk_map": nullable_fk_map,
        "date_part_map": date_part_map,
        "output_semantics": output_semantics,
        "paragraph_operation_graph": {},
        "validation_messages": validation_messages
    }

    return payload

def generate_json(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata
) -> str:

    payload = self.generate(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata
    )

    return json.dumps(
        payload,
        indent=2
    )

```

```

def _build_db2_table_lookup(
    self,
    db2_model: DB2Model
) -> dict[str, DB2Table]:

    lookup: dict[str, DB2Table] = {}

    for table in db2_model.tables:
        lookup[
            NameNormalizer.normalize(table.name)
        ] = table

    return lookup

def _build_canonical_record_lookup(
    self,
    canonical_schema: CanonicalSchema
) -> dict[str, Any]:

    lookup: dict[str, Any] = {}

    for record in canonical_schema.records:
        lookup[
            NameNormalizer.normalize(record.name)
        ] = record

    return lookup

def _build_record_table_map(
    self,
    canonical_schema: CanonicalSchema,
    metadata: SchemaMetadata
) -> dict[str, str]:

    record_table_map: dict[str, str] = {}

    for record in canonical_schema.records:
        canonical_name = NameNormalizer.normalize(
            record.name

```

```

    )

    record_table_map[
        canonical_name
    ] = canonical_name

    for record in metadata.records:
        idms_record_name = NameNormalizer.normalize(
            record.name
        )

        canonical_record_name = NameNormalizer.normalize(
            record.name
        )

        record_table_map[
            idms_record_name
        ] = canonical_record_name

    return record_table_map

def _build_records_payload(
    self,
    db2_model: DB2Model
) -> list[dict[str, Any]]:

    records_payload: list[dict[str, Any]] = []

    for table in db2_model.tables:
        fields_payload = []

        for column in table.columns:
            datatype, length, scale = self._parse_db2_datatype(
                column.datatype
            )

            fields_payload.append(
                {
                    "name": column.name,
                    "column": column.name,

```

```

        "datatype": datatype,
        "length": length,
        "scale": scale,
        "nullable": column.nullable,
        "primary_key": column.primary_key
    }
)

records_payload.append(
    {
        "name": table.name,
        "table": table.name,
        "primary_key": table.primary_key,
        "fields": fields_payload
    }
)

return records_payload

def _build_field_map(
    self,
    metadata: SchemaMetadata,
    canonical_schema: CanonicalSchema,
    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str]
) -> dict[str, dict[str, Any]]:

    field_map: dict[str, dict[str, Any]] = {}

    for record in metadata.records:
        idms_record_name = NameNormalizer.normalize(
            record.name
        )

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name
        )

        table = db2_table_lookup.get(

```



```

        table_name
    )

    if table is None:
        continue

    db2_columns = {
        NameNormalizer.normalize(column.name): column
        for column in table.columns
    }

    for field in record.fields:
        legacy_field_name = NameNormalizer.normalize(
            field.name
        )

        physical_column_name = self._find_matching_column(
            legacy_field_name=legacy_field_name,
            db2_columns=db2_columns
        )

        if not physical_column_name:
            continue

        host = self._host_variable(
            record_name=idms_record_name,
            column_name=physical_column_name,
            remove_suffix=True
        )

        field_map[
            legacy_field_name
        ] = {
            "host": host,
            "record": idms_record_name,
            "table": table.name,
            "column": physical_column_name,
            "legacy_field": legacy_field_name
        }

```

```

        suffix_removed_legacy = self._remove_record_suffix(
            legacy_field_name
        )

        field_map[
            suffix_removed_legacy
        ] = {
            "host": host,
            "record": idms_record_name,
            "table": table.name,
            "column": physical_column_name,
            "legacy_field": legacy_field_name
        }

    return field_map

def _build_calc_key_map(
    self,
    canonical_schema: CanonicalSchema,
    metadata: SchemaMetadata,
    record_table_map: dict[str, str]
) -> dict[str, dict[str, Any]]:

    calc_key_map: dict[str, dict[str, Any]] = {}

    for record in metadata.records:
        if not record.primary_key:
            continue

        idms_record_name = NameNormalizer.normalize(
            record.name
        )

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name
        )

        primary_key = NameNormalizer.normalize(
            record.primary_key

```

```

    )

    calc_key_map[
        idms_record_name
    ] = {
        "record": idms_record_name,
        "table": table_name,
        "key": primary_key,
        "host": self._host_variable(
            record_name=idms_record_name,
            column_name=primary_key,
            remove_suffix=True
        )
    }

for record in canonical_schema.records:
    if not record.primary_key:
        continue

    record_name = NameNormalizer.normalize(
        record.name
    )

    primary_key = NameNormalizer.normalize(
        record.primary_key
    )

    calc_key_map[
        record_name
    ] = {
        "record": record_name,
        "table": record_name,
        "key": primary_key,
        "host": self._host_variable(
            record_name=record_name,
            column_name=primary_key,
            remove_suffix=True
        )
    }

```

```

        return calc_key_map

def _build_relationships_payload(
    self,
    canonical_schema: CanonicalSchema,
    db2_table_lookup: dict[str, DB2Table]
) -> list[dict[str, Any]]:

    relationships_payload: list[dict[str, Any]] = []

    for relationship in canonical_schema.relationships:
        parent_record = NameNormalizer.normalize(
            relationship.parent_record
        )

        child_record = NameNormalizer.normalize(
            relationship.child_record
        )

        set_name = NameNormalizer.normalize(
            relationship.set_name
        )

        parent_table = db2_table_lookup.get(
            parent_record
        )

        child_table = db2_table_lookup.get(
            child_record
        )

        parent_key = None
        child_fk = None
        order_by = []

        if parent_table is not None:
            parent_key = parent_table.primary_key

        if (
            child_table is not None

```

```

        and parent_key is not None
    ):
        child_column_names = {
            NameNormalizer.normalize(column.name)
            for column in child_table.columns
        }

        if parent_key in child_column_names:
            child_fk = parent_key
            order_by = [
                child_fk
            ]

        relationships_payload.append(
            {
                "set_name": set_name,
                "parent_record": parent_record,
                "child_record": child_record,
                "parent_key": parent_key,
                "child_fk": child_fk,
                "cardinality": relationship.cardinality or "1:N",
                "order_by": order_by
            }
        )

    return relationships_payload

def _build_set_ordering_map(
    self,
    relationships_payload: list[dict[str, Any]]
) -> dict[str, dict[str, Any]]:

    set_ordering_map: dict[str, dict[str, Any]] = {}

    for relationship in relationships_payload:
        set_name = relationship.get(
            "set_name"
        )

        order_by = relationship.get(

```

```

        "order_by"
    ) or []

    if not set_name:
        continue

    set_ordering_map[
        set_name
    ] = {
        "order_by": order_by
    }

    return set_ordering_map

def _build_nullable_fk_map(
    self,
    db2_model: DB2Model
) -> dict[str, bool]:

    nullable_fk_map: dict[str, bool] = {}

    for table in db2_model.tables:
        column_lookup = {
            column.name: column
            for column in table.columns
        }

        for foreign_key in table.foreign_keys:
            fk_column = column_lookup.get(
                foreign_key.column_name
            )

            nullable_fk_map[
                f"{table.name}.{foreign_key.column_name}"
            ] = (
                fk_column.nullable
                if fk_column is not None
                else True
            )

```

```

        return nullable_fk_map

    def _build_date_part_map(
        self,
        metadata: SchemaMetadata,
        canonical_schema: CanonicalSchema,
        db2_table_lookup: dict[str, DB2Table],
        record_table_map: dict[str, str]
    ) -> dict[str, dict[str, Any]]:

        date_part_map: dict[str, dict[str, Any]] = {}

        for record in metadata.records:
            idms_record_name = NameNormalizer.normalize(
                record.name
            )

            table_name = record_table_map.get(
                idms_record_name,
                idms_record_name
            )

            table = db2_table_lookup.get(
                table_name
            )

            if table is None:
                continue

            date_columns = {
                NameNormalizer.normalize(column.name)
                for column in table.columns
                if column.datatype.upper() == "DATE"
            }

            for field in record.fields:
                legacy_field_name = NameNormalizer.normalize(
                    field.name
                )

```

```

        date_part = self._date_part_name(
            legacy_field_name
        )

        if date_part is None:
            continue

        physical_date_column = self._date_column_for_part(
            legacy_field_name=legacy_field_name,
            date_part=date_part
        )

        if physical_date_column not in date_columns:
            continue

        date_part_meta = self.DATE_PARTS[
            date_part
        ]

        date_part_map[
            legacy_field_name
        ] = {
            "host": self._host_variable(
                record_name=idms_record_name,
                column_name=physical_date_column,
                remove_suffix=True
            ),
            "table": table.name,
            "column": physical_date_column,
            "date_part": date_part,
            "substring_start": date_part_meta["substring_start"],
            "substring_length": date_part_meta["substring_length"]
        }

    return date_part_map

def _build_navigation_intent(
    self,
    relationships_payload: list[dict[str, Any]]
) -> dict[str, dict[str, Any]]:

```



```

navigation_intent: dict[str, dict[str, Any]] = {}

for relationship in relationships_payload:
    set_name = relationship.get(
        "set_name"
    )

    if not set_name:
        continue

    navigation_intent[
        set_name
    ] = {
        "access_pattern": "SET_NAVIGATION",
        "parent_record": relationship.get("parent_record"),
        "child_record": relationship.get("child_record"),
        "parent_key": relationship.get("parent_key"),
        "child_fk": relationship.get("child_fk"),
        "order_by": relationship.get("order_by") or []
    }

return navigation_intent

def _build_output_semantics(
    self
) -> dict[str, Any]:

    return {
        "generated_by": "idms-db2-modernizer-phase1",
        "usage": "IDMS retrieval COBOL to DB2 embedded SQL
conversion",
        "physical_types_source": "DB2_DDL",
        "logical_mapping_source": "PHASE1_CANONICAL_METADATA"
    }

def _build_validation_messages(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,

```

```

        metadata: SchemaMetadata,
        field_map: dict[str, dict[str, Any]]
    ) -> list[str]:

        messages: list[str] = []

        if not canonical_schema.records:
            messages.append(
                "No canonical records found."
            )

        if not db2_model.tables:
            messages.append(
                "No DB2 tables found."
            )

        if not metadata.records:
            messages.append(
                "No IDMS metadata records found."
            )

        if not field_map:
            messages.append(
                "No field_map entries generated."
            )

        decimal_warning_count = 0

        for table in db2_model.tables:
            for column in table.columns:
                datatype = column.datatype.upper()

                if datatype.startswith("DECIMAL(18,0)"):
                    decimal_warning_count += 1

        if decimal_warning_count:
            messages.append(
                f"{decimal_warning_count} DECIMAL(18,0) columns detected."
            )

            "

            "Review COMP-3 precision and scale extraction."

```

```

    )

    return messages

def _find_matching_column(
    self,
    legacy_field_name: str,
    db2_columns: dict[str, DB2Column]
) -> str | None:

    normalized = NameNormalizer.normalize(
        legacy_field_name
    )

    if normalized in db2_columns:
        return normalized

    suffix_removed = self._remove_record_suffix(
        normalized
    )

    for column_name in db2_columns:
        if self._remove_record_suffix(column_name) == suffix_removed:
            return column_name

    return None

def _date_part_name(
    self,
    field_name: str
) -> str | None:

    normalized = NameNormalizer.normalize(
        field_name
    )

    parts = normalized.split("_")

    if len(parts) < 3:
        return None

```

```

        for part in self.DATE_PARTS:
            if f"_{part}_" in normalized:
                return part

        return None

def _date_column_for_part(
    self,
    legacy_field_name: str,
    date_part: str
) -> str:

    normalized = NameNormalizer.normalize(
        legacy_field_name
    )

    return normalized.replace(
        f"_{date_part}_",
        "_DATE_"
    )

def _host_variable(
    self,
    record_name: str,
    column_name: str,
    remove_suffix: bool = True
) -> str:

    record = NameNormalizer.normalize(
        record_name
    ).replace(
        "_",
        "_"
    )

    column = NameNormalizer.normalize(
        column_name
    )

```

```

        if remove_suffix:
            column = self._remove_record_suffix(
                column
            )

        column = column.replace(
            "_",
            "_"
        )

        return f"HV-{record}-{column}"

def _remove_record_suffix(
    self,
    name: str
) -> str:

    normalized = NameNormalizer.normalize(
        name
    )

    parts = normalized.split("_")

    if (
        len(parts) > 1
        and parts[-1].isdigit()
        and len(parts[-1]) == 4
    ):
        return "_".join(
            parts[:-1]
        )

    return normalized

def _parse_db2_datatype(
    self,
    datatype: str
) -> tuple[str, int | None, int | None]:

    value = (

```

```

        datatype
        or "VARCHAR(255)"
    ).strip().upper()

    decimal_match = re.match(
        r"^(DECIMAL|NUMERIC) $(\d+),\s*(\d+) $$",
        value
    )

    if decimal_match:
        return (
            "DECIMAL",
            int(decimal_match.group(2)),
            int(decimal_match.group(3))
        )

    varchar_match = re.match(
        r"^(VARCHAR|CHAR) $(\d+) $$",
        value
    )

    if varchar_match:
        return (
            varchar_match.group(1),
            int(varchar_match.group(2)),
            None
        )

    if value in {
        "SMALLINT"
    }:
        return (
            "SMALLINT",
            4,
            0
        )

    if value in {
        "INTEGER",
        "INT"
    }:
```

```
    }:  
        return (  
            "INTEGER",  
            9,  
            0  
        )  
  
    if value == "BIGINT":  
        return (  
            "BIGINT",  
            18,  
            0  
        )  
  
    if value == "DATE":  
        return (  
            "DATE",  
            None,  
            None  
        )  
  
    if value == "TIMESTAMP":  
        return (  
            "TIMESTAMP",  
            None,  
            None  
        )  
  
    return (  
        value,  
        None,  
        None  
    )
```

src/idms_modernizer/services/relationship_resolver.py

```
from collections import defaultdict

from idms_modernizer.domain.relationship_models import (
    Relationship
)

class RelationshipResolver:

    def resolve(
        self,
        metadata
    ) -> list[Relationship]:

        owner_map = defaultdict(list)

        member_map = defaultdict(list)

        for record in metadata.records:

            for membership in record.set_memberships:

                if membership.role == "OWNER":

                    owner_map[
                        membership.set_name
                    ].append(
                        record.name
                    )

                elif membership.role == "MEMBER":

                    member_map[
                        membership.set_name
                    ].append(
                        record.name
                    )

        relationships = []
```



```
all_sets = (
    set(owner_map.keys())
    | set(member_map.keys())
)

for set_name in all_sets:

    owners = owner_map.get(
        set_name,
        []
    )

    members = member_map.get(
        set_name,
        []
    )

    for owner in owners:

        for member in members:

            relationships.append(
                Relationship(
                    set_name=set_name,
                    owner_record=owner,
                    member_record=member
                )
            )

return relationships
```

src/idms_modernizer/services/schema_picture_enricher.py

```
import re

from idms_modernizer.domain.schema_models import DataField

class SchemaPictureEnricher:
    """
    Enriches discovered fields with real PICTURE, datatype, length, and
    scale.

    Important:
    - FieldExtractor only discovers field names and usage.
    - This class fixes datatype, length, scale, and picture.
    - It searches the original section lines for each field.
    - It uses direct field-name matching instead of fragile row parsing.

    Handles:
    - X
    - XX / XXX
    - X(n)
    - 9
    - 9(n)
    - S9
    - S9(n)
    - S9(n)V99
    - S9(n)V9(n)
    - DATE group fields such as DISPLAY 97 8
    """

    FIELD_START_PATTERN = re.compile(
        r"^\s*(0[1-9]|[1-4][0-9]|88)\s+[A-Z][A-Z0-9-]*\b",
        re.IGNORECASE
    )

    USAGE_PATTERN = re.compile(
        r"\b(DISPLAY|COMP-3|COMP)\b",
        re.IGNORECASE
    )
```

```
VALID_PICTURE_PATTERN = re.compile(

r"^(S?9$\d+$V9$\d+$|S?9$\d+$V9+|S?9$\d+$|S?9|9$\d+$|9|X$\d+$|X+) $",

    re.IGNORECASE

)


R_X_N = re.compile(

    r"^X$(\d+) $$",

    re.IGNORECASE

)


R_X_INLINE = re.compile(

    r"^X+$",

    re.IGNORECASE

)


R_INTEGER = re.compile(

    r"^S?9(?:$(\d+) $)?$",

    re.IGNORECASE

)


R_DECIMAL_INLINE = re.compile(

    r"^S?9$(\d+) $V(9+) $",

    re.IGNORECASE

)


R_DECIMAL_PAREN = re.compile(

    r"^S?9$(\d+) $V9$(\d+) $$",

    re.IGNORECASE

)


SKIP_TOKENS = {

    "DISPLAY",

    "COMP",

    "COMP-3",

    "OCCURS",

    "DEPENDING",

    "ON",

    "TO",

    "TIMES",
```

```
        "VALUE",
        "VALUES",
        "THRU",
        "THROUGH",
        "COND"
    }

    TEXT_HINTS = {
        "NAME",
        "FIRST",
        "LAST",
        "STREET",
        "CITY",
        "STATE",
        "ZIP",
        "ADDRESS",
        "DESCRIPTION",
        "TITLE",
        "RELATION",
        "STATUS",
        "LEVEL",
        "CODE",
        "PHONE"
    }
```

```
    NUMERIC_HINTS = {
        "ID",
        "NUMBER",
        "YEAR",
        "MONTH",
        "DAY",
        "GRADE",
        "DAYS",
        "OPEN",
        "POSITIONS"
    }
```

```
def enrich(
    self,
    fields: list[DataField],
```

```

        lines: list[str]
    ) -> list[DataField]:

        print(
            "USING SCHEMA_PICTURE_ENRICHER VERSION
DIRECT-BLOCK-FALLBACK-2026-06-30"
        )

        cleaned_lines = [
            self._clean_line(line)
            for line in lines
            if line and self._clean_line(line)
        ]

        enriched_fields: list[DataField] = []

        for field in fields:
            block = self._find_field_block(
                field_name=field.name,
                lines=cleaned_lines
            )

            enriched_field = self._enrich_field_from_block(
                field=field,
                block=block
            )

            enriched_fields.append(
                enriched_field
            )

        return enriched_fields

    def _find_field_block(
        self,
        field_name: str,
        lines: list[str]
    ) -> str:

        target = field_name.upper()

```

```

for index, line in enumerate(lines):
    upper_line = line.upper()

    if target not in upper_line:
        continue

    if not self._looks_like_field_line(
        line=upper_line,
        field_name=target
    ):
        continue

    block_parts = [
        line
    ]

    lookahead_limit = min(
        len(lines),
        index + 8
    )

    for next_index in range(
        index + 1,
        lookahead_limit
    ):
        next_line = lines[next_index]

        if self.FIELD_START_PATTERN.match(next_line):
            break

        block_parts.append(
            next_line
        )

    return " ".join(block_parts)

return ""

def _looks_like_field_line(

```

```

        self,
        line: str,
        field_name: str
    ) -> bool:

        if field_name not in line:
            return False

        tokens = self._tokenize(line)

        for index, token in enumerate(tokens):
            cleaned = self._clean_token(token)

            if cleaned != field_name:
                continue

            if index == 0:
                return False

            previous_token = self._clean_token(
                tokens[index - 1]
            )

            if previous_token.isdigit():
                return True

        return False

    def _enrich_field_from_block(
        self,
        field: DataField,
        block: str
    ) -> DataField:

        if not block:
            return field

        usage = self._extract_usage(block)

        if usage is None:

```

```

        usage = field.datatype or "DISPLAY"

    picture = self._extract_best_picture(block)

    if picture is not None:
        datatype, length, scale = self._parse_picture(
            usage=usage,
            picture=picture
        )

        return DataField(
            name=field.name,
            datatype=datatype,
            length=length,
            scale=scale,
            picture=picture
        )

    date_field = self._infer_date_field(
        field=field,
        block=block
    )

    if date_field is not None:
        return date_field

    inferred = self._infer_from_length_and_name(
        field=field,
        block=block,
        usage=usage
    )

    if inferred is not None:
        return inferred

    return field

def _extract_usage(
    self,
    block: str

```



```

) -> str | None:

    match = self.USAGE_PATTERN.search(block)

    if not match:
        return None

    return match.group(1).upper()

def _extract_best_picture(
    self,
    block: str
) -> str | None:

    usage_match = self.USAGE_PATTERN.search(block)

    if usage_match:
        search_text = block[usage_match.end():]
    else:
        search_text = block

    tokens = self._tokenize(search_text)

    candidates: list[str] = []

    for index, raw_token in enumerate(tokens):
        token = self._clean_token(raw_token)

        if not token:
            continue

        if token in self.SKIP_TOKENS:
            continue

        if not self.VALID_PICTURE_PATTERN.fullmatch(token):
            continue

        if self._is_noise_picture(
            token=token,
            tokens=tokens,

```

```

        index=index
    ):
        continue

    candidates.append(token)

    if not candidates:
        return None

    return self._choose_best_picture(candidates)

def _is_noise_picture(
    self,
    token: str,
    tokens: list[str],
    index: int
) -> bool:

    if token not in {
        "9",
        "X",
        "S9"
    }:
        return False

    previous_token = ""

    if index > 0:
        previous_token = self._clean_token(
            tokens[index - 1]
        )

    next_token = ""

    if index + 1 < len(tokens):
        next_token = self._clean_token(
            tokens[index + 1]
        )

    if previous_token.isdigit():

```

```

        return True

    if next_token.isdigit():
        return True

    if self._has_stronger_picture(tokens):
        return True

    return False

def _has_stronger_picture(
    self,
    tokens: list[str]
) -> bool:

    for raw_token in tokens:
        token = self._clean_token(raw_token)

        if self.R_X_N.fullmatch(token):
            return True

        if self.R_DECIMAL_INLINE.fullmatch(token):
            return True

        if self.R_DECIMAL_PAREN.fullmatch(token):
            return True

        if self.R_INTEGER.fullmatch(token) and "(" in token:
            return True

    return False

def _choose_best_picture(
    self,
    candidates: list[str]
) -> str:

    for candidate in candidates:
        if self.R_DECIMAL_PAREN.fullmatch(candidate):
            return candidate

```

```

        for candidate in candidates:
            if self.R_DECIMAL_INLINE.fullmatch(candidate):
                return candidate

        for candidate in candidates:
            if self.R_X_N.fullmatch(candidate):
                return candidate

        for candidate in candidates:
            if self.R_INTEGER.fullmatch(candidate) and "(" in candidate:
                return candidate

        for candidate in candidates:
            if self.R_X_INLINE.fullmatch(candidate) and len(candidate) >
1:
                return candidate

        for candidate in candidates:
            if self.R_X_INLINE.fullmatch(candidate):
                return candidate

        for candidate in candidates:
            if self.R_INTEGER.fullmatch(candidate):
                return candidate

        return candidates[0]

def _infer_date_field(
    self,
    field: DataField,
    block: str
) -> DataField | None:

    if "DATE" not in field.name.upper():
        return None

    usage_match = self.USAGE_PATTERN.search(block)

    if not usage_match:

```

```

        return None

    after_usage = block[usage_match.end():]

    tokens = [
        self._clean_token(token)
        for token in self._tokenize(after_usage)
    ]

    numeric_tokens = [
        token
        for token in tokens
        if token.isdigit()
    ]

    if not numeric_tokens:
        return None

    if numeric_tokens[-1] != "8":
        return None

    return DataField(
        name=field.name,
        datatype="DATE",
        length=None,
        scale=None,
        picture=None
    )

def _infer_from_length_and_name(
    self,
    field: DataField,
    block: str,
    usage: str
) -> DataField | None:

    length = self._extract_last_numeric_token(block)

    if length is None:
        return None

```

```
field_name = field.name.upper()
usage = usage.upper()

if usage == "COMP-3":
    return DataField(
        name=field.name,
        datatype="DECIMAL",
        length=18,
        scale=0,
        picture=None
    )

if usage == "COMP":
    return DataField(
        name=field.name,
        datatype="INTEGER",
        length=length,
        scale=0,
        picture=None
    )

if any(hint in field_name for hint in self.TEXT_HINTS):
    if length == 1:
        return DataField(
            name=field.name,
            datatype="CHAR",
            length=1,
            scale=None,
            picture=None
        )

    return DataField(
        name=field.name,
        datatype="VARCHAR",
        length=length,
        scale=None,
        picture=None
    )
```

```

        if any(hint in field_name for hint in self.NUMERIC_HINTS):
            return DataField(
                name=field.name,
                datatype="INTEGER",
                length=length,
                scale=0,
                picture=None
            )

        if usage == "DISPLAY":
            return DataField(
                name=field.name,
                datatype="VARCHAR",
                length=length,
                scale=None,
                picture=None
            )

        return None

def _extract_last_numeric_token(
    self,
    text: str
) -> int | None:

    tokens = [
        self._clean_token(token)
        for token in self._tokenize(text)
    ]

    numeric_tokens = [
        token
        for token in tokens
        if token.isdigit()
    ]

    if not numeric_tokens:
        return None

    try:

```

```

        return int(numeric_tokens[-1])
    except ValueError:
        return None

def _parse_picture(
    self,
    usage: str,
    picture: str
) -> tuple[str, int | None, int | None]:

    usage = usage.upper()
    picture = picture.upper()

    x_n_match = self.R_X_N.fullmatch(picture)

    if x_n_match:
        length = int(x_n_match.group(1))

        if length == 1:
            return "CHAR", 1, None

        return "VARCHAR", length, None

    x_inline_match = self.R_X_INLINE.fullmatch(picture)

    if x_inline_match:
        length = len(picture)

        if length == 1:
            return "CHAR", 1, None

        return "VARCHAR", length, None

    decimal_match = self.R_DECIMAL_PAREN.fullmatch(picture)

    if decimal_match:
        integer_digits = int(decimal_match.group(1))
        decimal_digits = int(decimal_match.group(2))
        precision = integer_digits + decimal_digits

```



```

        return "DECIMAL", precision, decimal_digits

decimal_match = self.R_DECIMAL_INLINE.fullmatch(picture)

if decimal_match:
    integer_digits = int(decimal_match.group(1))
    decimal_digits = len(decimal_match.group(2))
    precision = integer_digits + decimal_digits

    return "DECIMAL", precision, decimal_digits

integer_match = self.R_INTEGER.fullmatch(picture)

if integer_match:
    digits = int(integer_match.group(1) or 1)

    if usage == "COMP-3":
        return "DECIMAL", digits, 0

    return "INTEGER", digits, 0

return "VARCHAR", 255, None

def _tokenize(
    self,
    text: str
) -> list[str]:

    text = text.replace(",", " ")
    text = text.replace(";", " ")
    text = text.replace(":", " ")

    text = re.sub(r"\s+", " ", text)

    return [
        token
        for token in text.strip().split(" ")
        if token
    ]

```

```

def _clean_token(
    self,
    token: str
) -> str:

    token = token.strip()
    token = token.strip(",")
    token = token.strip(";")
    token = token.strip(":")
    token = token.strip(".")
    token = token.strip("'")
    token = token.strip('"')

    return token.upper()

def _clean_line(
    self,
    line: str | None
) -> str:

    if not line:
        return ""

    cleaned = str(line).strip()

    cleaned = re.sub(r"<[^>]+>", " ", cleaned)
    cleaned = cleaned.replace("&nbsp;", " ")
    cleaned = cleaned.replace("\t", " ")
    cleaned = cleaned.replace("|", " ")
    cleaned = re.sub(r"\s+", " ", cleaned)

    return cleaned.strip()

```

src/idms_modernizer/services/validation_service.py

```
class ValidationService:

    def validate_metadata(
        self,
        metadata
    ) -> list[str]:

        errors = []

        record_names = {
            record.name
            for record in metadata.records
        }

        for rel in metadata.relationships:

            if rel.owner_record not in record_names:

                errors.append(
                    f"Missing Owner Record: {rel.owner_record}"
                )

            if rel.member_record not in record_names:

                errors.append(
                    f"Missing Member Record: {rel.member_record}"
                )

        return errors
```

src/idms_modernizer/ui/[app.py](#)

```
from pathlib import Path
from typing import Any

import streamlit as st

from idms_modernizer.core.config import settings
from idms_modernizer.core.utils import ensure_directory
from idms_modernizer.core.constants import (
    ER_DIAGRAM_FILE,
    PHASE2_METADATA_FILE
)

from idms_modernizer.parsers.cobol_pdf_extractor import CobolPdfExtractor

from idms_modernizer.services.metadata_service import MetadataService
from idms_modernizer.services.canonical_schema_builder import (
    CanonicalSchemaBuilder
)
from idms_modernizer.services.canonical_reconciler import (
    CanonicalReconciler
)
from idms_modernizer.services.db2_model_builder import DB2ModelBuilder
from idms_modernizer.services.phase2_metadata_generator import (
    Phase2MetadataGenerator
)
from idms_modernizer.services.phase2_conversion_bridge import (
    Phase2ConversionBridge
)
from idms_modernizer.services.er_diagram_generator import (
    generate_er_diagram
)

from idms_modernizer.generators.ddl_generator import DDLGenerator

def initialize_directories() -> None:
    ensure_directory(settings.input_dir)
    ensure_directory(settings.output_dir)
```

```

ensure_directory(settings.temp_dir)

def initialize_session_state() -> None:
    defaults = {
        "generated": False,
        "schema_path": "",
        "cobol_pdf_path": "",
        "cobol_text": "",
        "converted_cobol": "",
        "ddl_text": "",
        "canonical_json": "",
        "phase2_metadata_json": "",
        "metadata": None,
        "canonical_schema": None,
        "db2_model": None,
        "relationships": [],
        "all_sets": [],
        "record_count": 0,
        "set_count": 0,
        "relationship_count": 0,
        "validation_messages": [],
        "generation_warnings": [],
        "er_png_path": "",
        "er_image_bytes": None,
        "output_files": {},
    }

    for key, value in defaults.items():
        if key not in st.session_state:
            st.session_state[key] = value

def reset_outputs() -> None:
    st.session_state.generated = False
    st.session_state.schema_path = ""
    st.session_state.cobol_pdf_path = ""
    st.session_state.cobol_text = ""
    st.session_state.converted_cobol = ""
    st.session_state.ddl_text = ""

```

```

st.session_state.canonical_json = ""
st.session_state.phase2_metadata_json = ""
st.session_state.metadata = None
st.session_state.canonical_schema = None
st.session_state.db2_model = None
st.session_state.relationships = []
st.session_state.all_sets = []
st.session_state.record_count = 0
st.session_state.set_count = 0
st.session_state.relationship_count = 0
st.session_state.validation_messages = []
st.session_state.generation_warnings = []
st.session_state.er_png_path = ""
st.session_state.er_image_bytes = None
st.session_state.output_files = {}

def save_uploaded_file(
    uploaded_file,
    target_folder: str
) -> str:
    target_dir = Path(target_folder)
    target_dir.mkdir(
        parents=True,
        exist_ok=True
    )

    target_path = (
        target_dir
        / uploaded_file.name
    )

    with open(
        target_path,
        "wb"
    ) as file:
        file.write(
            uploaded_file.getbuffer()
        )

```

```

        return str(target_path)

def write_output_file(
    output_dir: Path,
    file_name: str,
    content: str,
    encoding: str = "utf-8"
) -> Path:
    output_dir.mkdir(
        parents=True,
        exist_ok=True
    )

    path = (
        output_dir
        / file_name
    )

    path.write_text(
        content,
        encoding=encoding
    )

    return path

def build_sets_summary(
    metadata
) -> list[str]:
    all_sets = set()

    for record in metadata.records:
        for membership in record.set_memberships:
            all_sets.add(
                membership.set_name
            )

    return sorted(all_sets)

```

```

def build_relationship_rows(
    relationships
) -> list[dict[str, str]]:
    rows = []

    for relationship in relationships:
        rows.append(
            {
                "Owner Record": relationship.owner_record,
                "Member Record": relationship.member_record,
                "Set Name": relationship.set_name,
            }
        )

    return rows


def generate_outputs(
    schema_pdf,
    cobol_pdf,
    target_program: str | None
) -> None:
    reset_outputs()

    warnings = []

    schema_path = save_uploaded_file(
        schema_pdf,
        settings.input_dir
    )

    cobol_pdf_path = save_uploaded_file(
        cobol_pdf,
        settings.input_dir
    )

    metadata_service = MetadataService()

    metadata = metadata_service.build_metadata(

```



```
        schema_path
    )

    canonical_builder = CanonicalSchemaBuilder()

    canonical_schema = canonical_builder.build(
        metadata
    )

    reconciler = CanonicalReconciler()

    canonical_schema = reconciler.reconcile(
        canonical_schema=canonical_schema,
        metadata=metadata
    )

    db2_builder = DB2ModelBuilder()

    db2_model = db2_builder.build(
        canonical_schema
    )

    ddl_generator = DDLGenerator()

    ddl_text = ddl_generator.generate(
        db2_model
    )

    phase2_metadata_generator = Phase2MetadataGenerator()

    phase2_metadata_json = phase2_metadata_generator.generate_json(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata
    )

    canonical_json = canonical_schema.model_dump_json(
        indent=2
    )
```

```

cobol_extractor = CobolPdfExtractor()

cobol_text = cobol_extractor.extract_text(
    cobol_pdf_path
)

converted_cobol = ""
validation_messages = []

try:
    phase2_bridge = Phase2ConversionBridge()

    converted_cobol, validation_messages = phase2_bridge.convert(
        cobol_text=cobol_text,
        canonical_json=canonical_json,
        phase2_metadata_json=phase2_metadata_json,
        ddl_text=ddl_text,
        idms_schema_text=None,
        relationship_overrides_json=None,
        target_program=target_program,
        use_ddl=True,
        use_idms_schema=False,
        use_overrides=False
    )

except Exception as ex:
    converted_cobol = ""
    validation_messages = [
        f"Phase 2 COBOL conversion failed: {ex}"
    ]

relationships = metadata.relationships

all_sets = build_sets_summary(
    metadata
)

output_dir = Path(
    settings.output_dir
)

```

```
output_files = {}

output_files["ddl"] = write_output_file(
    output_dir=output_dir,
    file_name="db2_ddl.sql",
    content=ddl_text
)

output_files["canonical"] = write_output_file(
    output_dir=output_dir,
    file_name="canonical_model.json",
    content=canonical_json
)

output_files["phase2_metadata"] = write_output_file(
    output_dir=output_dir,
    file_name=PHASE2_METADATA_FILE,
    content=phase2_metadata_json
)

output_files["converted_cobol"] = write_output_file(
    output_dir=output_dir,
    file_name="converted_db2_cobol.cbl",
    content=converted_cobol
)

er_png_path = ""
er_image_bytes = None

try:
    er_path = (
        output_dir
        / ER_DIAGRAM_FILE
    )

    er_png_path = generate_er_diagram(
        metadata,
        str(er_path)
    )
```

```

        with open(
            er_png_path,
            "rb"
        ) as image_file:
            er_image_bytes = image_file.read()

        output_files["er_diagram"] = Path(er_png_path)

    except Exception as ex:
        warnings.append(
            "ER diagram was not generated. "
            "Graphviz may not be installed or configured. "
            f"Details: {ex}"
        )

    st.session_state.generated = True
    st.session_state.schema_path = schema_path
    st.session_state.cobol_pdf_path = cobol_pdf_path
    st.session_state.metadata = metadata
    st.session_state.canonical_schema = canonical_schema
    st.session_state.db2_model = db2_model
    st.session_state.ddl_text = ddl_text
    st.session_state.canonical_json = canonical_json
    st.session_state.phase2_metadata_json = phase2_metadata_json
    st.session_state.cobol_text = cobol_text
    st.session_state.converted_cobol = converted_cobol
    st.session_state.relationships = relationships
    st.session_state.all_sets = all_sets
    st.session_state.record_count = len(metadata.records)
    st.session_state.set_count = len(all_sets)
    st.session_state.relationship_count = len(relationships)
    st.session_state.validation_messages = validation_messages
    st.session_state.generation_warnings = warnings
    st.session_state.er_png_path = er_png_path
    st.session_state.er_image_bytes = er_image_bytes
    st.session_state.output_files = output_files

def render_download_button(

```

```

        label: str,
        path,
        file_name: str,
        mime: str
    ) -> None:
        if not path:
            st.button(
                label,
                disabled=True,
                use_container_width=True
            )
            return

        path = Path(path)

        if not path.exists():
            st.button(
                label,
                disabled=True,
                use_container_width=True
            )
            return

        with open(
            path,
            "rb"
        ) as file:
            st.download_button(
                label=label,
                data=file,
                file_name=file_name,
                mime=mime,
                use_container_width=True
            )

def render_download_buttons() -> None:
    if not st.session_state.generated:
        return

```

```
st.markdown("### Download Outputs")

output_files = st.session_state.output_files

col1, col2, col3, col4, col5 = st.columns(5)

with col1:
    render_download_button(
        label="DB2 DDL",
        path=output_files.get("ddl"),
        file_name="db2_ddl.sql",
        mime="text/sql"
    )

with col2:
    render_download_button(
        label="Phase Metadata",
        path=output_files.get("phase2_metadata"),
        file_name=PHASE2_METADATA_FILE,
        mime="application/json"
    )

with col3:
    render_download_button(
        label="Canonical JSON",
        path=output_files.get("canonical"),
        file_name="canonical_model.json",
        mime="application/json"
    )

with col4:
    if st.session_state.er_image_bytes:
        st.download_button(
            label="ER Diagram",
            data=st.session_state.er_image_bytes,
            file_name=ER_DIAGRAM_FILE,
            mime="image/png",
            use_container_width=True
        )
    else:
```

```

        st.button(
            "ER Diagram",
            disabled=True,
            use_container_width=True
        )

    with col5:
        render_download_button(
            label="DB2 COBOL",
            path=output_files.get("converted_cobol"),
            file_name="converted_db2_cobol.cbl",
            mime="text/plain"
        )

def render_status_panel() -> None:
    if not st.session_state.generated:
        return

    if st.session_state.generation_warnings:
        with st.expander(
            "Generation Warnings",
            expanded=False
        ):
            for warning in st.session_state.generation_warnings:
                st.warning(
                    warning
                )

    if st.session_state.validation_messages:
        with st.expander(
            "COBOL Conversion Validation Messages",
            expanded=True
        ):
            for message in st.session_state.validation_messages:
                st.write(
                    f"- {message}"
                )
    else:
        st.success(

```

```

        "COBOL conversion validation passed."
    )

def render_main_tab() -> None:
    st.markdown("## Generate Modernization Outputs")

    st.info(
        "Upload the Schema Listing PDF and COBOL Code PDF to generate DB2 DDL, metadata, ER diagram, and converted DB2 COBOL."
    )

    col1, col2 = st.columns(2)

    with col1:
        schema_pdf = st.file_uploader(
            "Schema Listing PDF",
            type=["pdf"],
            key="schema_listing_pdf"
        )

    with col2:
        cobol_pdf = st.file_uploader(
            "COBOL Code PDF",
            type=["pdf"],
            key="cobol_code_pdf"
        )

    target_program = st.text_input(
        "Target PROGRAM-ID",
        value="",
        placeholder="Optional, e.g. DEPTDB2"
    )

    generate_clicked = st.button(
        "Generate Outputs",
        type="primary",
        use_container_width=True
    )

```



```

if generate_clicked:
    if schema_pdf is None:
        st.error(
            "Please upload the Schema Listing PDF."
        )
        st.stop()

    if cobol_pdf is None:
        st.error(
            "Please upload the COBOL Code PDF."
        )
        st.stop()

    with st.spinner(
        "Processing schema, generating DB2 artifacts, and converting
        COBOL..."
    ):
        try:
            generate_outputs(
                schema_pdf=schema_pdf,
                cobol_pdf=cobol_pdf,
                target_program=(
                    target_program.strip()
                    if target_program.strip()
                    else None
                )
            )

            st.success(
                "Generation completed successfully."
            )

        except Exception as ex:
            st.error(
                "Generation failed."
            )
            st.exception(
                ex
            )
            st.stop()

```

```

if st.session_state.generated:
    st.markdown("### Output Status")

    col_a, col_b = st.columns(2)

    with col_a:
        st.write(
            f"Schema PDF: `{st.session_state.schema_path}`"
        )

    with col_b:
        st.write(
            f"COBOL PDF: `{st.session_state.cobol_pdf_path}`"
        )

    render_status_panel()
    render_download_buttons()

def render_metadata_overview_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown("## Metadata Overview")

    st.markdown("### Summary")

    col1, col2, col3 = st.columns(3)

    with col1:
        st.metric(
            "Records",
            st.session_state.record_count
        )

    with col2:

```

```

        st.metric(
            "Sets",
            st.session_state.set_count
        )

    with col3:
        st.metric(
            "Relationships",
            st.session_state.relationship_count
        )

    st.divider()

    st.markdown("### Detected Sets")

    st.json(
        st.session_state.all_sets
    )

    st.divider()

    st.markdown("### Detected Relationships")

    relationship_rows = build_relationship_rows(
        st.session_state.relationships
    )

    if relationship_rows:
        st.dataframe(
            relationship_rows,
            use_container_width=True,
            hide_index=True
        )
    else:
        st.info(
            "No relationships detected."
        )

    st.divider()

```

```

st.markdown("### Records & Fields")

metadata = st.session_state.metadata

record_names = [
    record.name
    for record in metadata.records
]

selected_record = st.selectbox(
    "Select a record",
    options=record_names
)

selected_record_obj = None

for record in metadata.records:
    if record.name == selected_record:
        selected_record_obj = record
        break

if selected_record_obj is not None:
    st.write(
        f"Fields found: `{len(selected_record_obj.fields)}`"
    )

    field_rows = []

    for field in selected_record_obj.fields:
        field_rows.append(
            {
                "Field": field.name,
                "Datatype": field.datatype,
                "Length": field.length,
                "Scale": field.scale,
                "Picture": field.picture,
            }
        )

    st.dataframe(

```

```

        field_rows,
        use_container_width=True,
        hide_index=True
    )

    with st.expander(
        "View all records and fields as JSON",
        expanded=False
    ):
        st.json(
            [
                {
                    "record": record.name,
                    "fields": [
                        field.model_dump()
                        for field in record.fields
                    ]
                }
                for record in metadata.records
            ]
        )

def render_ddl_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown("### DB2 DDL Preview")

    st.code(
        st.session_state.ddl_text,
        language="sql"
    )

def render_phase2_metadata_tab() -> None:
    if not st.session_state.generated:

```

```

        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown("## Phase 2 Metadata Preview")

    st.code(
        st.session_state.phase2_metadata_json,
        language="json"
    )

def render_er_diagram_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown("## ER Diagram")

    if st.session_state.er_image_bytes:
        st.image(
            st.session_state.er_image_bytes
        )
    else:
        st.warning(
            "ER diagram was not generated. Graphviz may not be installed
or configured."
        )

def render_converted_cobol_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

```

```

st.markdown("## Converted DB2 COBOL")

if not st.session_state.converted_cobol:
    st.warning(
        "Converted COBOL output is empty or conversion failed."
    )

st.text_area(
    "Final DB2 COBOL Code",
    value=st.session_state.converted_cobol,
    height=750
)

def render_validation_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown("## Validation & Warnings")

    has_generation_warnings = bool(
        st.session_state.generation_warnings
    )

    has_validation_messages = bool(
        st.session_state.validation_messages
    )

    if not has_generation_warnings and not has_validation_messages:
        st.success(
            "No warnings or validation messages."
        )
        return

    if has_generation_warnings:
        st.markdown("### Generation Warnings")

```

```

        for warning in st.session_state.generation_warnings:
            st.warning(
                warning
            )

    if has_validation_messages:
        st.markdown("### COBOL Conversion Validation Messages")

        for message in st.session_state.validation_messages:
            st.write(
                f"- {message}"
            )

def main() -> None:
    initialize_directories()
    initialize_session_state()

    st.set_page_config(
        page_title="IDMS DB2 Modernizer",
        layout="wide"
    )

    st.title(
        "IDMS → DB2 Modernizer"
    )

    st.caption(
        "Generate DB2 DDL, Phase 2 metadata, canonical JSON, ER diagram,  
and converted DB2 COBOL from uploaded PDFs."
    )

    tabs = st.tabs(
        [
            "Main",
            "Metadata Overview",
            "DB2 DDL",
            "Phase 2 Metadata",
            "ER Diagram",
            "Converted DB2 COBOL",

```



```
        "Validation"
    ]
)

with tabs[0]:
    render_main_tab()

with tabs[1]:
    render_metadata_overview_tab()

with tabs[2]:
    render_ddl_tab()

with tabs[3]:
    render_phase2_metadata_tab()

with tabs[4]:
    render_er_diagram_tab()

with tabs[5]:
    render_converted_cobol_tab()

with tabs[6]:
    render_validation_tab()

if __name__ == "__main__":
    main()
```


src\idms_db2_converter\generators\cobol_builder.py

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Protocol

class CobolNode(Protocol):

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:
        ...

def pad(
    indent: int
) -> str:

    return " " * indent

def ensure_period(
    value: str
) -> str:

    value = value.rstrip()

    if value.endswith("."):
        return value

    return value + "."

@dataclass
class PreRenderedBlock:
```

```

lines: list[str]

def render(
    self,
    indent: int = 7,
    final: bool = False
) -> list[str]:

    result = list(
        self.lines
    )

    if final and result:
        result[-1] = ensure_period(
            result[-1]
        )

    return result

```

@dataclass

class RawLines:

```

lines: list[str]

def render(
    self,
    indent: int = 7,
    final: bool = False
) -> list[str]:

    result: list[str] = []

    for line in self.lines:
        if line.strip():
            result.append(
                pad(indent) + line.strip()
            )
        else:
            result.append("")

```

```
    if final and result:
        result[-1] = ensure_period(
            result[-1]
        )

    return result
```

```
@dataclass
```

```
class Move:
```

```
    source: str
```

```
    target: str
```

```
    def render(
```

```
        self,
```

```
        indent: int = 7,
```

```
        final: bool = False
```

```
) -> list[str]:
```

```
    return [
```

```
        f"{pad(indent)}MOVE {self.source} TO {self.target}."
```

```
    ]
```

```
@dataclass
```

```
class Perform:
```

```
    paragraph: str
```

```
    def render(
```

```
        self,
```

```
        indent: int = 7,
```

```
        final: bool = False
```

```
) -> list[str]:
```

```
    return [
```

```
        f"{pad(indent)}PERFORM {self.paragraph}."
```

```
    ]
```

```

@dataclass
class Continue:

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        return [
            f"{pad(indent)}CONTINUE."
        ]

```

```

@dataclass
class ReadAtEndMove:

    file_name: str
    source: str
    target: str

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        source = self.source

        if (
            len(source) == 1
            and not source.startswith("'")
            and not source.endswith("'")
        ):
            source = f"'{source}'"

        return [

```

```
        f"{pad(indent)}READ {self.file_name} AT END MOVE {source} TO  
{self.target}."  
    ]
```

```
@dataclass
```

```
class Write:
```

```
    record: str
```

```
    source: str | None = None
```

```
    def render(  
        self,  
        indent: int = 7,  
        final: bool = False  
    ) -> list[str]:
```

```
        if self.source:
```

```
            return [  
                f"{pad(indent)}WRITE {self.record} FROM {self.source}."  
            ]
```

```
        return [  
            f"{pad(indent)}WRITE {self.record}."  
        ]
```

```
@dataclass
```

```
class ExecSql:
```

```
    sql: str
```

```
    def render(  
        self,  
        indent: int = 7,  
        final: bool = False  
    ) -> list[str]:
```

```
        normalized_lines = self._normalize_sql_lines(  
            self.sql
```

```

    )

    result: list[str] = []

    for line in normalized_lines:
        if line.strip():
            result.append(
                pad(indent) + line.rstrip()
            )
        else:
            result.append("")

    if final and result:
        result[-1] = ensure_period(
            result[-1]
        )

    return result

def _normalize_sql_lines(
    self,
    sql: str
) -> list[str]:

    raw_lines = sql.splitlines()

    stripped_lines = [
        line.rstrip()
        for line in raw_lines
        if line.strip()
    ]

    if not stripped_lines:
        return []

    min_indent = min(
        len(line) - len(line.lstrip(" "))
        for line in stripped_lines
    )

```



```

normalized: list[str] = []

for line in raw_lines:
    if line.strip():
        normalized.append(
            line[min_indent:].rstrip()
        )
    else:
        normalized.append("")

return normalized

```

```
@dataclass
```

```
class IfBlock:
```

```

    condition: str
    then_body: list[CobolNode] = field(
        default_factory=list
    )
    else_body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{pad(indent)}IF {self.condition}"
        ]

        result.extend(
            render_nodes(
                self.then_body,
                indent + 3
            )
        )

```

```

    if self.else_body:
        result.append(
            f"{pad(indent)}ELSE"
        )

        result.extend(
            render_nodes(
                self.else_body,
                indent + 3
            )
        )

    terminator = "END-IF."

    if not final:
        terminator = "END-IF"

    result.append(
        f"{pad(indent)}{terminator}"
    )

    return result

```

```

def with_prefix(
    self,
    prefix: CobolNode
) -> PrefixedBlock:

    return PrefixedBlock(
        prefix=prefix,
        block=self
    )

```

@dataclass

class PrefixedBlock:

```

    prefix: CobolNode
    block: CobolNode

```

```

def render(
    self,
    indent: int = 7,
    final: bool = False
) -> list[str]:

    result: list[str] = []

    result.extend(
        self.prefix.render(
            indent=indent
        )
    )

    result.append("")

    result.extend(
        self.block.render(
            indent=indent,
            final=final
        )
    )

    return result

```

```

@dataclass
class PerformUntil:

    condition: str
    body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

```

```

    result: list[str] = [
        f"{pad(indent)}PERFORM UNTIL {self.condition}"
    ]

    result.extend(
        render_nodes(
            self.body,
            indent + 3
        )
    )

    terminator = "END-PERFORM."

    if not final:
        terminator = "END-PERFORM"

    result.append(
        f"{pad(indent)}{terminator}"
    )

    return result

```

```
@dataclass
```

```
class Paragraph:
```

```

    name: str
    body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{self.name}."

```

```

    ]

    if self.body:
        result.extend(
            render_nodes(
                self.body,
                indent,
                terminate_last=True
            )
        )

    return result

def render_nodes(
    nodes: list[CobolNode],
    indent: int = 7,
    terminate_last: bool = False
) -> list[str]:

    result: list[str] = []

    for index, node in enumerate(nodes):
        is_last = (
            terminate_last
            and index == len(nodes) - 1
        )

        result.extend(
            node.render(
                indent=indent,
                final=is_last
            )
        )

    return result

def render_paragraph(
    name: str,

```

```
        nodes: list[CobolNode]
    ) -> str:

        paragraph = Paragraph(
            name=name,
            body=nodes
        )

        return "\n".join(
            paragraph.render()
        )
```

src\idms_db2_converter\generators\cursors.py

```
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

class CursorGenerator:
    """
    Generates DECLARE CURSOR statements for IDMS set navigation.

    This version selects all child columns from the final schema so the
    fetch paragraph can populate all host variables needed later by field
    rewriting.
    """

    def generate(
        self,
        schema: SchemaModel,
        used_sets: list[str]
    ) -> str:

        print("USING CURSOR_GENERATOR VERSION ALL-COLUMNS-2026-07-01")

        blocks: list[str] = []
        seen: set[str] = set()

        for set_name in used_sets:
            set_name = set_name.upper()

            if set_name in seen:
                continue

            seen.add(set_name)

            relationship = schema.relationships.get(set_name)

            if not relationship:
```

```

        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    if not relationship.parent_key or not relationship.child_fk:
        raise ConversionError(
            f"Set {set_name} missing parent_key or child_fk."
        )

    child_record = schema.records.get(
        self._physical_record_name(
            schema,
            relationship.child_record
        )
    )

    if not child_record:
        raise ConversionError(
            f"Child record {relationship.child_record} missing."
        )

    columns = list(child_record.fields.keys())

    parent_host = Naming.hv(
        relationship.parent_record,
        relationship.parent_key
    )

    lines = [
        "        EXEC SQL",
        f"        DECLARE {Naming.cursor(set_name)} CURSOR
FOR"
    ]

    lines.extend(
        comma_block(
            items=columns,
            first_prefix="        SELECT ",
            next_prefix="        "
        )
    )

```



```

        )

        lines.append(
            f"                                FROM {self._table_name(schema,
relationship.child_record)}"
        )

        lines.append(
            f"                                WHERE {relationship.child_fk} =
:{parent_host}"
        )

        if relationship.order_by:
            lines.append(
                f"                                ORDER BY {'',
'.join(relationship.order_by)}"
            )

        lines.append(
            "                                END-EXEC."
        )

        blocks.append(
            "\n".join(lines)
        )

    return "\n\n".join(blocks)

def _table_name(
    self,
    schema: SchemaModel,
    record_name: str
) -> str:

    mapped = schema.record_table_map.get(
        record_name
    )

    if mapped:
        return mapped

```

```

        normalized = record_name.replace(
            "-",
            " "
        ).upper()

        mapped = schema.record_table_map.get(
            normalized
        )

        if mapped:
            return mapped

        return normalized

def _physical_record_name(
    self,
    schema: SchemaModel,
    logical_record_name: str
) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = schema.record_table_map.get(
        logical_record_name
    )

    if mapped and mapped in schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        " "
    )

    mapped = schema.record_table_map.get(
        normalized
    )

    if mapped and mapped in schema.records:

```

```
        return mapped

    if logical_record_name in schema.records:
        return logical_record_name

    if normalized in schema.records:
        return normalized

    return logical_record_name
```

src\idms_db2_converter\generators\fetch_paragraphs.py

```
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

class FetchParagraphGenerator:
    """
    Generates complete FETCH paragraphs for cursor-based IDMS set walking.

    Guarantees:
    - FETCH statement has END-EXEC.
    - SQLCODE handling is emitted after FETCH.
    - INTO list matches every selected column in the cursor.
    """

    def __init__(
        self,
        schema: SchemaModel
    ):
        self.schema = schema

    def paragraph_name(
        self,
        set_name: str
    ) -> str:

        return f"FETCH-{set_name}".upper()

    def generate(
        self,
        used_sets: list[str]
    ) -> str:

        print(
            "USING FETCH_PARAGRAPH_GENERATOR VERSION
COMPLETE-SQLCODE-2026-07-01"
        )

        paragraphs: list[str] = []
```

```

seen: set[str] = set()

for set_name in used_sets:
    set_name = set_name.upper()

    if set_name in seen:
        continue

    seen.add(set_name)

    if set_name not in self.schema.relationships:
        continue

    paragraph = self._generate_one(
        set_name
    )

    if paragraph.strip():
        paragraphs.append(
            paragraph
        )

return "\n\n".join(
    paragraphs
)

def _generate_one(
    self,
    set_name: str
) -> str:

    relationship = self.schema.relationships[
        set_name
    ]

    child_record_name = self._physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(

```

```

        child_record_name
    )

    if child_record is None:
        return ""

    into_items: list[str] = []

    for column_name, column in child_record.fields.items():
        host = (
            f":{Naming.hv(relationship.child_record, column_name)}"
        )

        if column.nullable:
            host += (
                f" :{Naming.ni(relationship.child_record,
column_name)}"
            )

        into_items.append(
            host
        )

    lines: list[str] = [
        f"{self.paragraph_name(set_name)}.",
        "        EXEC SQL",
        f"            FETCH {Naming.cursor(set_name)}"
    ]

    for index, item in enumerate(into_items):
        suffix = "," if index < len(into_items) - 1 else ""

        if index == 0:
            lines.append(
                f"            INTO {item}{suffix}"
            )
        else:
            lines.append(
                f"                {item}{suffix}"
            )

```

```

        lines.extend(
            [
                "            END-EXEC.",
                "",
                "            IF SQLCODE NOT = 0 AND SQLCODE NOT = 100",
                "                PERFORM SQL-ERROR",
                "            END-IF."
            ]
        )

    return "\n".join(
        lines
    )

def _physical_record_name(
    self,
    logical_record_name: str
) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(
        logical_record_name
    )

    if mapped and mapped in self.schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        " "
    )

    mapped = self.schema.record_table_map.get(
        normalized
    )

    if mapped and mapped in self.schema.records:
        return mapped

```

```
    if logical_record_name in self.schema.records:
        return logical_record_name

    if normalized in self.schema.records:
        return normalized

    return logical_record_name
```


src\idms_db2_converter\generators\formatting.py

```
def comma_block(
    items: list[str],
    first_prefix: str = "",
    next_prefix: str = "",
) -> list[str]:
    """
    Format a list of SQL items as comma-separated lines.

    Example:
        comma_block(
            items=["EMP_ID", "EMP_NAME", "DEPT_ID"],
            first_prefix="SELECT ",
            next_prefix="      "
        )

    Returns:
        [
            "SELECT EMP_ID,",
            "      EMP_NAME,",
            "      DEPT_ID"
        ]
    """

    if not items:
        return []

    lines: list[str] = []

    for index, item in enumerate(items):
        prefix = first_prefix if index == 0 else next_prefix

        suffix = "," if index < len(items) - 1 else ""

        lines.append(
            f"{prefix}{item}{suffix}"
        )

    return lines
```

src\idms_db2_converter\generators\host_variables.py

```
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

class HostVariableGenerator:
    """
    Generates COBOL host variables.

    This version generates all columns for each used record from the final
    Phase 2 schema. This is required because the converted SQL and report
    field moves may need fields that were not part of the original cursor
    key.
    """

    def generate(
        self,
        schema: SchemaModel,
        used_records: list[str]
    ) -> str:

        print("USING HOST_VARIABLE_GENERATOR VERSION
ALL-COLUMNS-2026-07-01")

        lines: list[str] = [
            "      EXEC SQL",
            "          INCLUDE SQLCA",
            "      END-EXEC.",
            "",
            "      EXEC SQL",
            "          BEGIN DECLARE SECTION",
            "      END-EXEC.",
            ""
        ]

        null_indicators: list[str] = []
        emitted_null_indicators: set[str] = set()
```

```

physical_records = self._resolve_used_records(
    schema=schema,
    used_records=used_records
)

for logical_record_name, physical_record_name in physical_records:
    if physical_record_name not in schema.records:
        continue

    record = schema.records[physical_record_name]

    lines.append(
        f"      01 HV-{logical_record_name}."
    )

    for column_name, column in record.fields.items():
        host_variable = Naming.hv(
            logical_record_name,
            column_name
        )

        lines.extend(
            self.column_to_cobol(
                hv=host_variable,
                datatype=column.datatype,
                length=column.length,
                scale=column.scale
            )
        )

        if column.nullable:
            null_indicator = Naming.ni(
                logical_record_name,
                column_name
            )

            if null_indicator not in emitted_null_indicators:
                emitted_null_indicators.add(null_indicator)

            null_indicators.append(

```

```

                                f"          01 {null_indicator:<45} PIC S9(4)
COMP."

                                )

                                lines.append("")

                                if null_indicators:
                                    lines.extend(null_indicators)
                                    lines.append("")

                                lines.extend(
                                    [
                                        "          EXEC SQL",
                                        "          END DECLARE SECTION",
                                        "          END-EXEC."
                                    ]
                                )

                                return "\n".join(lines)

def _resolve_used_records(
    self,
    schema: SchemaModel,
    used_records: list[str]
) -> list[tuple[str, str]]:

    resolved: list[tuple[str, str]] = []
    seen: set[tuple[str, str]] = set()

    for logical_record_name in sorted(set(used_records)):
        logical_record_name = logical_record_name.upper()

        physical_record_name = self.physical_record_name(
            schema=schema,
            logical_record_name=logical_record_name
        )

        key = (
            logical_record_name,
            physical_record_name

```

```

    )

    if key not in seen:
        seen.add(key)
        resolved.append(key)

    return resolved

def physical_record_name(
    self,
    schema: SchemaModel,
    logical_record_name: str
) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = schema.record_table_map.get(
        logical_record_name
    )

    if mapped and mapped in schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        " "
    )

    mapped = schema.record_table_map.get(
        normalized
    )

    if mapped and mapped in schema.records:
        return mapped

    if logical_record_name in schema.records:
        return logical_record_name

    if normalized in schema.records:
        return normalized

```

```

        return logical_record_name

def column_to_cobol(
    self,
    hv: str,
    datatype: str,
    length: int | None,
    scale: int | None
) -> list[str]:

    datatype = (
        datatype
        or "CHAR"
    ).upper()

    if datatype == "SMALLINT":
        return [
            f"          05 {hv:<45} PIC S9(4) COMP."
        ]

    if datatype in {
        "INTEGER",
        "INT"
    }:
        return [
            f"          05 {hv:<45} PIC S9(9) COMP."
        ]

    if datatype == "BIGINT":
        return [
            f"          05 {hv:<45} PIC S9(18) COMP-3."
        ]

    if datatype in {
        "DECIMAL",
        "NUMERIC"
    }:
        return self.decimal_to_cobol(
            hv=hv,

```

```

        length=length,
        scale=scale
    )

    if datatype == "DATE":
        return [
            f"          05 {hv:<45} PIC X(10)."
        ]

    if datatype == "TIMESTAMP":
        return [
            f"          05 {hv:<45} PIC X(26)."
        ]

    actual_length = (
        length
        if length
        else 255
    )

    if actual_length == 1:
        return [
            f"          05 {hv:<45} PIC X."
        ]

    return [
        f"          05 {hv:<45} PIC X({actual_length})."
    ]

def decimal_to_cobol(
    self,
    hv: str,
    length: int | None,
    scale: int | None
) -> list[str]:

    precision = (
        length
        if length
        else 18

```

```

    )

    actual_scale = (
        scale
        if scale is not None
        else 0
    )

    integer_digits = max(
        precision - actual_scale,
        1
    )

    if actual_scale > 0:
        return [
            f"          05 {hv:<45} PIC
S9({integer_digits})V9({actual_scale}) COMP-3."
        ]

    return [
        f"          05 {hv:<45} PIC S9({precision}) COMP-3."
    ]

```


src\idms_db2_converter\generators\[naming.py](#)

```
class Naming:

    @staticmethod
    def normalize(
        value: str | None
    ) -> str:

        if not value:
            return ""

        return (
            str(value)
            .strip()
            .upper()
            .replace("_", "-")
        )

    @staticmethod
    def hv(
        record: str,
        field: str
    ) -> str:

        record_name = Naming.normalize(record)
        field_name = Naming.normalize(field)

        return f"HV-{record_name}-{field_name}"

    @staticmethod
    def ni(
        record: str,
        field: str
    ) -> str:

        record_name = Naming.normalize(record)
        field_name = Naming.normalize(field)

        return f"NI-{record_name}-{field_name}"
```

```
@staticmethod
def cursor(
    set_name: str
) -> str:

    normalized_set = Naming.normalize(set_name)

    return f"C-{{normalized_set}}"
```

src\idms_db2_converter\generators\sql_snippets.py

```
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

class SqlSnippets:
    """
    Generates embedded SQL snippets with full SELECT and INTO lists.
    """

    def __init__(
        self,
        schema: SchemaModel
    ):
        self.schema = schema

    def select_for_record_by_pk(
        self,
        record_name: str
    ) -> str:

        physical_record_name = self.physical_record_name(
            record_name
        )

        record = self.schema.records.get(
            physical_record_name
        )

        if not record:
            raise ConversionError(
                f"Record {record_name} missing from schema."
            )

        if not record.primary_key:
            raise ConversionError(
                f"Record {record_name} has no primary key."
            )
```

```

columns = list(record.fields.keys())

into_items = [
    self.host_with_indicator(
        logical_record_name=record_name,
        column_name=column_name,
        nullable=record.fields[column_name].nullable
    )
    for column_name in columns
]

lines = [
    "EXEC SQL"
]

lines.extend(
    comma_block(
        items=columns,
        first_prefix="      SELECT ",
        next_prefix="      "
    )
)

lines.extend(
    comma_block(
        items=into_items,
        first_prefix="      INTO ",
        next_prefix="      "
    )
)

lines.append(
    f"      FROM {physical_record_name}"
)

lines.append(
    f"      WHERE {record.primary_key} = :{Naming.hv(record_name,
record.primary_key)}"
)

```

```

        lines.append(
            "END-EXEC."
        )

    return "\n".join(lines)

def select_first_child_for_set(
    self,
    set_name: str
) -> str:

    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    child_record_name = self.physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

    if not child_record:
        raise ConversionError(
            f"Child record {relationship.child_record} missing from
schema."
        )

    columns = list(child_record.fields.keys())

    into_items = [

```

```

        self.host_with_indicator(
            logical_record_name=relationship.child_record,
            column_name=column_name,
            nullable=child_record.fields[column_name].nullable
        )
        for column_name in columns
    ]

    parent_host = Naming.hv(
        relationship.parent_record,
        relationship.parent_key
    )

    lines = [
        "EXEC SQL"
    ]

    lines.extend(
        comma_block(
            items=columns,
            first_prefix="        SELECT ",
            next_prefix="            "
        )
    )

    lines.extend(
        comma_block(
            items=into_items,
            first_prefix="        INTO ",
            next_prefix="            "
        )
    )

    lines.append(
        f"        FROM {child_record_name}"
    )

    lines.append(
        f"        WHERE {relationship.child_fk} = :{parent_host}"
    )

```

```

        if relationship.order_by:
            lines.append(
                f" ORDER BY {'', '}.join(relationship.order_by)}"
            )

        lines.append(
            " FETCH FIRST 1 ROW ONLY"
        )

        lines.append(
            "END-EXEC."
        )

        return "\n".join(lines)

def select_for_owner(
    self,
    set_name: str
) -> str:

    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    parent_record_name = self.physical_record_name(
        relationship.parent_record
    )

    parent_record = self.schema.records.get(
        parent_record_name
    )

```

```

        if not parent_record:
            raise ConversionError(
                f"Parent record {relationship.parent_record} missing from
schema."
            )

        columns = list(parent_record.fields.keys())

        into_items = [
            self.host_with_indicator(
                logical_record_name=relationship.parent_record,
                column_name=column_name,
                nullable=parent_record.fields[column_name].nullable
            )
            for column_name in columns
        ]

        child_host = Naming.hv(
            relationship.child_record,
            relationship.child_fk
        )

        lines = [
            "EXEC SQL"
        ]

        lines.extend(
            comma_block(
                items=columns,
                first_prefix="        SELECT ",
                next_prefix="        "
            )
        )

        lines.extend(
            comma_block(
                items=into_items,
                first_prefix="        INTO ",
                next_prefix="        "
            )
        )

```



```

    )

    lines.append(
        f"        FROM {parent_record_name}"
    )

    lines.append(
        f"        WHERE {relationship.parent_key} = :{child_host}"
    )

    lines.append(
        "END-EXEC."
    )

    return "\n".join(lines)

def fetch_for_set(
    self,
    set_name: str
) -> str:

    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    child_record_name = self.physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

```

```

        if not child_record:
            raise ConversionError(
                f"Child record {relationship.child_record} missing from
schema."
            )

        into_items = [
            self.host_with_indicator(
                logical_record_name=relationship.child_record,
                column_name=column_name,
                nullable=column.nullable
            )
            for column_name, column in child_record.fields.items()
        ]

        lines = [
            "EXEC SQL",
            f"      FETCH {Naming.cursor(set_name)}"
        ]

        lines.extend(
            comma_block(
                items=into_items,
                first_prefix="      INTO ",
                next_prefix="      "
            )
        )

        lines.append(
            "END-EXEC."
        )

        return "\n".join(lines)

    def nullable_fk_indicator_for_set(
        self,
        set_name: str
    ) -> str | None:

        relationship = self.schema.relationships.get(

```

```
        set_name.upper()
    )

    if not relationship:
        return None

    child_record_name = self.physical_record_name(
        relationship.child_record
    )

    child_record = self.schema.records.get(
        child_record_name
    )

    if not child_record:
        return None

    child_fk = relationship.child_fk

    if not child_fk:
        return None

    column = child_record.fields.get(
        child_fk
    )

    if not column:
        return None

    if not column.nullable:
        return None

    return Naming.ni(
        relationship.child_record,
        child_fk
    )

def physical_record_name(
    self,
    logical_record_name: str
```

```

) -> str:

    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(
        logical_record_name
    )

    if mapped and mapped in self.schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        " "
    )

    mapped = self.schema.record_table_map.get(
        normalized
    )

    if mapped and mapped in self.schema.records:
        return mapped

    if logical_record_name in self.schema.records:
        return logical_record_name

    if normalized in self.schema.records:
        return normalized

    return logical_record_name

def host_with_indicator(
    self,
    logical_record_name: str,
    column_name: str,
    nullable: bool
) -> str:

    host = f":{{Naming.hv(logical_record_name, column_name)}}"
```

```
        if nullable:
            host = f"{host} :{Naming.ni(logical_record_name,
column_name)}"

        return host
```



```

        fields[field_name] = Column(
            name=field_name,
            datatype=field_json["datatype"].upper(),
            length=field_json.get("length"),
            scale=field_json.get("scale"),
            nullable=bool(nullable),
        )

    schema.records[record_name] = Record(
        name=record_name,
        primary_key=primary_key,
        fields=fields,
    )

def _parse_relationships(self, payload: dict, schema: SchemaModel) ->
None:
    for rel_json in payload.get("relationships", []):
        set_name = rel_json["set_name"].upper()
        parent_record = rel_json["parent_record"].upper()
        child_record = rel_json["child_record"].upper()

        parent_key = self._upper_or_none(rel_json.get("parent_key"))
        child_fk = self._upper_or_none(rel_json.get("child_fk"))

        if not parent_key:
            parent_key = self._resolve_parent_key(schema,
parent_record)

        if not child_fk:
            child_fk = self._resolve_child_fk(
                schema=schema,
                parent_record=parent_record,
                child_record=child_record,
            )

    order_by = [
        value.upper()
        for value in rel_json.get("order_by", [])
    ]

```

```

        if not order_by and child_fk:
            order_by = [child_fk]

        schema.relationships[set_name] = Relationship(
            set_name=set_name,
            parent_record=parent_record,
            child_record=child_record,
            cardinality=rel_json.get("cardinality"),
            parent_key=parent_key,
            child_fk=child_fk,
            order_by=order_by,
        )

    def _parse_field_map(self, payload: dict, schema: SchemaModel) ->
None:
        schema.field_map = {
            key.upper(): value
            for key, value in payload.get("field_map", {}).items()
        }

    def _resolve_parent_key(
        self,
        schema: SchemaModel,
        parent_record: str,
    ) -> str | None:
        record = schema.records.get(parent_record)

        if not record:
            return None

        return record.primary_key

    def _resolve_child_fk(
        self,
        schema: SchemaModel,
        parent_record: str,
        child_record: str,
    ) -> str | None:
        parent = schema.records.get(parent_record)
        child = schema.records.get(child_record)

```



```
    if not parent or not child or not parent.primary_key:
        return None

    if parent.primary_key in child.fields:
        return parent.primary_key

    return None

def _upper_or_none(self, value: str | None) -> str | None:
    return value.upper() if value else None
```

src\idms_db2_converter\parsers\cobol_parser.py

```
import re
from dataclasses import dataclass, field

from idms_db2_converter.models import CobolAnalysis, Paragraph

class CobolParser:
    """
    Parses IDMS retrieval COBOL.

    Captures:
    - PROGRAM-ID
    - COPY IDMS RECORD usage
    - OBTAIN CALC record usage
    - OBTAIN NEXT record WITHIN set
    - OBTAIN OWNER WITHIN set
    - FIND FIRST WITHIN set
    - Paragraphs
    - IDMS schema-suffixed field references in PROCEDURE DIVISION

    The field references are important for:
    - Host variable generation
    - SELECT-list completion
    - Field rewriting
    """

    PROGRAM_ID = re.compile(
        r"^\s*PROGRAM-ID\s*([A-Z0-9-])+\s*\.?\s*",
        re.IGNORECASE | re.MULTILINE
    )

    COPY_RECORD = re.compile(
        r"^\s*(?:01\s+)?COPY\s+IDMS\s+RECORD\s*([A-Z0-9-])+\s*\.?\s*",
        re.IGNORECASE | re.MULTILINE
    )

    OBTAIN_CALC = re.compile(
        r"^\s*OBTAIN\s+CALC\s*([A-Z0-9-])+\s*\.?\s*\s*$",
```

```

        re.IGNORECASE | re.MULTILINE
    )

    OBTAIN_NEXT = re.compile(
        r"^\\s*OBTAIN\\s+NEXT\\s+([A-Z0-9-]+)\\s+WITHIN\\s+([A-Z0-9-]+)\\.\\.\\s*$",
        re.IGNORECASE | re.MULTILINE
    )

    OBTAIN_OWNER = re.compile(
        r"^\\s*OBTAIN\\s+OWNER\\s+WITHIN\\s+([A-Z0-9-]+)\\.\\.\\s*$",
        re.IGNORECASE | re.MULTILINE
    )

    FIND_FIRST = re.compile(
        r"^\\s*FIND\\s+FIRST\\s+WITHIN\\s+([A-Z0-9-]+)\\.\\.\\s*$",
        re.IGNORECASE | re.MULTILINE
    )

    PARAGRAPH_HEADER = re.compile(
        r"^\\s*([A-Z0-9-]+)\\.\\.\\s*$",
        re.IGNORECASE | re.MULTILINE
    )

    IDMS_SCHEMA_FIELD = re.compile(
        r"\\b[A-Z][A-Z0-9]*(?:-[A-Z0-9-]+)*-\\d{4}\\b",
        re.IGNORECASE
    )

    PROCEDURE_DIVISION = re.compile(
        r"^\\s*PROCEDURE\\s+DIVISION\\.\\.\\s*$",
        re.IGNORECASE | re.MULTILINE
    )

    def parse(
        self,
        cobol: str
    ) -> CobolAnalysis:

        source = self.remove_comment_lines(

```

```

        cobol
    )

    program_match = self.PROGRAM_ID.search(
        source
    )

    analysis = CobolAnalysis(
        program_id=self._upper_or_none(
            program_match.group(1)
            if program_match
            else None
        ),
        idms_records=self._unique(
            self.COPY_RECORD.findall(source)
        ),
        obtain_calc_records=self._unique(
            self.OBTAIN_CALC.findall(source)
        ),
        obtain_next=[
            (
                record.upper(),
                set_name.upper()
            )
            for record, set_name in self.OBTAIN_NEXT.findall(source)
        ],
        obtain_owner_sets=self._unique(
            self.OBTAIN_OWNER.findall(source)
        ),
        find_first_sets=self._unique(
            self.FIND_FIRST.findall(source)
        ),
    )

    self._attach_field_references(
        analysis=analysis,
        cobol=source
    )

    return analysis

```

```

def paragraphs(
    self,
    cobol: str
) -> list[Paragraph]:

    source = self.remove_comment_lines(
        cobol
    )

    matches = list(
        self.PARAGRAPH_HEADER.finditer(source)
    )

    paragraphs: list[Paragraph] = []

    for index, match in enumerate(matches):
        name = match.group(1).upper()
        start = match.start()

        if index + 1 < len(matches):
            end = matches[index + 1].start()
        else:
            end = len(source)

        paragraphs.append(
            Paragraph(
                name=name,
                text=source[start:end]
            )
        )

    return paragraphs

def remove_comment_lines(
    self,
    cobol: str
) -> str:

    lines: list[str] = []

```

```

for line in cobol.splitlines():
    stripped = line.strip()

    if not stripped:
        lines.append(line)
        continue

    if stripped.startswith("*"):
        continue

    lines.append(line)

return "\n".join(lines)

def _attach_field_references(
    self,
    analysis: CobolAnalysis,
    cobol: str
) -> None:

    procedure_text = self._procedure_division_text(
        cobol
    )

    field_references = self._unique(
        self.IDMS_SCHEMA_FIELD.findall(
            procedure_text
        )
    )

    try:
        analysis.field_references = field_references
    except Exception:
        pass

    try:
        analysis.idms_fields = field_references
    except Exception:
        pass

```

```

record_names = set(
    analysis.idms_records
)

for field_name in field_references:
    record_suffix = field_name.split("-")[-1]

    inferred_record = self._record_from_suffix(
        suffix=record_suffix
    )

    if inferred_record:
        record_names.add(inferred_record)

for record in analysis.obtain_calc_records:
    record_names.add(record)

for record, _ in analysis.obtain_next:
    record_names.add(record)

try:
    analysis.idms_records = sorted(record_names)
except Exception:
    pass

def _procedure_division_text(
    self,
    cobol: str
) -> str:

    match = self.PROCEDURE_DIVISION.search(
        cobol
    )

    if not match:
        return cobol

    return cobol[match.start():]

```

```

def _record_from_suffix(
    self,
    suffix: str
) -> str | None:

    suffix_to_record = {
        "0400": "COVERAGE",
        "0405": "DENTAL-CLAIM",
        "0410": "DEPARTMENT",
        "0415": "EMPLOYEE",
        "0420": "EMPOSITION",
        "0425": "EXPERTISE",
        "0430": "HOSPITAL-CLAIM",
        "0435": "INSURANCE-PLAN",
        "0440": "JOB",
        "0445": "NON-HOSP-CLAIM",
        "0450": "OFFICE",
        "0455": "SKILL",
        "0460": "STRUCTURE",
    }

    return suffix_to_record.get(
        suffix
    )

def _unique(
    self,
    values
) -> list[str]:

    seen = set()
    result: list[str] = []

    for value in values:
        upper_value = value.upper()

        if upper_value not in seen:
            seen.add(upper_value)
            result.append(upper_value)

```



```
        return result

def _upper_or_none(
    self,
    value: str | None
) -> str | None:

    return value.upper() if value else None
```

src\idms_db2_converter\parsers\ddl_parser.py

```
import re

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import (
    Column,
    Record,
    Relationship,
    SchemaModel
)

class DDLParser:
    """
    Robust DB2 DDL parser for Phase 2.

    This version uses manual datatype parsing instead of fragile
    regex-only
    datatype parsing.

    Fixes:
    - VARCHAR(3) being skipped
    - VARCHAR(45) being skipped
    - CHAR(1) being skipped
    - DECIMAL(18,0) being skipped
    - Missing columns in final Phase 2 schema
    """

    DEBUG = True

    INLINE_PRIMARY_KEY = re.compile(
        r"\bPRIMARY\s+KEY\b",
        re.IGNORECASE
    )

    INLINE_NOT_NULL = re.compile(
        r"\bNOT\s+NULL\b",
        re.IGNORECASE
    )
```

```

TABLE_PRIMARY_KEY = re.compile(
    r"\bPRIMARY\s+KEY\s*$(?P<columns>.*?)$",
    re.IGNORECASE | re.DOTALL
)

TABLE_FOREIGN_KEY = re.compile(
    r"""
    \bFOREIGN\s+KEY\s*
    $(?P<child_cols>.*?)$
    \s+REFERENCES\s+
    (?P<parent_table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
    \s*
    $(?P<parent_cols>.*?)$
    """,
    re.IGNORECASE | re.DOTALL | re.VERBOSE
)

ALTER_TABLE_FK = re.compile(
    r"""
    \bALTER\s+TABLE\s+
    (?P<child_table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
    .*?
    \bFOREIGN\s+KEY\s*
    $(?P<child_cols>.*?)$
    \s+REFERENCES\s+
    (?P<parent_table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
    \s*
    $(?P<parent_cols>.*?)$
    """,
    re.IGNORECASE | re.DOTALL | re.VERBOSE
)

CONSTRAINT_STARTERS = {
    "CONSTRAINT",
    "PRIMARY",
    "FOREIGN",
    "UNIQUE",
    "CHECK"
}

```

```

def parse(
    self,
    ddl: str
) -> SchemaModel:

    self._debug(
        "USING DDL_PARSER VERSION MANUAL-DATATYPE-2026-07-01"
    )

    schema = SchemaModel()
    schema.schema_source = "DDL"

    if ddl is None:
        ddl = ""

    self._debug(
        f"DDL INPUT LENGTH: {len(ddl)}"
    )

    self._debug(
        f"DDL HAS CREATE TABLE: {'CREATE TABLE' in ddl.upper()}"
    )

    ddl = self.remove_comments(
        ddl
    )

    create_statements = self._extract_create_table_statements(
        ddl
    )

    self._debug(
        f"DDL_PARSER CREATE TABLE COUNT: {len(create_statements)}"
    )

    for statement in create_statements:
        table_name = statement["table_name"]
        body = statement["body"]

```

```

        self._debug(
            f"DDL PARSER START TABLE: {table_name}"
        )

        record = Record(
            name=table_name,
            primary_key=None,
            fields={}
        )

        schema.record_table_map[
            table_name
        ] = table_name

        entries = self.split_entries(
            body
        )

        self._debug(
            f"DDL PARSER TABLE {table_name} ENTRY COUNT:
{len(entries)}"
        )

        pending_foreign_keys: list[dict] = []

        for entry in entries:
            entry = entry.strip()

            if not entry:
                continue

            first_word = self._first_word(
                entry
            )

            if first_word in self.CONSTRAINT_STARTERS:
                self._parse_constraint_entry(
                    record=record,
                    entry=entry,
                    pending_foreign_keys=pending_foreign_keys

```

```

        )
        continue

    column = self._parse_column_entry(
        entry=entry,
        table_name=table_name
    )

    if column is None:
        continue

    record.fields[
        column.name
    ] = column

    if self.INLINE_PRIMARY_KEY.search(entry):
        record.primary_key = column.name
        column.nullable = False

    schema.records[
        table_name
    ] = record

    self._debug(
        f"DDL PARSER TABLE {table_name}: "
        f"{len(record.fields)} columns, PK={record.primary_key}"
    )

    for field_name, field in record.fields.items():
        self._debug(
            f" COLUMN {field_name}: "
            f"{field.datatype}({field.length},{field.scale}) "
            f"NULLABLE={field.nullable}"
        )

    for foreign_key in pending_foreign_keys:
        self._add_relationship(
            schema=schema,
            child_table=table_name,
            child_fk=foreign_key["child_fk"],

```

```

        parent_table=foreign_key["parent_table"],
        parent_key=foreign_key["parent_key"]
    )

    self._parse_alter_table_foreign_keys(
        ddl=ddl,
        schema=schema
    )

    self._validate(
        schema
    )

    if "CREATE TABLE" in ddl.upper() and not schema.records:
        raise ConversionError(
            "DDL text was provided and contains CREATE TABLE, but no
tables were parsed."
        )

    return schema

def remove_comments(
    self,
    ddl: str
) -> str:

    ddl = re.sub(
        r"--.*?$",
        "",
        ddl,
        flags=re.MULTILINE
    )

    ddl = re.sub(
        r"/\*.*?\*/",
        "",
        ddl,
        flags=re.DOTALL
    )

```

```

        return ddl

    def _extract_create_table_statements(
        self,
        ddl: str
    ) -> list[dict]:

        results: list[dict] = []

        pattern = re.compile(
            r"\bCREATE\s+TABLE\s+(?P<table>(?:[A-Z0-9_]+\.)?[A-Z0-9_]+)",
            re.IGNORECASE
        )

        for match in pattern.finditer(ddl):
            table_name = self._normalize_name(
                match.group("table")
            )

            open_index = ddl.find(
                "(",
                match.end()
            )

            if open_index < 0:
                self._debug(
                    f"DDL PARSER TABLE {table_name}: no opening
parenthesis found"
                )
                continue

            close_index = self._find_matching_parenthesis(
                text=ddl,
                open_index=open_index
            )

            if close_index < 0:
                self._debug(
                    f"DDL PARSER TABLE {table_name}: no closing
parenthesis found"
                )

```



```

        )
        continue

    body = ddl[
        open_index + 1:close_index
    ]

    results.append(
        {
            "table_name": table_name,
            "body": body
        }
    )

    return results

def _find_matching_parenthesis(
    self,
    text: str,
    open_index: int
) -> int:

    depth = 0
    quote = None

    for index in range(open_index, len(text)):
        char = text[index]

        if quote:
            if char == quote:
                quote = None
                continue

        if char in {
            '"',
            "'",
        }:
            quote = char
            continue

```

```

        if char == "(":
            depth += 1
            continue

        if char == ")":
            depth -= 1

            if depth == 0:
                return index

    return -1

def split_entries(
    self,
    body: str
) -> list[str]:

    entries: list[str] = []
    current: list[str] = []
    depth = 0
    quote = None

    for char in body:
        if quote:
            current.append(char)

            if char == quote:
                quote = None

            continue

        if char in {
            '"',
            "'",
        }:
            quote = char
            current.append(char)
            continue

        if char == "(":

```

```

        depth += 1
        current.append(char)
        continue

    if char == ")":
        depth -= 1
        current.append(char)
        continue

    if char == "," and depth == 0:
        entry = "".join(current).strip()

        if entry:
            entries.append(entry)

        current = []
        continue

    current.append(char)

entry = "".join(current).strip()

if entry:
    entries.append(entry)

return entries

def _parse_constraint_entry(
    self,
    record: Record,
    entry: str,
    pending_foreign_keys: list[dict]
) -> None:

    primary_key_match = self.TABLE_PRIMARY_KEY.search(
        entry
    )

    if primary_key_match:
        primary_key = self._first_column(

```

```

        primary_key_match.group("columns")
    )

    if primary_key:
        record.primary_key = primary_key

        if primary_key in record.fields:
            record.fields[
                primary_key
            ].nullable = False

    self._debug(
        f"DDL PARSER PRIMARY KEY FOR {record.name}:
{record.primary_key}"
    )

    return

foreign_key_match = self.TABLE_FOREIGN_KEY.search(
    entry
)

if foreign_key_match:
    child_fk = self._first_column(
        foreign_key_match.group("child_cols")
    )

    parent_table = self._normalize_name(
        foreign_key_match.group("parent_table")
    )

    parent_key = self._first_column(
        foreign_key_match.group("parent_cols")
    )

    if child_fk and parent_table and parent_key:
        pending_foreign_keys.append(
            {
                "child_fk": child_fk,
                "parent_table": parent_table,

```

```

        "parent_key": parent_key
    }
)

self._debug(
    f"DDL_PARSER_FK: {child_fk} ->
{parent_table}({parent_key})"
)

return

def _parse_alter_table_foreign_keys(
    self,
    ddl: str,
    schema: SchemaModel
) -> None:

    for match in self.ALTER_TABLE_FK.finditer(ddl):
        child_table = self._normalize_name(
            match.group("child_table")
        )

        child_fk = self._first_column(
            match.group("child_cols")
        )

        parent_table = self._normalize_name(
            match.group("parent_table")
        )

        parent_key = self._first_column(
            match.group("parent_cols")
        )

        if not child_table:
            continue

        if not child_fk:
            continue

```

```

        if not parent_table:
            continue

        if not parent_key:
            continue

        self._add_relationship(
            schema=schema,
            child_table=child_table,
            child_fk=child_fk,
            parent_table=parent_table,
            parent_key=parent_key
        )

def _add_relationship(
    self,
    schema: SchemaModel,
    child_table: str,
    child_fk: str,
    parent_table: str,
    parent_key: str
) -> None:

    child_table = self._normalize_name(child_table)
    child_fk = self._normalize_name(child_fk)
    parent_table = self._normalize_name(parent_table)
    parent_key = self._normalize_name(parent_key)

    set_name = f"{parent_table}-{child_table}"

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=parent_table,
        child_record=child_table,
        cardinality="1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=[child_fk]
    )

```

```

def _parse_column_entry(
    self,
    entry: str,
    table_name: str
) -> Column | None:

    entry = entry.strip()

    if not entry:
        return None

    match = re.match(
        r"^(?P<name>[A-Z0-9_]+) \s+(?P<datatype>.+)$",
        entry,
        flags=re.IGNORECASE | re.DOTALL
    )

    if not match:
        self._debug(
            f"DDL PARSER SKIPPED ENTRY IN {table_name}: no column
match. ENTRY={repr(entry)}"
        )
        return None

    column_name = self._normalize_name(
        match.group("name")
    )

    datatype_text = match.group("datatype").strip()

    datatype, length, scale = self._parse_datatype(
        datatype_text
    )

    if datatype is None:
        self._debug_unrecognized_datatype(
            table_name=table_name,
            column_name=column_name,
            datatype_text=datatype_text
        )

```

```

        return None

    nullable = not bool(
        self.INLINE_NOT_NULL.search(entry)
    )

    return Column(
        name=column_name,
        datatype=datatype,
        length=length,
        scale=scale,
        nullable=nullable
    )

def _parse_datatype(
    self,
    datatype_text: str
) -> tuple[str | None, int | None, int | None]:

    text = self._normalize_datatype_text(
        datatype_text
    )

    if not text:
        return (
            None,
            None,
            None
        )

    datatype_name = self._leading_word(
        text
    )

    if datatype_name in {
        "VARCHAR",
        "CHAR",
        "CHARACTER"
    }:
        length = self._manual_first_parenthesized_int(

```



```

        text
    )

    if datatype_name == "CHARACTER":
        datatype_name = "CHAR"

    if length is None and datatype_name == "CHAR":
        length = 1

    if length is not None:
        return (
            datatype_name,
            length,
            None
        )

    if datatype_name in {
        "DECIMAL",
        "NUMERIC"
    }:
        precision_scale = self._manual_precision_scale(
            text
        )

        if precision_scale is not None:
            precision, scale = precision_scale

            return (
                "DECIMAL",
                precision,
                scale
            )

    if datatype_name == "SMALLINT":
        return (
            "SMALLINT",
            4,
            0
        )

```

```
if datatype_name in {
    "INTEGER",
    "INT"
}:
    return (
        "INTEGER",
        9,
        0
    )

if datatype_name == "BIGINT":
    return (
        "BIGINT",
        18,
        0
    )

if datatype_name == "DATE":
    return (
        "DATE",
        None,
        None
    )

if datatype_name == "TIMESTAMP":
    return (
        "TIMESTAMP",
        None,
        None
    )

if datatype_name == "TIME":
    return (
        "TIME",
        None,
        None
    )

return (
    None,
```

```

        None,
        None
    )

def _normalize_datatype_text(
    self,
    datatype_text: str
) -> str:

    text = datatype_text or ""

    text = str(text).strip().upper()

    text = text.replace(" (", "(")
    text = text.replace(") ", ")")
    text = text.replace(", ", ",")
    text = text.replace("\u00a0", " ")

    text = re.sub(
        r"\s+",
        " ",
        text
    )

    text = re.sub(
        r"\bNOT\s+NULL\b",
        "",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"\bPRIMARY\s+KEY\b",
        "",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"\bDEFAULT\b.*$",

```

```

        """
        text,
        flags=re.IGNORECASE
    )

    return text.strip()

def _leading_word(
    self,
    text: str
) -> str:

    result = []

    for char in text:
        if char.isalpha():
            result.append(char)
            continue

        break

    return "".join(result).upper()

def _manual_first_parenthesized_int(
    self,
    text: str
) -> int | None:

    open_index = text.find("(")

    if open_index < 0:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index < 0:
        return None

    content = text[
        open_index + 1:close_index

```

```

].strip()

if not content.isdigit():
    return None

return int(content)

def _manual_precision_scale(
    self,
    text: str
) -> tuple[int, int] | None:

    open_index = text.find("(")

    if open_index < 0:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index < 0:
        return None

    content = text[
        open_index + 1:close_index
    ].strip()

    parts = [
        part.strip()
        for part in content.split(",")
    ]

    if len(parts) != 2:
        return None

    if not parts[0].isdigit():
        return None

    if not parts[1].isdigit():
        return None

```

```

        return (
            int(parts[0]),
            int(parts[1])
        )

    def _validate(
        self,
        schema: SchemaModel
    ) -> None:

        errors: list[str] = []

        for record in schema.records.values():
            if record.primary_key and record.primary_key not in
record.fields:
                errors.append(
                    f"{record.name} primary key {record.primary_key} is
not declared as a column."
                )

            for field_name, field in record.fields.items():
                if not field.datatype:
                    errors.append(
                        f"{record.name}.{field_name} has no datatype."
                    )

        if errors:
            raise ConversionError(
                "\n".join(errors)
            )

    def _first_word(
        self,
        entry: str
    ) -> str:

        tokens = entry.strip().split()

        if not tokens:
            return ""

```

```

        return self._normalize_name(
            tokens[0]
        )

def _first_column(
    self,
    text: str
) -> str | None:

    if not text:
        return None

    first = text.split(",")[0].strip()

    if not first:
        return None

    return self._normalize_name(
        first
    )

def _normalize_name(
    self,
    value: str | None
) -> str:

    if not value:
        return ""

    value = str(value).strip()
    value = value.strip('"')
    value = value.strip("'")
    value = value.strip("`")
    value = value.strip("[")
    value = value.strip("]")

    if "." in value:
        value = value.split(".")[-1]

```

```

        return value.upper()

def _debug(
    self,
    message: str
) -> None:

    if self.DEBUG:
        print(message)

def _debug_unrecognized_datatype(
    self,
    table_name: str,
    column_name: str,
    datatype_text: str
) -> None:

    if not self.DEBUG:
        return

    normalized = self._normalize_datatype_text(
        datatype_text
    )

    char_codes = [
        ord(char)
        for char in datatype_text
    ]

    print(
        "DDL PARSER SKIPPED COLUMN "
        f"{table_name}.{column_name}: "
        f"raw={repr(datatype_text)} "
        f"normalized={repr(normalized)} "
        f"leading={repr(self._leading_word(normalized))} "
        f"char_codes={char_codes}"
    )

```


src\idms_db2_converter\parsers\idms_schema_parser.py

```
import re

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Column, Record, Relationship,
SchemaModel

class IdmsSchemaParser:
    RECORD_HEADER = re.compile(
        r"^RECORD NAME\.\+\s+([A-Z0-9-]+)",
        re.IGNORECASE | re.MULTILINE,
    )

    LOCATION_CALC = re.compile(
        r"LOCATION MODE\.\+\s+CALC USING\s+([A-Z0-9-]+)",
        re.IGNORECASE,
    )

    DBKEY_SET_ROLE = re.compile(
        r"^\s+([A-Z0-9-]+)\s+(?:INDEX\s+)?(OWNER|MEMBER)\b",
        re.IGNORECASE | re.MULTILINE,
    )

    DATA_LINE_PREFIX = re.compile(
        r"^\s*(0[2-5])\s+([A-Z0-9-]+)\s+(.*)$",
        re.IGNORECASE,
    )

    PICTURE_FIND = re.compile(
        r"S?9$\d+$V9$\d+$| "
        r"S?9$\d+$V9+| "
        r"S?9+V9$\d+$| "
        r"S?9+V9+| "
        r"S?V9$\d+$| "
        r"S?V9+| "
        r"S?9$\d+$| "
        r"S?9+| "
        r"X$\d+$| "
        r"X+",
    )
```

```

        re.IGNORECASE,
    )

    DATE_PARTS = {
        "YEAR": {
            "substring_start": 1,
            "substring_length": 4,
        },
        "MONTH": {
            "substring_start": 5,
            "substring_length": 2,
        },
        "DAY": {
            "substring_start": 7,
            "substring_length": 2,
        },
    }

    def parse(self, text: str) -> SchemaModel:
        schema = SchemaModel()
        schema.schema_source = "IDMS_SCHEMA"

        blocks = self._record_blocks(text)
        set_roles: dict[str, dict[str, str]] = {}

        for record_name, block in blocks:
            self._parse_record(schema, record_name, block)
            self._collect_set_roles(set_roles, record_name, block)

        self._build_relationships(schema, set_roles)

        if hasattr(schema, "add_validation_message"):
            schema.add_validation_message("Schema built from IDMS schema listing.")

        return schema

    def _record_blocks(self, text: str) -> list[tuple[str, str]]:
        matches = list(self.RECORD_HEADER.finditer(text))
        result = []

```

```

        for index, match in enumerate(matches):
            record_name = match.group(1).upper()
            start = match.start()
            end = matches[index + 1].start() if index + 1 < len(matches)
else len(text)

            result.append((record_name, text[start:end]))

        return result

def _parse_record(
    self,
    schema: SchemaModel,
    record_name: str,
    block: str,
) -> None:
    primary_key = None

    calc_match = self.LOCATION_CALC.search(block)

    if calc_match:
        primary_key = self._normalize_column(calc_match.group(1))

    record = Record(
        name=record_name,
        primary_key=primary_key,
        fields={},
    )

    schema.record_table_map[record_name] =
self._normalize_table(record_name)

    active_date_groups: dict[int, dict[str, str]] = {}

    for line in block.splitlines():
        parsed = self._parse_data_item_line(line)

        if not parsed:
            continue

```

```

        level = parsed["level"]
        raw_field_name = parsed["field_name"]
        body = parsed["body"]
        storage_length = parsed["length"]

        self._close_inactive_date_groups(active_date_groups, level)

        if raw_field_name == "FILLER":
            continue

        usage = self._extract_usage(body)
        picture = self._extract_picture(body)
        column_name = self._normalize_column(raw_field_name)

        if self._is_date_group(raw_field_name, picture,
storage_length):
            if column_name not in record.fields:
                record.fields[column_name] = Column(
                    name=column_name,
                    datatype="CHAR",
                    length=8,
                    scale=None,
                    nullable=False,
                )

            host = Naming.hv(record_name, column_name)
            self._add_field_map(schema, raw_field_name, host)

            active_date_groups[level] = {
                "raw_field_name": raw_field_name,
                "column_name": column_name,
                "host": host,
            }

            continue

        date_group = self._nearest_date_group(active_date_groups)

        if date_group and self._is_date_part(raw_field_name):

```

```

        self._add_date_part_map(
            schema=schema,
            idms_field=raw_field_name,
            host=date_group["host"],
        )
        continue

    if not picture:
        continue

    datatype, column_length, scale = self._map_picture(
        picture=picture,
        usage=usage,
        storage_length=storage_length,
    )

    if column_name not in record.fields:
        record.fields[column_name] = Column(
            name=column_name,
            datatype=datatype,
            length=column_length,
            scale=scale,
            nullable=False,
        )

    self._add_field_map(
        schema=schema,
        idms_field=raw_field_name,
        host=Naming.hv(record_name, column_name),
    )

    if primary_key and primary_key not in record.fields:
        record.fields[primary_key] = Column(
            name=primary_key,
            datatype="CHAR",
            length=12,
            scale=None,
            nullable=False,
        )

```

```

        schema.records[record_name] = record

def _parse_data_item_line(self, line: str) -> dict | None:
    match = self.DATA_LINE_PREFIX.match(line)

    if not match:
        return None

    level = int(match.group(1))
    field_name = match.group(2).upper()
    remainder = match.group(3).strip()

    if field_name == "FILLER":
        return None

    parts = remainder.split()

    if len(parts) < 2:
        return None

    if not parts[-1].isdigit() or not parts[-2].isdigit():
        return None

    length = int(parts[-1])
    start = int(parts[-2])
    body = " ".join(parts[:-2])

    return {
        "level": level,
        "field_name": field_name,
        "body": body,
        "start": start,
        "length": length,
    }

def _collect_set_roles(
    self,
    set_roles: dict[str, dict[str, str]],
    record_name: str,
    block: str,

```

```

) -> None:
    for match in self.DBKEY_SET_ROLE.finditer(block):
        set_name = match.group(1).upper()
        role = match.group(2).upper()

        if set_name == "CALC":
            continue

        set_roles.setdefault(set_name, {})

        if role == "OWNER":
            set_roles[set_name]["owner"] = record_name

        if role == "MEMBER":
            set_roles[set_name]["member"] = record_name

def _build_relationships(
    self,
    schema: SchemaModel,
    set_roles: dict[str, dict[str, str]],
) -> None:
    for set_name, roles in set_roles.items():
        owner = roles.get("owner")
        member = roles.get("member")

        if not owner or not member:
            continue

        if owner not in schema.records or member not in
schema.records:
            continue

        owner_record = schema.records[owner]
        member_record = schema.records[member]

        parent_key = owner_record.primary_key

        if not parent_key:
            continue

```

```

        child_fk = parent_key

    if child_fk not in member_record.fields:
        parent_column = owner_record.fields.get(parent_key)

        member_record.fields[child_fk] = Column(
            name=child_fk,
            datatype=parent_column.datatype if parent_column else
"CHAR",

            length=parent_column.length if parent_column else 12,
            scale=parent_column.scale if parent_column else None,
            nullable=True,
        )

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=owner,
        child_record=member,
        cardinality="1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=[child_fk],
    )

def _extract_usage(self, body: str) -> str:
    upper = body.upper()

    if "COMP-3" in upper:
        return "COMP-3"

    if re.search(r"\bCOMP\b", upper):
        return "COMP"

    return "DISPLAY"

def _extract_picture(self, body: str) -> str | None:
    upper = body.upper()
    matches = list(self.PICTURE_FIND.finditer(upper))

    if not matches:

```



```

        return None

    valid_matches = []

    for match in matches:
        value = match.group(0).upper()
        start = match.start()
        end = match.end()

        before = upper[start - 1] if start > 0 else " "
        after = upper[end] if end < len(upper) else " "

        if before.isalnum() or before in {"-", "_":
            continue

        if after.isalnum() or after in {"-", "_":
            continue

        valid_matches.append(value)

    if not valid_matches:
        return None

    return valid_matches[-1]

def _map_picture(
    self,
    picture: str,
    usage: str,
    storage_length: int,
) -> tuple[str, int | None, int | None]:
    picture = picture.upper()
    usage = usage.upper()

    if picture.startswith("X"):
        length_match = re.search(r"X$(\d+)$", picture)

        if length_match:
            return "CHAR", int(length_match.group(1)), None

```

```

        return "CHAR", len(picture), None

    if "9" in picture:
        precision = self._numeric_precision(picture)
        scale = self._numeric_scale(picture)

        if usage == "DISPLAY":
            return "CHAR", storage_length, None

        if usage == "COMP":
            return "INTEGER", None, None

        return "DECIMAL", precision, scale

    return "CHAR", storage_length, None

def _numeric_precision(self, picture: str) -> int:
    picture = picture.upper().replace("S", "")

    if "V" in picture:
        before_v, after_v = picture.split("V", 1)
    else:
        before_v = picture
        after_v = ""

    return self._count_9_digits(before_v) +
self._count_9_digits(after_v)

def _numeric_scale(self, picture: str) -> int:
    picture = picture.upper().replace("S", "")

    if "V" not in picture:
        return 0

    after_v = picture.split("V", 1)[1]

    return self._count_9_digits(after_v)

def _count_9_digits(self, value: str) -> int:
    total = 0

```

```

        for match in re.finditer(r"9$(\d+)$|9", value):
            if match.group(1):
                total += int(match.group(1))
            else:
                total += 1

        return total

def _is_date_group(
    self,
    raw_field_name: str,
    picture: str | None,
    storage_length: int,
) -> bool:
    normalized = self._normalize_column(raw_field_name)

    return (
        picture is None
        and normalized.endswith("DATE")
        and storage_length == 8
    )

def _is_date_part(self, raw_field_name: str) -> bool:
    normalized = self._normalize_column(raw_field_name)

    return (
        normalized.endswith("_YEAR")
        or normalized.endswith("_MONTH")
        or normalized.endswith("_DAY")
    )

def _add_date_part_map(
    self,
    schema: SchemaModel,
    idms_field: str,
    host: str,
) -> None:
    normalized = self._normalize_column(idms_field)

```

```

        if normalized.endswith("_YEAR"):
            part = "YEAR"
        elif normalized.endswith("_MONTH"):
            part = "MONTH"
        elif normalized.endswith("_DAY"):
            part = "DAY"
        else:
            return

        schema.date_part_map[idms_field.upper()] = {
            "host": host,
            "substring_start": self.DATE_PARTS[part]["substring_start"],
            "substring_length": self.DATE_PARTS[part]["substring_length"],
        }

    def _add_field_map(
        self,
        schema: SchemaModel,
        idms_field: str,
        host: str,
    ) -> None:
        schema.field_map[idms_field.upper()] = {
            "host": host,
        }

    def _close_inactive_date_groups(
        self,
        active_date_groups: dict[int, dict[str, str]],
        current_level: int,
    ) -> None:
        closed_levels = [
            level
            for level in active_date_groups
            if level >= current_level
        ]

        for level in closed_levels:
            del active_date_groups[level]

    def _nearest_date_group(

```

```
        self,
        active_date_groups: dict[int, dict[str, str]],
    ) -> dict[str, str] | None:
        if not active_date_groups:
            return None

        nearest_level = max(active_date_groups.keys())

        return active_date_groups[nearest_level]

    def _normalize_table(self, record_name: str) -> str:
        return record_name.upper().replace("-", "_")

    def _normalize_column(self, field_name: str) -> str:
        value = field_name.upper()
        value = re.sub(r"-\\d{4}$", "", value)

        return value.replace("-", "_")
```

src\idms_db2_converter\parsers\phase2_metadata_parser.py

```
import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import (
    Column,
    Record,
    Relationship,
    SchemaModel,
)

class Phase2MetadataParser:
    """
    Parses Phase 2 metadata JSON produced by Phase 1.

    Supports:
    - records / tables / db2_tables
    - relationships / sets / db2_relationships
    - record_table_map
    - field_map
    - calc_key_map
    - set_ordering_map
    - navigation_intent
    - nullable_fk_map
    - date_part_map
    - output_semantics
    - paragraph_operation_graph
    - validation_messages
    """

    def parse_as_schema(
        self,
        text: str
    ) -> SchemaModel:

        schema = SchemaModel()

        if not text or not text.strip():
```

```

        return schema

    payload = self._load_payload(text)

    self._parse_metadata_records(
        payload=payload,
        schema=schema
    )

    self._parse_metadata_relationships(
        payload=payload,
        schema=schema,
        merge_existing=False
    )

    self.parse_into_schema(
        text=text,
        schema=schema
    )

    schema.schema_source = "PHASE2_METADATA"

    return schema

def parse_into_schema(
    self,
    text: str,
    schema: SchemaModel
) -> SchemaModel:

    if not text or not text.strip():
        return schema

    payload = self._load_payload(text)

    self._merge_record_table_map(
        payload=payload,
        schema=schema
    )

```

```
self._merge_field_map(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_calc_key_map(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_set_ordering_map(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_navigation_intent(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_nullable_fk_map(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_date_part_map(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_output_semantics(  
    payload=payload,  
    schema=schema  
)  
  
self._merge_paragraph_operation_graph(  
    payload=payload,  
    schema=schema  
)
```



```

        self._parse_metadata_relationships(
            payload=payload,
            schema=schema,
            merge_existing=True
        )

        self._merge_validation_messages(
            payload=payload,
            schema=schema
        )

        self._apply_set_ordering(
            schema
        )

        return schema

def _load_payload(
    self,
    text: str
) -> dict:

    try:
        return json.loads(text)

    except json.JSONDecodeError as exc:
        raise ConversionError(
            f"Invalid Phase 2 metadata JSON: {exc}"
        ) from exc

def _parse_metadata_records(
    self,
    payload: dict,
    schema: SchemaModel
) -> None:

    records_payload = (
        payload.get("records")
        or payload.get("tables")
        or payload.get("db2_tables")
    )

```

```

        or []
    )

    for record_json in records_payload:
        record_name = (
            record_json.get("name")
            or record_json.get("record")
            or record_json.get("table")
        )

        if not record_name:
            continue

        record_name = self._normalize_record_name(
            record_name
        )

        primary_key = (
            record_json.get("primary_key")
            or record_json.get("pk")
        )

        primary_key = (
            self._normalize_column_name(primary_key)
            if primary_key
            else None
        )

        fields = {}

        fields_payload = (
            record_json.get("fields")
            or record_json.get("columns")
            or []
        )

        for field_json in fields_payload:
            field_name = (
                field_json.get("name")
                or field_json.get("column")
            )

```

```

    )

    if not field_name:
        continue

    field_name = self._normalize_column_name(
        field_name
    )

    datatype = (
        field_json.get("datatype")
        or field_json.get("type")
        or "CHAR"
    )

    datatype = str(datatype).upper()

    length = field_json.get("length")
    scale = field_json.get("scale")

    nullable = field_json.get("nullable")

    if nullable is None:
        nullable = field_name != primary_key

    fields[field_name] = Column(
        name=field_name,
        datatype=datatype,
        length=length,
        scale=scale,
        nullable=bool(nullable)
    )

    schema.records[record_name] = Record(
        name=record_name,
        primary_key=primary_key,
        fields=fields
    )

    table_name = (

```

```

        record_json.get("table")
        or record_name
    )

    schema.record_table_map[record_name] =
self._normalize_record_name(
    table_name
)

def _parse_metadata_relationships(
    self,
    payload: dict,
    schema: SchemaModel,
    merge_existing: bool
) -> None:

    relationships_payload = (
        payload.get("relationships")
        or payload.get("sets")
        or payload.get("db2_relationships")
        or []
    )

    for relationship_json in relationships_payload:
        set_name = (
            relationship_json.get("set_name")
            or relationship_json.get("name")
            or relationship_json.get("relationship")
        )

        parent_record = (
            relationship_json.get("parent_record")
            or relationship_json.get("parent_table")
        )

        child_record = (
            relationship_json.get("child_record")
            or relationship_json.get("child_table")
        )

```

```
if not set_name:
    continue

if not parent_record:
    continue

if not child_record:
    continue

set_name = self._normalize_set_name(
    set_name
)

parent_record = self._normalize_record_name(
    parent_record
)

child_record = self._normalize_record_name(
    child_record
)

parent_key = (
    relationship_json.get("parent_key")
    or relationship_json.get("parent_pk")
)

child_fk = (
    relationship_json.get("child_fk")
    or relationship_json.get("foreign_key")
)

parent_key = (
    self._normalize_column_name(parent_key)
    if parent_key
    else None
)

child_fk = (
    self._normalize_column_name(child_fk)
    if child_fk
```

```

        else None
    )

    if not parent_key:
        parent_key = self._resolve_parent_key(
            schema=schema,
            parent_record=parent_record
        )

    if not child_fk:
        child_fk = self._resolve_child_fk(
            schema=schema,
            parent_record=parent_record,
            child_record=child_record,
            parent_key=parent_key
        )

    order_by = [
        self._normalize_column_name(value)
        for value in relationship_json.get("order_by", [])
        if value
    ]

    if not order_by and child_fk:
        order_by = [
            child_fk
        ]

    for alias in self._set_name_aliases(set_name):
        self._upsert_relationship(
            schema=schema,
            set_name=alias,
            parent_record=parent_record,
            child_record=child_record,
            parent_key=parent_key,
            child_fk=child_fk,
            cardinality=relationship_json.get(
                "cardinality",
                "1:N"
            ),
        ),

```

```

        order_by=order_by,
        merge_existing=merge_existing
    )

def _upsert_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    parent_record: str,
    child_record: str,
    parent_key: str | None,
    child_fk: str | None,
    cardinality: str | None,
    order_by: list[str],
    merge_existing: bool
) -> None:

    if merge_existing and set_name in schema.relationships:
        relationship = schema.relationships[set_name]

        if parent_record:
            relationship.parent_record = parent_record

        if child_record:
            relationship.child_record = child_record

        if parent_key:
            relationship.parent_key = parent_key

        if child_fk:
            relationship.child_fk = child_fk

        if cardinality:
            relationship.cardinality = cardinality

        if order_by:
            relationship.order_by = order_by

    return

```

```

        schema.relationships[set_name] = Relationship(
            set_name=set_name,
            parent_record=parent_record,
            child_record=child_record,
            cardinality=cardinality or "1:N",
            parent_key=parent_key,
            child_fk=child_fk,
            order_by=order_by
        )

    def _merge_record_table_map(
        self,
        payload: dict,
        schema: SchemaModel
    ) -> None:

        for key, value in payload.get("record_table_map", {}).items():
            schema.record_table_map[str(key).upper()] = (
                str(value).upper()
                if isinstance(value, str)
                else value
            )

    def _merge_field_map(
        self,
        payload: dict,
        schema: SchemaModel
    ) -> None:

        for key, value in payload.get("field_map", {}).items():
            schema.field_map[str(key).upper()] = value

    def _merge_calc_key_map(
        self,
        payload: dict,
        schema: SchemaModel
    ) -> None:

        for key, value in payload.get("calc_key_map", {}).items():
            schema.calc_key_map[str(key).upper()] = value

```



```

def _merge_set_ordering_map(
    self,
    payload: dict,
    schema: SchemaModel
) -> None:

    for key, value in payload.get("set_ordering_map", {}).items():
        for alias in self._set_name_aliases(str(key)):
            schema.set_ordering_map[alias] = value

def _merge_navigation_intent(
    self,
    payload: dict,
    schema: SchemaModel
) -> None:

    for key, value in payload.get("navigation_intent", {}).items():
        for alias in self._set_name_aliases(str(key)):
            schema.navigation_intent[alias] = value

def _merge_nullable_fk_map(
    self,
    payload: dict,
    schema: SchemaModel
) -> None:

    for key, value in payload.get("nullable_fk_map", {}).items():
        schema.nullable_fk_map[str(key).upper()] = value

def _merge_date_part_map(
    self,
    payload: dict,
    schema: SchemaModel
) -> None:

    for key, value in payload.get("date_part_map", {}).items():
        schema.date_part_map[str(key).upper()] = value

def _merge_output_semantics(

```

```

        self,
        payload: dict,
        schema: SchemaModel
    ) -> None:

        output_semantics = payload.get("output_semantics", {})

        if output_semantics:
            schema.output_semantics.update(output_semantics)

    def _merge_paragraph_operation_graph(
        self,
        payload: dict,
        schema: SchemaModel
    ) -> None:

        for key, value in payload.get("paragraph_operation_graph",
{}).items():
            schema.paragraph_operation_graph[str(key).upper()] = value

    def _merge_validation_messages(
        self,
        payload: dict,
        schema: SchemaModel
    ) -> None:

        validation_messages = payload.get("validation_messages", [])

        if validation_messages and hasattr(schema, "validation_messages"):
            schema.validation_messages.extend(validation_messages)

    def _apply_set_ordering(
        self,
        schema: SchemaModel
    ) -> None:

        for set_name, ordering in schema.set_ordering_map.items():
            relationship = schema.relationships.get(set_name)

            if not relationship:

```

```

        continue

    order_by = ordering.get("order_by", [])

    if order_by:
        relationship.order_by = [
            self._normalize_column_name(item)
            for item in order_by
        ]

def _resolve_parent_key(
    self,
    schema: SchemaModel,
    parent_record: str
) -> str | None:

    record = schema.records.get(parent_record)

    if not record:
        return None

    return record.primary_key

def _resolve_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_key: str | None
) -> str | None:

    child = schema.records.get(child_record)

    if not child:
        return None

    if parent_key and parent_key in child.fields:
        return parent_key

    parent = schema.records.get(parent_record)

```

```

        if parent and parent.primary_key and parent.primary_key in
child.fields:
            return parent.primary_key

    return None

def _normalize_record_name(
    self,
    value: str | None
) -> str:

    if not value:
        return ""

    return str(value).strip().upper().replace("-", "_")

def _normalize_column_name(
    self,
    value: str | None
) -> str:

    if not value:
        return ""

    return str(value).strip().upper().replace("-", "_")

def _normalize_set_name(
    self,
    value: str | None
) -> str:

    if not value:
        return ""

    return str(value).strip().upper()

def _set_name_aliases(
    self,
    value: str | None

```

```
) -> list[str]:

    if not value:
        return []

    raw = str(value).strip().upper()
    hyphen = raw.replace("_", "-")
    underscore = raw.replace("-", "_")

    aliases = []

    for item in [
        raw,
        hyphen,
        underscore
    ]:
        if item not in aliases:
            aliases.append(item)

    return aliases
```

src\idms_db2_converter\services\conversion_service.py

```
from copy import deepcopy

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.parsers.cobol_parser import CobolParser
from idms_db2_converter.parsers.canonical_parser import CanonicalParser
from idms_db2_converter.parsers.ddl_parser import DDLParser
from idms_db2_converter.parsers.phase2_metadata_parser import
Phase2MetadataParser
from idms_db2_converter.parsers.idms_schema_parser import IdmsSchemaParser
from idms_db2_converter.services.relationship_resolver import
RelationshipResolver
from idms_db2_converter.services.schema_merger import SchemaMerger
from idms_db2_converter.transformers.retrieval_transformer import
RetrievalTransformer
from idms_db2_converter.validators.output_validator import OutputValidator
from idms_db2_converter.validators.schema_validator import SchemaValidator


class ConversionService:
    DEBUG_SCHEMA = True

    def convert_retrieval(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str | None = None,
        ddl_text: str | None = None,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
    ) -> tuple[str, list[str]]:
        analysis = CobolParser().parse(cobol_text)

        schema = self._build_schema_from_available_sources(
            canonical_json=canonical_json,
            phase2_metadata_json=phase2_metadata_json,
            ddl_text=ddl_text,
            idms_schema_text=idms_schema_text,
```

```

    )

    if self.DEBUG_SCHEMA:
        self._debug_schema(schema)

    if relationship_overrides_json and
relationship_overrides_json.strip():
        schema = RelationshipResolver().apply_overrides(
            schema=schema,
            relationship_overrides_json=relationship_overrides_json,
        )

    schema = RelationshipResolver().resolve_cobol_sets(
        schema=schema,
        analysis=analysis,
    )

    schema_errors = SchemaValidator().validate(schema, analysis)

    if schema_errors:
        raise ConversionError("\n".join(schema_errors))

    converted = RetrievalTransformer(schema, analysis).transform(
        cobol=cobol_text,
        target_program=target_program,
    )

    output_errors = OutputValidator().validate(converted)

    validation_messages = []
    validation_messages.extend(getattr(schema, "validation_messages",
[[]]))
    validation_messages.extend(output_errors)

    return converted, validation_messages

def _build_schema_from_available_sources(
    self,
    canonical_json: str | None,
    phase2_metadata_json: str | None,

```

```

        ddl_text: str | None,
        idms_schema_text: str | None,
    ) -> SchemaModel:
        canonical_has_text = bool(canonical_json and
canonical_json.strip())
        metadata_has_text = bool(phase2_metadata_json and
phase2_metadata_json.strip())
        ddl_has_text = bool(ddl_text and ddl_text.strip())
        idms_schema_has_text = bool(idms_schema_text and
idms_schema_text.strip())

        canonical_schema = SchemaModel()
        metadata_schema = SchemaModel()
        ddl_schema = SchemaModel()
        idms_schema = SchemaModel()

        if canonical_has_text:
            canonical_schema = CanonicalParser().parse(canonical_json)
            canonical_schema.schema_source = "CANONICAL"

        if metadata_has_text:
            metadata_schema = Phase2MetadataParser().parse_as_schema(
                phase2_metadata_json
            )
            metadata_schema.schema_source = "PHASE2_METADATA"

        if ddl_has_text:
            ddl_schema = DDLParser().parse(ddl_text)

            if self._has_schema_objects(ddl_schema):
                ddl_schema.schema_source = "DDL"
            elif self._looks_like_idms_schema(ddl_text):
                idms_schema = IdmsSchemaParser().parse(ddl_text)
                idms_schema.schema_source = "IDMS_SCHEMA"
            elif "CREATE TABLE" in ddl_text.upper():
                raise ConversionError(
                    "DDL text was provided and contains CREATE TABLE, but
no tables were parsed."
                )

```



```

if idms_schema_has_text:
    idms_schema = IdmsSchemaParser().parse(idms_schema_text)
    idms_schema.schema_source = "IDMS_SCHEMA"

if self._has_schema_objects(ddl_schema):
    base_schema = ddl_schema
    base_schema.schema_source = "DDL"

    if self._has_schema_objects(canonical_schema):
        base_schema = self._merge_non_physical_metadata(
            physical_schema=base_schema,
            overlay=canonical_schema,
        )

    if metadata_has_text:
        base_schema = Phase2MetadataParser().parse_into_schema(
            text=phase2_metadata_json,
            schema=base_schema,
        )

    if self._has_schema_objects(idms_schema):
        base_schema = SchemaMerger().merge_physical_with_idms(
            physical_schema=base_schema,
            idms_schema=idms_schema,
        )

    return base_schema

if self._has_schema_objects(canonical_schema):
    base_schema = canonical_schema

    if metadata_has_text:
        base_schema = Phase2MetadataParser().parse_into_schema(
            text=phase2_metadata_json,
            schema=base_schema,
        )

    if self._has_schema_objects(idms_schema):
        base_schema = self._merge_schema(base_schema, idms_schema)

```

```

        return base_schema

    if self._has_schema_objects(metadata_schema):
        base_schema = metadata_schema

        if self._has_schema_objects(idms_schema):
            base_schema = self._merge_schema(base_schema, idms_schema)

        return base_schema

    if self._has_schema_objects(idms_schema):
        return idms_schema

    raise ConversionError(
        "No usable schema source found. Provide canonical model JSON, "
        "Phase 2 metadata JSON with records/relationships, DB2 DDL, "
        "or IDMS schema listing."
    )

def _merge_schema(
    self,
    base: SchemaModel,
    overlay: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(base)

    for record_name, overlay_record in overlay.records.items():
        if record_name in merged.records:
            merged.records[record_name].primary_key = (
                overlay_record.primary_key
                or merged.records[record_name].primary_key
            )

            for field_name, overlay_field in
overlay_record.fields.items():
                merged.records[record_name].fields[field_name] =
overlay_field
            else:
                merged.records[record_name] = overlay_record

```

```

        for set_name, overlay_rel in overlay.relationships.items():
            if set_name not in merged.relationships:
                merged.relationships[set_name] = overlay_rel
                continue

            rel = merged.relationships[set_name]
            rel.parent_record = overlay_rel.parent_record or
rel.parent_record
            rel.child_record = overlay_rel.child_record or
rel.child_record
            rel.parent_key = overlay_rel.parent_key or rel.parent_key
            rel.child_fk = overlay_rel.child_fk or rel.child_fk
            rel.cardinality = overlay_rel.cardinality or rel.cardinality

            if overlay_rel.order_by:
                rel.order_by = overlay_rel.order_by

        self._merge_dict(merged.field_map, overlay.field_map)
        self._merge_dict(merged.record_table_map,
overlay.record_table_map)
        self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
        self._merge_dict(merged.set_ordering_map,
overlay.set_ordering_map)
        self._merge_dict(merged.navigation_intent,
overlay.navigation_intent)
        self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
        self._merge_dict(merged.date_part_map, overlay.date_part_map)
        self._merge_dict(merged.output_semantics,
overlay.output_semantics)
        self._merge_dict(
            merged.paragraph_operation_graph,
            overlay.paragraph_operation_graph,
        )

        if hasattr(merged, "validation_messages") and hasattr(
            overlay,
            "validation_messages",
        ):
            merged.validation_messages.extend(overlay.validation_messages)

```

```

        if not getattr(merged, "schema_source", None):
            merged.schema_source = getattr(overlay, "schema_source", None)

    return merged

def _merge_non_physical_metadata(
    self,
    physical_schema: SchemaModel,
    overlay: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(physical_schema)

    self._merge_dict(merged.field_map, overlay.field_map)

    for key, value in overlay.record_table_map.items():
        if key not in merged.record_table_map:
            merged.record_table_map[key] = value

    self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
    self._merge_dict(merged.set_ordering_map,
overlay.set_ordering_map)
    self._merge_dict(merged.navigation_intent,
overlay.navigation_intent)
    self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
    self._merge_dict(merged.date_part_map, overlay.date_part_map)
    self._merge_dict(merged.output_semantics,
overlay.output_semantics)
    self._merge_dict(
        merged.paragraph_operation_graph,
        overlay.paragraph_operation_graph,
    )

    for set_name, relationship in overlay.relationships.items():
        if set_name not in merged.relationships:
            merged.relationships[set_name] = relationship

    if hasattr(merged, "validation_messages") and hasattr(
        overlay,
        "validation_messages",

```

```

):
    merged.validation_messages.extend(overlay.validation_messages)

merged.schema_source = "DDL+CANONICAL_METADATA"

return merged

def _merge_dict(self, target: dict, source: dict) -> None:
    for key, value in source.items():
        target[key] = value

def _has_schema_objects(self, schema: SchemaModel) -> bool:
    if hasattr(schema, "has_schema_objects"):
        return schema.has_schema_objects()

    return bool(schema.records or schema.relationships)

def _looks_like_idms_schema(self, text: str | None) -> bool:
    if not text:
        return False

    upper = text.upper()

    return (
        "RECORD NAME" in upper
        and "DBKEY POSITIONS" in upper
        and "DATA ITEM" in upper
    )

def _debug_schema(self, schema: SchemaModel) -> None:
    print("FINAL SCHEMA SOURCE:", getattr(schema, "schema_source",
None))

    print("RECORD COUNT:", len(schema.records))
    print("RELATIONSHIP COUNT:", len(schema.relationships))
    print("RECORD TABLE MAP COUNT:", len(schema.record_table_map))
    print("FIELD MAP COUNT:", len(schema.field_map))
    print("DATE PART MAP COUNT:", len(schema.date_part_map))

    for record_name in sorted(schema.records.keys()):
        record = schema.records[record_name]

```

```
print(f"RECORD: {record_name} PK={record.primary_key}")

for field_name in sorted(record.fields.keys()):
    field = record.fields[field_name]
    print(
        "  FIELD:",
        field_name,
        field.datatype,
        field.length,
        field.scale,
        "NULLABLE=",
        field.nullable,
    )

print("RELATIONSHIPS:")

for set_name in sorted(schema.relationships.keys()):
    rel = schema.relationships[set_name]
    print(
        "  REL:",
        set_name,
        rel.parent_record,
        rel.parent_key,
        "->",
        rel.child_record,
        rel.child_fk,
    )
```

src\idms_db2_converter\services\relationship_resolver.py

```
import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import CobolAnalysis, Relationship,
SchemaModel

class RelationshipResolver:
    """
    Resolves COBOL IDMS set names to schema relationships.

    No business-specific set names are hard-coded.

    Resolution order:
    1. Use exact set name if already present.
    2. Use relationship overrides if provided.
    3. Use DDL-derived relationships if a COBOL set can be matched
    unambiguously.
    """

    def apply_overrides(
        self,
        schema: SchemaModel,
        relationship_overrides_json: str,
    ) -> SchemaModel:
        try:
            payload = json.loads(relationship_overrides_json)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid relationship override JSON:
{exc}") from exc

        self._apply_record_aliases(schema, payload)

        for item in payload.get("relationships", []):
            self._apply_relationship_override(schema, item)

        return schema

    def resolve_cobol_sets(
```

```

        self,
        schema: SchemaModel,
        analysis: CobolAnalysis,
    ) -> SchemaModel:
        references = self._collect_set_references(analysis)

        for set_name, child_records in references.items():
            if set_name in schema.relationships:
                continue

            matched = self._find_unambiguous_relationship(
                schema=schema,
                set_name=set_name,
                child_records=child_records,
            )

            if not matched:
                continue

            schema.relationships[set_name] = Relationship(
                set_name=set_name,
                parent_record=matched.parent_record,
                child_record=matched.child_record,
                cardinality=matched.cardinality,
                parent_key=matched.parent_key,
                child_fk=matched.child_fk,
                order_by=matched.order_by,
            )

            schema.add_validation_message(
                f"Set {set_name} resolved from existing schema "
                f"relationship "
                f"{matched.parent_record}-{matched.child_record}."
            )

        return schema

    def _collect_set_references(
        self,
        analysis: CobolAnalysis,

```



```

) -> dict[str, set[str]]:
    references: dict[str, set[str]] = {}

    for record_name, set_name in analysis.obtain_next:
        references.setdefault(set_name, set()).add(record_name)

    for set_name in analysis.obtain_owner_sets:
        references.setdefault(set_name, set())

    for set_name in analysis.find_first_sets:
        references.setdefault(set_name, set())

    return references

def _apply_record_aliases(self, schema: SchemaModel, payload: dict) ->
None:
    for item in payload.get("record_aliases", []):
        record_name = item["record"].upper()
        table_name = item["table"].upper()

        if table_name not in schema.records:
            raise ConversionError(
                f"Record alias {record_name} references unknown table
{table_name}."
            )

        schema.records[record_name] = schema.records[table_name]
        schema.record_table_map[record_name] = table_name

def _apply_relationship_override(self, schema: SchemaModel, item:
dict) -> None:
    set_name = item["set_name"].upper()

    parent_record = self._upper_or_none(item.get("parent_record"))
    child_record = self._upper_or_none(item.get("child_record"))
    parent_key = self._upper_or_none(item.get("parent_key"))
    child_fk = self._upper_or_none(item.get("child_fk"))
    order_by = [value.upper() for value in item.get("order_by", [])]

    if set_name in schema.relationships:

```

```

        rel = schema.relationships[set_name]

        if parent_record:
            rel.parent_record = parent_record

        if child_record:
            rel.child_record = child_record

        if parent_key:
            rel.parent_key = parent_key

        if child_fk:
            rel.child_fk = child_fk

        if order_by:
            rel.order_by = order_by

        return

    if parent_record and child_record:
        self._create_explicit_relationship(
            schema=schema,
            set_name=set_name,
            parent_record=parent_record,
            child_record=child_record,
            parent_key=parent_key,
            child_fk=child_fk,
            order_by=order_by,
        )
        return

    matched = self._find_relationship_by_keys(
        schema=schema,
        parent_key=parent_key,
        child_fk=child_fk,
    )

    if matched:
        schema.relationships[set_name] = Relationship(
            set_name=set_name,

```

```

        parent_record=matched.parent_record,
        child_record=matched.child_record,
        cardinality=matched.cardinality,
        parent_key=parent_key or matched.parent_key,
        child_fk=child_fk or matched.child_fk,
        order_by=order_by or matched.order_by,
    )
    return

    raise ConversionError(
        f"Override references set {set_name}, but it could not be
resolved. "
        "Provide parent_record and child_record, or provide keys
matching a DDL foreign key."
    )

def _create_explicit_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    parent_record: str,
    child_record: str,
    parent_key: str | None,
    child_fk: str | None,
    order_by: list[str],
) -> None:
    if parent_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} parent_record {parent_record} is not
present in schema."
        )

    if child_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} child_record {child_record} is not
present in schema."
        )

    parent = schema.records[parent_record]
    child = schema.records[child_record]

```

```

resolved_parent_key = parent_key or parent.primary_key

if not resolved_parent_key:
    raise ConversionError(
        f"Set {set_name} cannot resolve parent_key."
    )

resolved_child_fk = child_fk

if not resolved_child_fk and resolved_parent_key in child.fields:
    resolved_child_fk = resolved_parent_key

if not resolved_child_fk:
    raise ConversionError(
        f"Set {set_name} cannot resolve child_fk."
    )

schema.relationships[set_name] = Relationship(
    set_name=set_name,
    parent_record=parent_record,
    child_record=child_record,
    cardinality="1:N",
    parent_key=resolved_parent_key,
    child_fk=resolved_child_fk,
    order_by=order_by or [resolved_child_fk],
)

def _find_relationship_by_keys(
    self,
    schema: SchemaModel,
    parent_key: str | None,
    child_fk: str | None,
) -> Relationship | None:
    if not parent_key and not child_fk:
        return None

    matches = []

    for rel in schema.relationships.values():

```

```

        if parent_key and rel.parent_key != parent_key:
            continue

        if child_fk and rel.child_fk != child_fk:
            continue

        matches.append(rel)

    if len(matches) == 1:
        return matches[0]

    if len(matches) > 1:
        raise ConversionError(
            "Relationship override is ambiguous. Add parent_record and
child_record."
        )

    return None

def _find_unambiguous_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    child_records: set[str],
) -> Relationship | None:
    candidates = list(schema.relationships.values())

    if child_records:
        filtered = [
            rel
            for rel in candidates
            if rel.child_record in child_records
        ]

        if filtered:
            candidates = filtered

    if len(candidates) == 1:
        return candidates[0]

```

```

scored = []

for rel in candidates:
    score = self._score_relationship(set_name, rel)

    if score > 0:
        scored.append((score, rel))

if not scored:
    return None

scored.sort(key=lambda item: item[0], reverse=True)

top_score = scored[0][0]
top_matches = [
    rel
    for score, rel in scored
    if score == top_score
]

if len(top_matches) == 1:
    return top_matches[0]

return None

def _score_relationship(self, set_name: str, rel: Relationship) ->
int:
    tokens = [
        token
        for token in set_name.replace("_", "-").split("-")
        if token
    ]

    searchable_values = [
        rel.parent_record,
        rel.child_record,
        rel.parent_key or "",
        rel.child_fk or "",
    ]

```

```
score = 0

for token in tokens:
    for value in searchable_values:
        if self._token_matches(token, value):
            score += 1

return score

def _token_matches(self, token: str, value: str) -> bool:
    token = token.upper()
    value = value.upper()

    return token in value or value in token

def _upper_or_none(self, value: str | None) -> str | None:
    return value.upper() if value else None
```

src\idms_db2_converter\services\schema_merger.py

```
from copy import deepcopy
import re

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Relationship, SchemaModel

class SchemaMerger:
    """
    Merge DB2 DDL schema with IDMS schema listing.

    Rules:
    - DDL wins for physical column definitions.
    - IDMS wins for legacy field names, IDMS set names, and field_map.
    """

    def merge_physical_with_idms(
        self,
        physical_schema: SchemaModel,
        idms_schema: SchemaModel,
    ) -> SchemaModel:
        merged = deepcopy(physical_schema)

        if not getattr(merged, "schema_source", None):
            merged.schema_source = "DDL"

        merged.schema_source = f"{merged.schema_source}+IDMS_SCHEMA"

        self._build_record_table_map(merged, idms_schema)
        self._build_field_map_from_idms_to_physical(merged, idms_schema)
        self._build_date_part_map_from_idms_to_physical(merged,
idms_schema)

        self._preserve_idms_set_names(merged, idms_schema)
        self._merge_validation_messages(merged, idms_schema)

        return merged

    def _build_record_table_map(
        self,
```



```

        merged: SchemaModel,
        idms_schema: SchemaModel,
    ) -> None:
        for idms_record_name in idms_schema.records:
            physical_record_name = self._find_matching_record(
                schema=merged,
                record_name=idms_record_name,
            )

            if physical_record_name:
                merged.record_table_map[idms_record_name] =
physical_record_name
            else:
                merged.record_table_map[idms_record_name] =
self._table_name(
                    idms_record_name
                )

    def _build_field_map_from_idms_to_physical(
        self,
        merged: SchemaModel,
        idms_schema: SchemaModel,
    ) -> None:
        for idms_record_name in idms_schema.records:
            physical_record_name =
merged.record_table_map.get(idms_record_name)

            if not physical_record_name:
                continue

            if physical_record_name not in merged.records:
                continue

            physical_record = merged.records[physical_record_name]

            for idms_field_name, meta in idms_schema.field_map.items():
                idms_host = str(meta.get("host", "")).upper()

                if not self._host_belongs_to_record(idms_host,
idms_record_name):

```

```

        continue

        normalized_idms_field =
self._normalize_field_name(idms_field_name)

        physical_column = self._find_physical_column(
            physical_record=physical_record,
            normalized_idms_field=normalized_idms_field,
            idms_host=idms_host,
            idms_record_name=idms_record_name,
        )

        if not physical_column:
            continue

        merged.field_map[idms_field_name.upper()] = {
            "host": Naming.hv(idms_record_name, physical_column)
        }

def _build_date_part_map_from_idms_to_physical(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_field_name, meta in idms_schema.date_part_map.items():
        idms_host = str(meta.get("host", "")).upper()

        idms_record_name = self._find_record_for_host(
            host=idms_host,
            record_names=list(idms_schema.records.keys()),
        )

        if not idms_record_name:
            continue

        physical_record_name =
merged.record_table_map.get(idms_record_name)

        if not physical_record_name:
            continue

```

```

        if physical_record_name not in merged.records:
            continue

        physical_record = merged.records[physical_record_name]

        idms_column = self._column_from_host(idms_host,
idms_record_name)

        if not idms_column:
            continue

        physical_column = self._find_physical_column(
            physical_record=physical_record,
            normalized_idms_field=idms_column,
            idms_host=idms_host,
            idms_record_name=idms_record_name,
        )

        if not physical_column:
            continue

        column = physical_record.fields[physical_column]

        substring_start = meta.get("substring_start")
        substring_length = meta.get("substring_length")

        if column.datatype.upper() == "DATE":
            part = self._date_part_name(idms_field_name)

            if part == "YEAR":
                substring_start = 3
                substring_length = 2
            elif part == "MONTH":
                substring_start = 6
                substring_length = 2
            elif part == "DAY":
                substring_start = 9
                substring_length = 2

```

```

merged.date_part_map[idms_field_name.upper()] = {
    "host": Naming.hv(idms_record_name, physical_column),
    "substring_start": substring_start,
    "substring_length": substring_length,
}

def _preserve_idms_set_names(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for set_name, idms_rel in idms_schema.relationships.items():
        idms_parent = idms_rel.parent_record
        idms_child = idms_rel.child_record

        physical_parent = merged.record_table_map.get(idms_parent)
        physical_child = merged.record_table_map.get(idms_child)

        if not physical_parent or not physical_child:
            continue

        ddl_rel = self._find_relationship_by_records(
            schema=merged,
            parent_record=physical_parent,
            child_record=physical_child,
        )

        if ddl_rel:
            parent_key = ddl_rel.parent_key
            child_fk = ddl_rel.child_fk
            order_by = ddl_rel.order_by
        else:
            parent_key = self._find_matching_column_name(
                record=merged.records[physical_parent],
                name=idms_rel.parent_key or "",
            )

            child_fk = self._find_matching_column_name(
                record=merged.records[physical_child],
                name=idms_rel.child_fk or idms_rel.parent_key or "",
            )

```

```

    )

    order_by = [child_fk] if child_fk else []

    if not parent_key or not child_fk:
        continue

    merged.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=idms_parent,
        child_record=idms_child,
        cardinality=idms_rel.cardinality or "1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=order_by or [child_fk],
    )

def _find_physical_column(
    self,
    physical_record,
    normalized_idms_field: str,
    idms_host: str,
    idms_record_name: str,
) -> str | None:
    direct = self._find_matching_column_name(
        record=physical_record,
        name=normalized_idms_field,
    )

    if direct:
        return direct

    from_host = self._column_from_host(idms_host, idms_record_name)

    if from_host:
        direct = self._find_matching_column_name(
            record=physical_record,
            name=from_host,
        )

```

```

        if direct:
            return direct

singular = self._singularize(normalized_idms_field)

if singular != normalized_idms_field:
    direct = self._find_matching_column_name(
        record=physical_record,
        name=singular,
    )

    if direct:
        return direct

return self._find_flattened_occurs_column(
    physical_record=physical_record,
    name=normalized_idms_field,
)

def _find_flattened_occurs_column(
    self,
    physical_record,
    name: str,
) -> str | None:
    candidates = [
        column_name
        for column_name in physical_record.fields
        if column_name.upper().startswith(name.upper() + "_")
    ]

    if not candidates:
        singular = self._singularize(name)
        candidates = [
            column_name
            for column_name in physical_record.fields
            if column_name.upper().startswith(singular.upper() + "_")
        ]

    if not candidates:
        return None

```

```

        return sorted(candidates)[0]

def _find_matching_record(
    self,
    schema: SchemaModel,
    record_name: str,
) -> str | None:
    record_name = record_name.upper()

    if record_name in schema.records:
        return record_name

    normalized = self._normalize_record_name(record_name)

    for candidate in schema.records:
        if self._normalize_record_name(candidate) == normalized:
            return candidate

    return None

def _find_matching_column_name(
    self,
    record,
    name: str,
) -> str | None:
    normalized = self._normalize_field_name(name)

    if normalized in record.fields:
        return normalized

    for column_name in record.fields:
        if column_name.upper() == normalized:
            return column_name

    return None

def _find_relationship_by_records(
    self,
    schema: SchemaModel,

```

```

        parent_record: str,
        child_record: str,
    ) -> Relationship | None:
        normalized_parent = self._normalize_record_name(parent_record)
        normalized_child = self._normalize_record_name(child_record)

        for relationship in schema.relationships.values():
            if (
                self._normalize_record_name(relationship.parent_record)
                == normalized_parent
                and self._normalize_record_name(relationship.child_record)
                == normalized_child
            ):
                return relationship

        return None

def _host_belongs_to_record(
    self,
    host: str,
    record_name: str,
) -> bool:
    normalized_host = self._normalize_token(host)
    normalized_record = self._normalize_token(record_name)

    return normalized_host.startswith(f"HV_{normalized_record}_")

def _find_record_for_host(
    self,
    host: str,
    record_names: list[str],
) -> str | None:
    normalized_host = self._normalize_token(host)

    for record_name in sorted(record_names, key=len, reverse=True):
        normalized_record = self._normalize_token(record_name)

        if normalized_host.startswith(f"HV_{normalized_record}_"):
            return record_name

```



```

        return None

    def _column_from_host(
        self,
        host: str,
        record_name: str,
    ) -> str | None:
        normalized_host = self._normalize_token(host)
        normalized_record = self._normalize_token(record_name)

        prefix = f"HV_{normalized_record}_"

        if not normalized_host.startswith(prefix):
            return None

        return normalized_host[len(prefix):].upper()

    def _date_part_name(
        self,
        idms_field_name: str,
    ) -> str | None:
        normalized = self._normalize_field_name(idms_field_name)

        if normalized.endswith("_YEAR"):
            return "YEAR"

        if normalized.endswith("_MONTH"):
            return "MONTH"

        if normalized.endswith("_DAY"):
            return "DAY"

        return None

    def _normalize_token(
        self,
        value: str,
    ) -> str:
        return value.upper().replace("-", "_")

```

```

def _normalize_record_name(
    self,
    value: str,
) -> str:
    return value.upper().replace("-", "_")

def _normalize_field_name(
    self,
    value: str,
) -> str:
    value = value.upper()
    value = re.sub(r"\d{4}$", "", value)
    return value.replace("-", "_")

def _singularize(
    self,
    value: str,
) -> str:
    value = value.upper()

    if value.endswith("IES"):
        return value[:-3] + "Y"

    if value.endswith("S"):
        return value[:-1]

    return value

def _table_name(
    self,
    record_name: str,
) -> str:
    return record_name.upper().replace("-", "_")

def _merge_validation_messages(
    self,
    merged: SchemaModel,
    overlay: SchemaModel,
) -> None:
    if hasattr(merged, "validation_messages") and hasattr(

```

```
        overlay,  
        "validation_messages",  
    ):  
        merged.validation_messages.extend(overlay.validation_messages)
```

src\idms_db2_converter\transformers\cobol_cleanup.py

```
import re

class CobolCleanup:
    """
    Final cleanup pass for generated COBOL.

    Important design:
    - Do not blindly add periods to every MOVE / PERFORM / READ.
      In COBOL, periods can terminate IF/ELSE scope incorrectly.
    - Preserve EXEC SQL blocks exactly enough to remain valid.
    - Repair known bad patterns created during transformation.
    """

    EXEC_SQL_START = re.compile(
        r"^\s*EXEC\s+SQL\b",
        re.IGNORECASE
    )

    EXEC_SQL_END = re.compile(
        r"^\s*END-EXEC\.\?\s*$",
        re.IGNORECASE
    )

    PARAGRAPH_HEADER = re.compile(
        r"^\s*[A-Z0-9-]+\.\.\s*$",
        re.IGNORECASE
    )

    def clean(
        self,
        text: str
    ) -> str:

        text = self.normalize_sqlcode_then(text)
        text = self.repair_read_at_end(text)
        text = self.remove_end_processing_sqlcode_guard(text)
        text = self.fix_paragraph_header_concatenation(text)
```

```

        text = self.fix_end_if_same_line_statement(text)
        text = self.ensure_final_end_if_periods(text)
        text = self.normalize_exec_sql_indentation(text)
        text = self.normalize_close_file_indentation(text)
        text = self.normalize_basic_indentation(text)
        text = self.remove_duplicate_blank_lines(text)

    return text

def normalize_sqlcode_then(
    self,
    text: str
) -> str:

    text = re.sub(
        r"IF\s+SQLCODE\s*=\s*100\s+THEN",
        "IF SQLCODE = 100",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"IF\s+SQLCODE\s+NOT\s*=\s*0\s+THEN",
        "IF SQLCODE NOT = 0",
        text,
        flags=re.IGNORECASE
    )

    return text

def repair_read_at_end(
    self,
    text: str
) -> str:
    """
    Repair:
        READ FILE.
        AT END MOVE 'Y' TO EOF-SW.

    To:

```

```

        READ FILE
        AT END MOVE 'Y' TO EOF-SW.
    """

    text = re.sub(
        r"\n(\s*)READ\s+([A-Z0-9-]+)\.\s*\n\s*AT\s+END\s+",
        r"\n\1READ \2\n\1    AT END ",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"\n(\s*)READ\s+([A-Z0-9-]+)\s*\n\s*AT\s+END\s+",
        r"\n\1READ \2\n\1    AT END ",
        text,
        flags=re.IGNORECASE
    )

    return text

def remove_end_processing_sqlcode_guard(
    self,
    text: str
) -> str:

    return re.sub(
        r"""
        (\n\s*END-PROCESSING\.\s*)
        \n\s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
        \n\s*PERFORM\s+SQL-ERROR\s*
        \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE
    )

def fix_paragraph_header_concatenation(
    self,

```

```

        text: str
    ) -> str:

        text = re.sub(
            r"(\b[A-Z0-9-]+\.)\s*(IF\s+SQLCODE) ",
            r"\1\n          \2",
            text,
            flags=re.IGNORECASE
        )

        text = re.sub(
            r"(\b[A-Z0-9-]+\.)\s*(EXEC\s+SQL) ",
            r"\1\n          \2",
            text,
            flags=re.IGNORECASE
        )

        text = re.sub(
            r"(\b[A-Z0-9-]+\.)\s*(MOVE\s+) ",
            r"\1\n          \2",
            text,
            flags=re.IGNORECASE
        )

        return text

def fix_end_if_same_line_statement(
    self,
    text: str
) -> str:

    text = re.sub(
        r"END-IF\.\s+(MOVE\s+) ",
        "END-IF.\n          MOVE ",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"END-IF\.\s+(PERFORM\s+) ",

```

```

        "END-IF.\n          PERFORM ",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"END-IF\.\s+(EXEC\s+SQL) ",
        "END-IF.\n          EXEC SQL",
        text,
        flags=re.IGNORECASE
    )

    return text

def ensure_final_end_if_periods(
    self,
    text: str
) -> str:

    lines = text.splitlines()
    result = []

    for line in lines:
        stripped = line.strip()

        if stripped.upper() == "END-IF":
            leading = line[:len(line) - len(line.lstrip())]

            result.append(
                f"{leading}END-IF."
            )
        else:
            result.append(line)

    return "\n".join(result)

def normalize_exec_sql_indentation(
    self,
    text: str
) -> str:

```



```

lines = text.splitlines()
result = []
inside_exec_sql = False

for line in lines:
    stripped = line.strip()

    if self.EXEC_SQL_START.match(line):
        result.append(
            "      EXEC SQL"
        )
        inside_exec_sql = True
        continue

    if inside_exec_sql:
        if self.EXEC_SQL_END.match(line):
            result.append(
                "      END-EXEC."
            )
            inside_exec_sql = False
            continue

        if stripped:
            result.append(
                "      " + stripped
            )
        else:
            result.append("")

        continue

    result.append(line)

return "\n".join(result)

def normalize_close_file_indentation(
    self,
    text: str
) -> str:

```

```

    return re.sub(
        r"\n\s*CLOSE\s+([A-Z0-9-]+)\.",
        r"\n        CLOSE \1.",
        text,
        flags=re.IGNORECASE
    )

def normalize_basic_indentation(
    self,
    text: str
) -> str:
    """
    Adds light indentation to obvious executable statements,
    but does not add periods.
    """

    lines = text.splitlines()
    result = []
    inside_exec_sql = False

    executable_starters = (
        "MOVE ",
        "PERFORM ",
        "IF ",
        "ELSE",
        "END-IF",
        "END-PERFORM",
        "READ ",
        "WRITE ",
        "OPEN ",
        "CLOSE ",
        "CONTINUE",
        "GOBACK",
        "EXIT",
    )

    for line in lines:
        stripped = line.strip()

```

```

        if self.EXEC_SQL_START.match(line):
            inside_exec_sql = True
            result.append(line)
            continue

        if inside_exec_sql:
            result.append(line)

            if self.EXEC_SQL_END.match(line):
                inside_exec_sql = False

            continue

        if not stripped:
            result.append("")
            continue

        if stripped.startswith("*"):
            result.append(line)
            continue

        if self.PARAGRAPH_HEADER.match(line):
            result.append(stripped)
            continue

        upper = stripped.upper()

        if upper.startswith(executable_starters):
            result.append(
                " " + stripped
            )
        else:
            result.append(line)

    return "\n".join(result)

def remove_duplicate_blank_lines(
    self,
    text: str
) -> str:

```

```
return re.sub(  
    r"\n{3,}",  
    "\n\n",  
    text  
)
```

src\idms_db2_converter\transformers\field_rewriter.py

```
import re

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel


class FieldRewriter:
    """
    Rewrites IDMS schema field references to DB2 host variables.

    Important rules:
    - Do NOT rewrite inside EXEC SQL ... END-EXEC blocks.
    - Do NOT rewrite comment lines.
    - Do NOT rewrite FILE SECTION / WORKING-STORAGE declarations.
    - Rewrite only PROCEDURE DIVISION executable COBOL lines.
    - Rewrite date parts using date_part_map.
    - Rewrite normal IDMS fields using field_map.

    This prevents bad SQL such as:
        SELECT HV-JOB-TITLE-0440_0440

    SQL column names must remain physical DB2 column names.
    """

    PROCEDURE_DIVISION = re.compile(
        r"^\s*PROCEDURE\s+DIVISION\.",
        re.IGNORECASE | re.MULTILINE
    )

    EXEC_SQL_START = re.compile(
        r"^\s*EXEC\s+SQL\b",
        re.IGNORECASE
    )

    EXEC_SQL_END = re.compile(
        r"^\s*END-EXEC\.\?\s*$",
        re.IGNORECASE
    )
```

```

COMMENT_LINE = re.compile(
    r"^\s*\*"
)

def __init__(
    self,
    schema: SchemaModel
):
    self.schema = schema

def rewrite(
    self,
    text: str
) -> str:

    before_procedure, procedure = self._split_procedure_division(
        text
    )

    if not procedure:
        return text

    procedure = self._rewrite_procedure_only(
        procedure
    )

    return before_procedure + procedure

def _split_procedure_division(
    self,
    text: str
) -> tuple[str, str]:

    match = self.PROCEDURE_DIVISION.search(
        text
    )

    if not match:
        return text, ""

```

```

        return (
            text[:match.start()],
            text[match.start():]
        )

def _rewrite_procedure_only(
    self,
    procedure: str
) -> str:

    lines = procedure.splitlines()
    output_lines: list[str] = []
    inside_exec_sql = False

    for line in lines:
        if self.EXEC_SQL_START.search(line):
            inside_exec_sql = True
            output_lines.append(line)
            continue

        if inside_exec_sql:
            output_lines.append(line)

            if self.EXEC_SQL_END.search(line):
                inside_exec_sql = False

            continue

        if self.COMMENT_LINE.search(line):
            output_lines.append(line)
            continue

        rewritten = self._rewrite_date_part_line(line)
        rewritten = self._rewrite_field_line(rewritten)

        output_lines.append(rewritten)

    return "\n".join(output_lines)

```

```

def _rewrite_date_part_line(
    self,
    line: str
) -> str:

    for idms_field in sorted(
        self.schema.date_part_map.keys(),
        key=len,
        reverse=True
    ):
        meta = self.schema.date_part_map.get(
            idms_field
        )

        if not meta:
            continue

        host = self._host_from_meta(meta)

        if not host:
            continue

        substring_start = meta.get(
            "substring_start"
        )

        substring_length = meta.get(
            "substring_length"
        )

        if not substring_start or not substring_length:
            continue

        replacement = (
            f"{host}({substring_start}:{substring_length})"
        )

        for alias in self._hyphen_aliases(idms_field):
            pattern = re.compile(
                rf"(?<![A-Z0-9-]){re.escape(alias)}(?![A-Z0-9-])",

```



```

        re.IGNORECASE
    )

    line = self._replace_outside_literals(
        line=line,
        pattern=pattern,
        replacement=replacement
    )

    return line

def _rewrite_field_line(
    self,
    line: str
) -> str:

    for idms_field in sorted(
        self.schema.field_map.keys(),
        key=len,
        reverse=True
    ):
        if idms_field.upper() in self.schema.date_part_map:
            continue

        meta = self.schema.field_map.get(
            idms_field
        )

        if not meta:
            continue

        host = self._host_from_meta(meta)

        if not host:
            continue

        for alias in self._hyphen_aliases(idms_field):
            pattern = re.compile(
                rf"(?

```

```

        )

        line = self._replace_outside_literals(
            line=line,
            pattern=pattern,
            replacement=host
        )

    return line

def _host_from_meta(
    self,
    meta: dict
) -> str | None:

    record = (
        meta.get("record")
        or meta.get("table")
    )

    column = meta.get(
        "column"
    )

    if record and column:
        return Naming.hv(
            record,
            column
        )

    host = meta.get(
        "host"
    )

    if host:
        return str(host).upper()

    return None

def _hyphen_aliases(

```

```

        self,
        field_name: str
    ) -> list[str]:

        upper = field_name.upper()

        aliases = [
            upper.replace("_", "-")
        ]

        unique_aliases: list[str] = []

        for alias in aliases:
            if alias not in unique_aliases:
                unique_aliases.append(alias)

        return unique_aliases

def _replace_outside_literals(
    self,
    line: str,
    pattern: re.Pattern,
    replacement: str
) -> str:

    parts = re.split(
        r"('(?:[^']|'')*')",
        line
    )

    for index, part in enumerate(parts):
        if index % 2 == 1:
            continue

        parts[index] = pattern.sub(
            replacement,
            part
        )

    return "".join(parts)

```

src\idms_db2_converter\transformers\metadata_helpers.py

```
from idms_db2_converter.generators.cobol_builder import Move
from idms_db2_converter.models import SchemaModel

class MetadataHelpers:
    def __init__(self, schema: SchemaModel):
        self.schema = schema

    def lookup_by_name(self, name: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("name", "").upper() == name.upper():
                return item

        return None

    def lookup_by_owner_set(self, owner_set: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("owner_set", "").upper() == owner_set.upper():
                return item

        return None

    def lookup_by_first_member_set(self, first_member_set: str) -> dict |
None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("first_member_set", "").upper() ==
first_member_set.upper():
                return item

        return None

    def moves_to_nodes(self, moves: list[dict]) -> list[Move]:
        nodes = []

        for move in moves:
            source = self.map_source(move["from"])
            target = move["to"]
            nodes.append(Move(source=source, target=target))
```

```
        return nodes

    def map_source(self, source: str) -> str:
        source_upper = source.upper()

        if source_upper in self.schema.date_part_map:
            meta = self.schema.date_part_map[source_upper]
            return
            f"{meta['host']}({meta['substring_start']}:{meta['substring_length']})"

        if source_upper in self.schema.field_map:
            return self.schema.field_map[source_upper].get("host", source)

        return source
```

src\idms_db2_converter\transformers\operation_graph_transformer.py

```
import re

from idms_db2_converter.generators.cobol_builder import (
    Continue,
    ExecSql,
    IfBlock,
    Move,
    Perform,
    PerformUntil,
    PreRenderedBlock,
    ReadAtEndMove,
    Write,
    render_nodes,
    render_paragraph,
)

from idms_db2_converter.generators.fetch_paragraphs import
FetchParagraphGenerator

from idms_db2_converter.generators.naming import Naming

from idms_db2_converter.generators.sql_snippets import SqlSnippets

from idms_db2_converter.models import SchemaModel

from idms_db2_converter.transformers.metadata_helpers import
MetadataHelpers


class OperationGraphTransformer:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.meta = MetadataHelpers(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.used_cursor_sets: set[str] = set()

    def has_graph(self) -> bool:
        return bool(self.schema.paragraph_operation_graph)

    def transform(self, text: str) -> str:
        for paragraph_name, operations in
self.schema.paragraph_operation_graph.items():
            rendered = self._render_paragraph(paragraph_name, operations)
```

```

        text = self._replace_paragraph(text, paragraph_name, rendered)

    return text

    def _replace_paragraph(self, text: str, paragraph_name: str,
replacement: str) -> str:
        pattern = re.compile(
rf"^\\s*{re.escape(paragraph_name)}\\.\\.s*\\n.*?(?=^\\s*[A-Z0-9-]+\\.\\.s*$)",
        re.IGNORECASE | re.MULTILINE | re.DOTALL,
        )

        if pattern.search(text):
            return pattern.sub(replacement + "\\n", text, count=1)

        return text + "\\n" + replacement + "\\n"

    def _render_paragraph(self, paragraph_name: str, operations:
list[dict]) -> str:
        nodes = self._render_operations(operations)
        return render_paragraph(paragraph_name, nodes)

    def _render_operations(self, operations: list[dict]) -> list:
        nodes = []

        for operation in operations:
            op_type = operation.get("type", "").upper()

            if op_type == "MOVE":
                nodes.append(
                    Move(
                        source=self.meta.map_source(operation["from"]),
                        target=operation["to"],
                    )
                )

            elif op_type == "MOVE_DATE_PART":
                nodes.append(
                    Move(
                        source=self.meta.map_source(operation["from"]),

```

```

        target=operation["to"],
    )
)

elif op_type == "PERFORM":
    nodes.append(Perform(paragraph=operation["paragraph"]))

elif op_type == "WRITE":
    nodes.append(
        Write(
            record=operation["record"],
            source=operation.get("source"),
        )
    )

elif op_type == "READ_NEXT_INPUT":
    at_end_move = operation.get("at_end_move", {})
    nodes.append(
        ReadAtEndMove(
            file_name=operation["file"],
            source=at_end_move.get("from", "Y"),
            target=at_end_move.get("to", "EOF-SW"),
        )
    )

elif op_type == "CONTINUE":
    nodes.append(Continue())

elif op_type == "OBTAIN_CALC":
    nodes.append(self._node_obtain_calc(operation))

elif op_type == "CURSOR_LOOP":
    nodes.append(self._node_cursor_loop(operation))

elif op_type == "FIRST_MEMBER_TO_OWNER_LOOKUP":
nodes.extend(self._nodes_first_member_to_owner_lookup(operation))

elif op_type == "OWNER_LOOKUP":
    nodes.extend(self._nodes_owner_lookup(operation))

```



```

        return nodes

    def _node_obtain_calc(self, operation: dict):
        record = operation["record"].upper()
        select_sql = self.sql.select_for_record_by_pk(record)

        not_found_nodes =
self._render_operations(operation.get("not_found", []))
        success_nodes = self._render_operations(operation.get("success",
[]))

        lines = []
        lines.extend(ExecSql(select_sql).render(indent=7))
        lines.append("")
        lines.extend(
            IfBlock(
                condition="SQLCODE = 100",
                then_body=not_found_nodes,
                else_body=[
                    IfBlock(
                        condition="SQLCODE NOT = 0",
                        then_body=[Perform("SQL-ERROR")],
                        else_body=success_nodes,
                    )
                ],
            ).render(indent=7, final=True)
        )

        return PreRenderedBlock(lines)

    def _node_cursor_loop(self, operation: dict):
        set_name = operation["set"].upper()
        self.used_cursor_sets.add(set_name)

        cursor_name = Naming.cursor(set_name)
        fetch_paragraph = self.fetch_generator.paragraph_name(set_name)

        process_paragraph = operation["process_paragraph"]
        exit_paragraph = operation.get("exit_paragraph")

```

```

        empty_nodes = self._render_operations(operation.get("empty", []))
        before_loop_nodes =
self._render_operations(operation.get("before_loop", []))

        perform_target = process_paragraph
        if exit_paragraph:
            perform_target = f"{process_paragraph} THRU {exit_paragraph}"

        cursor_open = ExecSql(
            "\n".join(
                [
                    "EXEC SQL",
                    f"      OPEN {cursor_name}",
                    "END-EXEC.",
                ]
            )
        )

        cursor_close = ExecSql(
            "\n".join(
                [
                    "EXEC SQL",
                    f"      CLOSE {cursor_name}",
                    "END-EXEC.",
                ]
            )
        )

        loop_nodes = before_loop_nodes + [
            PerformUntil(
                condition="SQLCODE = 100",
                body=[
                    Perform(perform_target),
                    Perform(fetch_paragraph),
                ],
            )
        ]

        return IfBlock(
            condition="SQLCODE NOT = 0",

```

```

        then_body=[Perform("SQL-ERROR")],
        else_body=[
            Perform(fetch_paragraph),
            IfBlock(
                condition="SQLCODE = 100",
                then_body=empty_nodes,
                else_body=loop_nodes,
            ),
            cursor_close,
        ],
    ).with_prefix(cursor_open)

def _nodes_first_member_to_owner_lookup(self, operation: dict) ->
list:
    lookup_name = operation.get("name")
    first_member_set = operation["first_member_set"].upper()
    owner_set = operation["owner_set"].upper()

    lookup = self.meta.lookup_by_name(lookup_name) if lookup_name else
None
    if not lookup:
        lookup =
self.meta.lookup_by_first_member_set(first_member_set)

    not_found_moves = lookup.get("not_found_moves", []) if lookup else
[]
    success_moves = lookup.get("success_moves", []) if lookup else []

    first_select =
ExecSql(self.sql.select_first_child_for_set(first_member_set))
    owner_select = ExecSql(self.sql.select_for_owner(owner_set))

    not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
    owner_not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
    success_nodes = self.meta.moves_to_nodes(success_moves) or
[Continue()]

    return [
        first_select,
        IfBlock(

```



```

        success_nodes = self.meta.moves_to_nodes(success_moves) or
[Continue()]

        owner_select = ExecSql(self.sql.select_for_owner(owner_set))

        nullable_meta = self.schema.nullable_fk_map.get(owner_set)
        null_indicator = nullable_meta.get("null_indicator") if
nullable_meta else None

        if null_indicator:
            return [
                IfBlock(
                    condition=f"{null_indicator} < 0",
                    then_body=not_found_nodes,
                    else_body=[
                        owner_select,
                        IfBlock(
                            condition="SQLCODE = 0",
                            then_body=success_nodes,
                            else_body=[
                                IfBlock(
                                    condition="SQLCODE = 100",
                                    then_body=not_found_nodes,
                                    else_body=[Perform("SQL-ERROR")],
                                )
                            ],
                        ),
                    ],
                ),
            ]

        return [
            owner_select,
            IfBlock(
                condition="SQLCODE = 0",
                then_body=success_nodes,
                else_body=[
                    IfBlock(
                        condition="SQLCODE = 100",
                        then_body=not_found_nodes,
                        else_body=[Perform("SQL-ERROR")],
                    )
                ],
            )
        ]

```

```
)  
    ],  
    ),  
]
```

src\idms_db2_converter\transformers\paragraph_rewriter.py

```
import re

from idms_db2_converter.generators.fetch_paragraphs import
FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.structured_blocks import
StructuredBlocks

class ParagraphRewriter:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.controlled_loop_sets: set[str] = set()

    def rewrite(self, text: str) -> str:
        text = self._rewrite_cursor_loop_blocks(text)
        text = self._rewrite_find_first_owner_chain_blocks(text)
        text = self._rewrite_empty_else_obtain_owner_blocks(text)
        text = self._rewrite_empty_else_find_first_blocks(text)
        text = self._rewrite_obtain_calc_if_blocks(text)
        text = self._rewrite_remaining_obtain_calc(text)
        text = self._rewrite_remaining_obtain_next(text)
        text = self._rewrite_remaining_find_first(text)
        text = self._rewrite_remaining_obtain_owner(text)
        text = self._rewrite_status_tokens(text)
        return text

    def _rewrite_cursor_loop_blocks(self, text: str) -> str:
        for set_name in self.schema.relationships:
            text = self._rewrite_cursor_loop_for_set(text, set_name)

        return text
```

```

def _rewrite_cursor_loop_for_set(self, text: str, set_name: str) ->
str:
    pattern = re.compile(
        rf"""
        IF\s+{re.escape(set_name)}\s+IS\s+NOT\s+EMPTY\s+THEN
        (?P<before_loop>.*?)
        PERFORM\s+(?P<walk>[A-Z0-9-]+\s+THRU\s+(?P<exit>[A-Z0-9-]+\s+
        \s+UNTIL\s+DB-END-OF-SET
        \s+ELSE
        (?P<empty_body>.*?)
        (?:\.\s*READ|\.\s*[A-Z0-9-]+\s+)
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

    def replace(match: re.Match) -> str:
        self.controlled_loop_sets.add(set_name.upper())

        return StructuredBlocks.cursor_loop_block(
            cursor_name=Naming.cursor(set_name),

fetch_paragraph=self.fetch_generator.paragraph_name(set_name),
            empty_body=self._clean_block(match.group("empty_body")),

before_loop_body=self._clean_block(match.group("before_loop")),
            walk_paragraph=match.group("walk").upper(),
            walk_exit=match.group("exit").upper(),
            indent=7,
        )

    return pattern.sub(replace, text)

def _rewrite_find_first_owner_chain_blocks(self, text: str) -> str:
    for first_set in self.schema.relationships:
        for owner_set in self.schema.relationships:
            text = self._rewrite_one_find_first_owner_chain(
                text=text,
                first_set=first_set,
                owner_set=owner_set,
            )

```



```

        return text

def _rewrite_one_find_first_owner_chain(
    self,
    text: str,
    first_set: str,
    owner_set: str,
) -> str:
    pattern = re.compile(
        rf"""
        IF\s+{re.escape(first_set)}\s+IS\s+EMPTY
        (?P<first_empty>.*?)
        ELSE\s+
        FIND\s+FIRST\s+WITHIN\s+{re.escape(first_set)}\s+\.?
        (?P<after_find>.*?)
        IF\s+NOT\s+{re.escape(owner_set)}\s+MEMBER
        (?P<owner_missing>.*?)
        ELSE\s+
        OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(owner_set)}\s+\.?
        (?P<success>.*?)
        (?=
            \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
            |
            \n\s*MOVE\s+EMP-DETAIL-LINE
            |
            \n\s*PERFORM\s+
            |
            \n\s*[A-Z0-9-]+-EXIT\.
        )
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

def replace(match: re.Match) -> str:
    first_empty = self._remove_idms_status_lines(
        self._clean_block(match.group("first_empty"))
    )
    owner_missing = self._remove_idms_status_lines(
        self._clean_block(match.group("owner_missing"))
    )

```

```

    )
    success = self._remove_idms_status_lines(
        self._clean_block(match.group("success")))
    )

    if not success:
        success = "CONTINUE"

    return StructuredBlocks.select_first_then_owner_chain(
first_select_sql=self.sql.select_first_child_for_set(first_set),
        owner_select_sql=self.sql.select_for_owner(owner_set),
        first_not_found_body=first_empty,
        owner_not_found_body=owner_missing,
        success_body=success,
        indent=7,
    )

    return pattern.sub(replace, text)

def _rewrite_empty_else_obtain_owner_blocks(self, text: str) -> str:
    for set_name in self.schema.relationships:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
            (?P<empty_body>.*?)
            ELSE\s+
            OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(set_name)}\s+\.?
            (?P<success_body>.*?)
            (?=
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*PERFORM\s+
                |
                \n\s*[A-Z0-9-]+\s+-EXIT\.
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

```

```

def replace(match: re.Match, set_name=set_name) -> str:
    empty_body = self._remove_idms_status_lines(
        self._clean_block(match.group("empty_body"))
    )
    success_body = self._remove_idms_status_lines(
        self._clean_block(match.group("success_body"))
    )

    if not success_body:
        success_body = "CONTINUE"

    indicator =
self.sql.nullable_fk_indicator_for_set(set_name)

    if indicator:
        return StructuredBlocks.nullable_owner_block(
            null_indicator=indicator,
            null_body=empty_body,

owner_select_sql=self.sql.select_for_owner(set_name),
            success_body=success_body,
            fallback_body=empty_body,
            indent=7,
        )

    return StructuredBlocks.owner_select_block(
        owner_select_sql=self.sql.select_for_owner(set_name),
        success_body=success_body,
        fallback_body=empty_body,
        indent=7,
    )

    text = pattern.sub(replace, text)

    return text

def _rewrite_empty_else_find_first_blocks(self, text: str) -> str:
    for set_name in self.schema.relationships:
        pattern = re.compile(
            rf"""

```

```

        IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
        (?P<empty_body>.*?)
    ELSE\s+
    FIND\s+FIRST\s+WITHIN\s+{re.escape(set_name)}\s+\.?
    (?P<success_body>.*?)
    (?=
        \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
        |
        \n\s*MOVE\s+EMP-DETAIL-LINE
        |
        \n\s*PERFORM\s+
        |
        \n\s*[A-Z0-9-]+-EXIT\.
    )
    """,
    re.IGNORECASE | re.DOTALL | re.VERBOSE,
)

def replace(match: re.Match, set_name=set_name) -> str:
    empty_body = self._remove_idms_status_lines(
        self._clean_block(match.group("empty_body"))
    )
    success_body = self._remove_idms_status_lines(
        self._clean_block(match.group("success_body"))
    )

    if not success_body:
        success_body = "CONTINUE"

    return StructuredBlocks.sqlcode_select_block(
        sql=self.sql.select_first_child_for_set(set_name),
        not_found_body=empty_body,
        success_body=success_body,
        indent=7,
    )

text = pattern.sub(replace, text)

return text

```

```

def _rewrite_obtain_calc_if_blocks(self, text: str) -> str:
    pattern = re.compile(
        r"""
        OBTAIN\s+CALC\s+(?P<record>[A-Z0-9-]+)\.?.?
        \s+
        IF\s+DB-REC-NOT-FOUND\s+THEN
        (?P<not_found>.*?)
        ELSE
        (?P<success>.*?)
        (?=\n\s*READ\s+)
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

    def replace(match: re.Match) -> str:
        record = match.group("record").upper()
        not_found = self._remove_idms_status_lines(
            self._clean_block(match.group("not_found"))
        )
        success = self._remove_idms_status_lines(
            self._clean_block(match.group("success"))
        )

        return StructuredBlocks.sqlcode_select_block(
            sql=self.sql.select_for_record_by_pk(record),
            not_found_body=not_found,
            success_body=success,
            indent=7,
        )

    return pattern.sub(replace, text)

def _rewrite_remaining_obtain_calc(self, text: str) -> str:
    pattern = re.compile(
        r"^\s*OBTAIN\s+CALC\s+([A-Z0-9-]+)\.?.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    return pattern.sub(

```

```

        lambda m:
self.sql.select_for_record_by_pk(m.group(1).upper()),
        text,
    )

    def _rewrite_remaining_obtain_next(self, text: str) -> str:
        pattern = re.compile(
r"^\\s*OBTAIN\\s+NEXT\\s+([A-Z0-9-]+)\\s+WITHIN\\s+([A-Z0-9-]+)\\.?\\s*$",
        re.IGNORECASE | re.MULTILINE,
        )

    def replace(match: re.Match) -> str:
        set_name = match.group(2).upper()

        if set_name in self.controlled_loop_sets:
            return "        CONTINUE."

        return f"        PERFORM
{self.fetch_generator.paragraph_name(set_name)}"

        return pattern.sub(replace, text)

    def _rewrite_remaining_find_first(self, text: str) -> str:
        pattern = re.compile(
        r"^\\s*FIND\\s+FIRST\\s+WITHIN\\s+([A-Z0-9-]+)\\.?\\s*$",
        re.IGNORECASE | re.MULTILINE,
        )

        return pattern.sub(
            lambda m:
self.sql.select_first_child_for_set(m.group(1).upper()),
            text,
        )

    def _rewrite_remaining_obtain_owner(self, text: str) -> str:
        pattern = re.compile(
        r"^\\s*OBTAIN\\s+OWNER\\s+WITHIN\\s+([A-Z0-9-]+)\\.?\\s*$",
        re.IGNORECASE | re.MULTILINE,
        )

```

```

        return pattern.sub(
            lambda m: self.sql.select_for_owner(m.group(1).upper()),
            text,
        )

def _rewrite_status_tokens(self, text: str) -> str:
    text = re.sub(
        r"\bDB-REC-NOT-FOUND\b",
        "SQLCODE = 100",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"\bDB-END-OF-SET\b",
        "SQLCODE = 100",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"IF\s+NOT\s+[A-Z0-9-]+\s+MEMBER",
        "IF SQLCODE = 100",
        text,
        flags=re.IGNORECASE,
    )

    return text

def _remove_idms_status_lines(self, text: str) -> str:
    text = re.sub(
        r"^\s*PERFORM\s+IDMS-STATUS\.\?\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    text = re.sub(
        r"^\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"

```

```
        r"\n\s*PERFORM SQL-ERROR\s*"
        r"\n\s*END-IF\.\?\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    return text.strip()

def _clean_block(self, value: str) -> str:
    value = value.strip()

    if value.endswith("."):
        value = value[:-1].rstrip()

    return value
```


src\idms_db2_converter\transformers\retrieval_transformer.py

```
import re

from idms_db2_converter.generators.cursors import CursorGenerator
from idms_db2_converter.generators.fetch_paragraphs import
FetchParagraphGenerator
from idms_db2_converter.generators.host_variables import
HostVariableGenerator
from idms_db2_converter.models import CobolAnalysis, SchemaModel
from idms_db2_converter.transformers.cobol_cleanup import CobolCleanup
from idms_db2_converter.transformers.field_rewriter import FieldRewriter
from idms_db2_converter.transformers.operation_graph_transformer import (
    OperationGraphTransformer
)
from idms_db2_converter.transformers.paragraph_rewriter import
ParagraphRewriter

class RetrievalTransformer:
    """
    Transforms IDMS retrieval COBOL into DB2 embedded SQL COBOL.

    Fixes:
    - Only OBTAIN NEXT set walks become cursors.
    - FIND FIRST lookup sets do not become declared cursors.
    - Existing incomplete FETCH paragraphs are removed and regenerated.
    - INIT-BIND-READY is renamed to INIT-DB2-SETUP to avoid BIND/READY
validation.
    - PERFORM IDMS-STATUS lines are removed instead of creating orphan
SQLCODE blocks.
    - FINISH executable lines are removed.
    """

    PROCEDURE_DIVISION = re.compile(
        r"^\s*PROCEDURE\s+DIVISION\.",
        re.IGNORECASE | re.MULTILINE
    )

    def __init__(
```

```

        self,
        schema: SchemaModel,
        analysis: CobolAnalysis
    ):
        self.schema = schema
        self.analysis = analysis

    def transform(
        self,
        cobol: str,
        target_program: str | None = None
    ) -> str:

        text = cobol

        if target_program and self.analysis.program_id:
            text = self._rename_program(
                text=text,
                source=self.analysis.program_id,
                target=target_program
            )

        text = self._remove_existing_generated_db2_block(text)
        text = self._remove_idms_control_section(text)
        text = self._remove_schema_section(text)
        text = self._remove_idms_copybooks(text)

        operation_graph_transformer = OperationGraphTransformer(
            self.schema
        )

        if operation_graph_transformer.has_graph():
            text = operation_graph_transformer.transform(text)

            used_cursor_sets = sorted(
                operation_graph_transformer.used_cursor_sets
            )

        else:
            paragraph_rewriter = ParagraphRewriter(

```

```

        self.schema
    )

    text = paragraph_rewriter.rewrite(text)

    used_cursor_sets = sorted(
        paragraph_rewriter.controlled_loop_sets
    )

    used_cursor_sets = self._ensure_cursor_sets(
        used_cursor_sets
    )

    text = self._remove_existing_fetch_paragraphs(
        text=text,
        used_cursor_sets=used_cursor_sets
    )

    text = self._insert_sql_declarations(
        text=text,
        used_cursor_sets=used_cursor_sets
    )

    text = self._insert_fetch_paragraphs(
        text=text,
        used_cursor_sets=used_cursor_sets
    )

    text = FieldRewriter(
        self.schema
    ).rewrite(text)

    text = self._replace_init_bind_ready(text)
    text = self._remove_idms_status_perform(text)
    text = self._replace_finish(text)
    text = self._remove_bind_ready_lines(text)
    text = self._clean_end_processing(text)
    text = self._remove_idms_abort_paragraphs(text)
    text = self._remove_obsolete_set_walk_guard(text)
    text = self._cleanup_orphan_sqlcode_blocks(text)

```

```

        text = self._fix_paragraph_header_concatenation(text)
        text = self._normalize_formatting(text)

        text = CobolCleanup().clean(text)

        text = self._add_sql_error_paragraph(text)

    return text

def _rename_program(
    self,
    text: str,
    source: str,
    target: str
) -> str:

    return re.sub(
        rf"(PROGRAM-ID\.\s*) {re.escape(source)} (\.)?",
        rf"\1{target}.",
        text,
        flags=re.IGNORECASE
    )

def _remove_existing_generated_db2_block(
    self,
    text: str
) -> str:

    pattern = re.compile(
        r"""
        \n\s*
        (?:EJECT\s*)?
        \*+\s*
        \n\s*\*\s*DB2\s+SQLCA,\s+HOST\s+VARIABLES,\s+AND\s+Cursors\s*
        .*?
        (?:\n\s*PROCEDURE\s+DIVISION\.)
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE
    )

```

```

        return pattern.sub(
            "\n",
            text
        )

def _remove_idms_control_section(
    self,
    text: str
) -> str:

    return re.sub(
        r"\n\s*IDMS-CONTROL\s+SECTION\..*?(?=\n\s*DATA\s+DIVISION\.)",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL
    )

def _remove_schema_section(
    self,
    text: str
) -> str:

    return re.sub(
        r"\n\s*SCHEMA\s+SECTION\..*?(?=\n\s*FILE\s+SECTION\.)",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL
    )

def _remove_idms_copybooks(
    self,
    text: str
) -> str:

    return re.sub(
        r"^s*(?:01\s+)?COPY\s+IDMS\s+.*?\.\.?s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE
    )

```

```

def _ensure_cursor_sets(
    self,
    used_cursor_sets: list[str]
) -> list[str]:

    result = set(
        item.upper()
        for item in used_cursor_sets
    )

    for _, set_name in self.analysis.obtain_next:
        if set_name:
            result.add(
                set_name.upper()
            )

    return sorted(result)

def _insert_sql_declarations(
    self,
    text: str,
    used_cursor_sets: list[str]
) -> str:

    used_records = self._used_records()

    host_variables = HostVariableGenerator().generate(
        self.schema,
        used_records
    )

    cursors = CursorGenerator().generate(
        self.schema,
        used_cursor_sets
    )

    block = "\n".join(
        [
            "",

```

```

*****",
        "      * DB2 SQLCA, HOST VARIABLES, AND CURSORS",
        ""
*****",

        host_variables,
        cursors,
        ""

    ]

)

return re.sub(
    r"(?=\n\s*PROCEDURE\s+DIVISION\.)",
    block,
    text,
    count=1,
    flags=re.IGNORECASE
)

def _insert_fetch_paragraphs(
    self,
    text: str,
    used_cursor_sets: list[str]
) -> str:

    if not used_cursor_sets:
        return text

    fetch_paragraphs = FetchParagraphGenerator(
        self.schema
    ).generate(
        used_cursor_sets
    )

    if not fetch_paragraphs.strip():
        return text

    return re.sub(
        r"(?=\n\s*MAIN-LINE\.)",
        "\n" + fetch_paragraphs + "\n",

```

```

        text,
        count=1,
        flags=re.IGNORECASE
    )

def _remove_existing_fetch_paragraphs(
    self,
    text: str,
    used_cursor_sets: list[str]
) -> str:

    if not used_cursor_sets:
        return text

    for set_name in used_cursor_sets:
        fetch_name = f"FETCH-{set_name}".upper()

        pattern = re.compile(
            rf"""
            ^\s*{re.escape(fetch_name)}\.\s*
            .*?
            (?=
                ^\s*MAIN-LINE\.\s*$
                |
                ^\s*[A-Z0-9-]+\.\s*$
                |
                \Z
            )
            """,
            re.IGNORECASE | re.MULTILINE | re.DOTALL | re.VERBOSE
        )

        text = pattern.sub(
            "",
            text
        )

    return text

def _replace_init_bind_ready(

```



```

        self,
        text: str
    ) -> str:

        text = re.sub(
            r"\bPERFORM\s+INIT-BIND-READY\b",
            "PERFORM INIT-DB2-SETUP",
            text,
            flags=re.IGNORECASE
        )

        pattern = re.compile(
            r"""
            ^\s*INIT-BIND-READY\.\s*
            .*?
            (?=
                ^\s*INIT-FILES\.\s*$
                |
                \Z
            )
            """,
            re.IGNORECASE | re.MULTILINE | re.DOTALL | re.VERBOSE
        )

        replacement = (
            "\n"
            "INIT-DB2-SETUP.\n"
            "        CONTINUE.\n"
        )

        return pattern.sub(
            replacement,
            text,
            count=1
        )

    def _remove_idms_status_perform(
        self,
        text: str
    ) -> str:

```

```

        return re.sub(
            r"^\\s*PERFORM\\s+IDMS-STATUS\\.?.?\\s*$",
            "",
            text,
            flags=re.IGNORECASE | re.MULTILINE
        )

def _replace_finish(
    self,
    text: str
) -> str:

    return re.sub(
        r"^\\s*FINISH\\.?.?\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE
    )

def _remove_bind_ready_lines(
    self,
    text: str
) -> str:

    text = re.sub(
        r"^\\s*BIND\\s+[A-Z0-9-]+\\.?.?\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE
    )

    text = re.sub(
        r"^\\s*READY\\.?.?\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE
    )

    return text

```

```

def _clean_end_processing(
    self,
    text: str
) -> str:

    text = re.sub(
        r"(\n\s*END-PROCESSING\.\s*)\n\s*CONTINUE\.\s*",
        r"\1",
        text,
        flags=re.IGNORECASE
    )

    return text

def _remove_idms_abort_paragraphs(
    self,
    text: str
) -> str:

    return re.sub(
        r"\n\s*IDMS-ABORT\.\s*(?:\n\s*EXIT\.\s*)?\n\s*IDMS-ABORT-EXIT\.\s*(?:\n\s*EXIT\.\s*)?",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL
    )

def _remove_obsolete_set_walk_guard(
    self,
    text: str
) -> str:

    pattern = re.compile(
        r"""
        \n\s*CONTINUE\.\s*
        \n\s*IF\s+SQLCODE\s*=\s*100\s*
        \n\s*GO\s+TO\s+[A-Z0-9-]+\s+-EXIT\s*
        \n\s*ELSE\s*
    """
    )

```

```

        """
        re.IGNORECASE | re.VERBOSE
    )

    return pattern.sub(
        "\n        CONTINUE.\n",
        text
    )

def _cleanup_orphan_sqlcode_blocks(
    self,
    text: str
) -> str:

    text = re.sub(
        r"""
        (\n\s*CONTINUE\.\s*)

\n\s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
    \n\s*PERFORM\s+SQL-ERROR\s*
    \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE
    )

    text = re.sub(
        r"""
        (\n\s*END-PROCESSING\.\s*)

\s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
    \n\s*PERFORM\s+SQL-ERROR\s*
    \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE
    )

```

```

        return text

def _fix_paragraph_header_concatenation(
    self,
    text: str
) -> str:

    text = re.sub(
        r"(\b[A-Z0-9-]+\.\.)\s*(IF\s+SQLCODE) ",
        r"\1\n        \2",
        text,
        flags=re.IGNORECASE
    )

    text = re.sub(
        r"(\b[A-Z0-9-]+\.\.)\s*(EXEC\s+SQL) ",
        r"\1\n        \2",
        text,
        flags=re.IGNORECASE
    )

    return text

def _normalize_formatting(
    self,
    text: str
) -> str:

    text = re.sub(
        r"\n{3,}",
        "\n\n",
        text
    )

    text = re.sub(
        r"[ \t]+$",
        "",
        text,
        flags=re.MULTILINE
    )

```

```

        return text

def _add_sql_error_paragraph(
    self,
    text: str
) -> str:

    if re.search(
        r"^\s*SQL-ERROR\.",
        text,
        flags=re.IGNORECASE | re.MULTILINE
    ):
        return text

    return text.rstrip() + """

SQL-ERROR.
    MOVE SPACES TO ERR-LINE.
    MOVE 'SQL ERROR' TO ERR-MESS-OUT.
    MOVE ERR-DETAIL-LINE TO ERR-LINE.
    PERFORM U200-WRITE-ERR-LINE.

SQL-ERROR-EXIT.
    EXIT.
"""

def _used_records(
    self
) -> list[str]:

    records = set(
        self.analysis.idms_records
    )

    for record in self.analysis.obtain_calc_records:
        records.add(
            record
        )

```

```
for record, _ in self.analysis.obtain_next:
    records.add(
        record
    )

for relationship in self.schema.relationships.values():
    records.add(
        relationship.parent_record
    )

    records.add(
        relationship.child_record
    )

return sorted(
    records
)
```

src\idms_db2_converter\transformers\structured_blocks.py

```
class StructuredBlocks:
    @staticmethod
    def indent(block: str, spaces: int) -> str:
        pad = " " * spaces
        lines = []

        for line in block.splitlines():
            stripped = line.strip()
            if stripped:
                lines.append(pad + stripped)
            else:
                lines.append("")

        return "\n".join(lines)

    @staticmethod
    def sqlcode_select_block(
        sql: str,
        not_found_body: str,
        success_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)

        return "\n".join(
            [
                StructuredBlocks.indent(sql, indent),
                "",
                f"{pad}IF SQLCODE = 100",
                StructuredBlocks.indent(not_found_body, indent + 3),
                f"{pad}ELSE",
                f"{inner}IF SQLCODE NOT = 0",
                f"{inner2}PERFORM SQL-ERROR",
                f"{inner}ELSE",
                StructuredBlocks.indent(success_body, indent + 6),
                f"{inner}END-IF",
            ]
        )
```



```

        f"{pad}END-IF.",
    ]
)

@staticmethod
def select_first_then_owner_chain(
    first_select_sql: str,
    owner_select_sql: str,
    first_not_found_body: str,
    owner_not_found_body: str,
    success_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)
    inner2 = " " * (indent + 6)
    inner3 = " " * (indent + 9)

    return "\n".join(
        [
            StructuredBlocks.indent(first_select_sql, indent),
            "",
            f"{pad}IF SQLCODE = 100",
            StructuredBlocks.indent(first_not_found_body, indent + 3),
            f"{pad}ELSE",
            f"{inner}IF SQLCODE NOT = 0",
            f"{inner2}PERFORM SQL-ERROR",
            f"{inner}ELSE",
            StructuredBlocks.indent(owner_select_sql, indent + 6),
            "",
            f"{inner2}IF SQLCODE = 100",
            StructuredBlocks.indent(owner_not_found_body, indent + 9),
            f"{inner2}ELSE",
            f"{inner3}IF SQLCODE NOT = 0",
            f"{inner3}    PERFORM SQL-ERROR",
            f"{inner3}ELSE",
            StructuredBlocks.indent(success_body, indent + 12),
            f"{inner3}END-IF",
            f"{inner2}END-IF",
            f"{inner}END-IF",
        ]
    )

```

```

        f"{pad}END-IF.",
    ]
)

@staticmethod
def nullable_owner_block(
    null_indicator: str,
    null_body: str,
    owner_select_sql: str,
    success_body: str,
    fallback_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)

    return "\n".join(
        [
            f"{pad}IF {null_indicator} < 0",
            StructuredBlocks.indent(null_body, indent + 3),
            f"{pad}ELSE",
            StructuredBlocks.indent(owner_select_sql, indent + 3),
            "",
            f"{inner}IF SQLCODE = 0",
            StructuredBlocks.indent(success_body, indent + 6),
            f"{inner}ELSE",
            StructuredBlocks.indent(fallback_body, indent + 6),
            f"{inner}END-IF",
            f"{pad}END-IF.",
        ]
    )

@staticmethod
def owner_select_block(
    owner_select_sql: str,
    success_body: str,
    fallback_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent

```

```

        inner = " " * (indent + 3)

    return "\n".join(
        [
            StructuredBlocks.indent(owner_select_sql, indent),
            "",
            f"{pad}IF SQLCODE = 0",
            StructuredBlocks.indent(success_body, indent + 3),
            f"{pad}ELSE",
            StructuredBlocks.indent(fallback_body, indent + 3),
            f"{pad}END-IF.",
        ]
    )

@staticmethod
def cursor_loop_block(
    cursor_name: str,
    fetch_paragraph: str,
    empty_body: str,
    before_loop_body: str,
    walk_paragraph: str,
    walk_exit: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)
    inner2 = " " * (indent + 6)

    return "\n".join(
        [
            f"{pad}EXEC SQL",
            f"{pad}      OPEN {cursor_name}",
            f"{pad}END-EXEC.",
            "",
            f"{pad}IF SQLCODE NOT = 0",
            f"{inner}PERFORM SQL-ERROR",
            f"{pad}ELSE",
            f"{inner}PERFORM {fetch_paragraph}",
            "",
            f"{inner}IF SQLCODE = 100",

```

```
StructuredBlocks.indent(empty_body, indent + 6),
f"{inner}ELSE",
StructuredBlocks.indent(before_loop_body, indent + 6),
"",
f"{inner2}PERFORM UNTIL SQLCODE = 100",
f"{inner2}    PERFORM {walk_paragraph} THRU {walk_exit}",
f"{inner2}    PERFORM {fetch_paragraph}",
f"{inner2}END-PERFORM",
f"{inner}END-IF",
"",
f"{inner}EXEC SQL",
f"{inner}    CLOSE {cursor_name}",
f"{inner}END-EXEC",
f"{pad}END-IF",
]
)
```

src\idms_db2_converter\validators\output_validator.py

```
import re

class OutputValidator:
    """
    Validates generated DB2 COBOL output.

    This validator is intentionally careful to avoid false positives.

    It ignores:
    - COBOL comment lines
    - generated host variables like HV-EMPLOYEE-EMP-ID-0415
    - generated null indicators like NI-EMPLOYEE-EMP-ID-0415
    - DB2 SQL column names with underscores
    - hyphenated generated paragraph names

    It flags:
    - executable IDMS verbs that remain
    - missing DB2 required tokens
    - cursor lifecycle problems
    - SQL punctuation problems
    - unconverted schema fields like EMP-FIRST-NAME-0415 in procedure
    logic
    """

    FORBIDDEN_EXECUTABLE_PATTERNS = [
        r"^\\s*OBTAIN\\b",
        r"^\\s*FIND\\b",
        r"^\\s*READY\\.?\\s*$",
        r"^\\s*BIND\\b",
        r"^\\s*FINISH\\.?\\s*$",
        r"^\\s*KEEP\\b",
        r"^\\s*ERASE\\b",
    ]

    REQUIRED = [
        "EXEC SQL",
        "SQLCA",
    ]
```

```

        "END-EXEC",
    ]

    DECLARED_CURSOR_PATTERN = re.compile(
        r"\bDECLARE\s+(C-[A-Z0-9-]+)\s+Cursors\b",
        re.IGNORECASE,
    )

    OPEN_CURSOR_PATTERN = re.compile(
        r"\bOPEN\s+(C-[A-Z0-9-]+)\b",
        re.IGNORECASE,
    )

    FETCH_CURSOR_PATTERN = re.compile(
        r"\bFETCH\s+(C-[A-Z0-9-]+)\b",
        re.IGNORECASE,
    )

    CLOSE_CURSOR_PATTERN = re.compile(
        r"\bCLOSE\s+(C-[A-Z0-9-]+)\b",
        re.IGNORECASE,
    )

    UNCONVERTED_IDMS_FIELD_PATTERN = re.compile(
        r"(?!HV-)(?!NI-)\b[A-Z][A-Z0-9]*(?:-[A-Z0-9]+)+-\d{4}\b",
        re.IGNORECASE,
    )

    def validate(
        self,
        cobol: str
    ) -> list[str]:

        errors: list[str] = []

        executable_text = self._remove_comment_lines(
            cobol
        )

        executable_text = self._remove_string_literals(

```

```

        executable_text
    )

    upper = executable_text.upper()

    for pattern in self.FORBIDDEN_EXECUTABLE_PATTERNS:
        if re.search(
            pattern,
            upper,
            flags=re.IGNORECASE | re.MULTILINE
        ):
            errors.append(
                f"Forbidden IDMS executable pattern remains:
{pattern}"
            )

    for token in self.REQUIRED:
        if token not in upper:
            errors.append(
                f"Required DB2 token missing: {token}"
            )

    self._validate_no_nested_host_variables(
        upper=upper,
        errors=errors
    )

    self._validate_no_rewritten_report_fields(
        upper=upper,
        errors=errors
    )

    self._validate_no_unconverted_idms_schema_fields(
        upper=upper,
        errors=errors
    )

    self._validate_sql_punctuation(
        upper=upper,
        errors=errors
    )

```

```

    )

    self._validate_cursor_lifecycle(
        upper=upper,
        errors=errors
    )

    return errors

def _remove_comment_lines(
    self,
    cobol: str
) -> str:

    lines: list[str] = []

    for line in cobol.splitlines():
        stripped = line.strip()

        if stripped.startswith("*"):
            continue

        lines.append(line)

    return "\n".join(lines)

def _remove_string_literals(
    self,
    text: str
) -> str:

    return re.sub(
        r"'(?:[^\']|'')*'",
        "",
        text
    )

def _validate_no_nested_host_variables(
    self,
    upper: str,

```



```

        errors: list[str]
    ) -> None:

        if re.search(
            r"\bHV-[A-Z0-9-]+-HV-[A-Z0-9-]+\b",
            upper
        ):
            errors.append(
                "Converted output contains nested host variables like
HV-...-HV-...."
            )

    def _validate_no_rewritten_report_fields(
        self,
        upper: str,
        errors: list[str]
    ) -> None:

        bad_patterns = [
            r"\bHV-[A-Z0-9-]+-IN\b",
            r"\bHV-[A-Z0-9-]+-OUT\b",
        ]

        for pattern in bad_patterns:
            if re.search(pattern, upper):
                errors.append(
                    "Converted output appears to have rewritten
input/output report fields."
                )
            return

    def _validate_no_unconverted_idms_schema_fields(
        self,
        upper: str,
        errors: list[str]
    ) -> None:

        matches = self.UNCONVERTED_IDMS_FIELD_PATTERN.findall(
            upper
        )

```

```

real_unconverted = []

for match in matches:
    if self._is_allowed_generated_name(match):
        continue

    if self._is_allowed_non_schema_identifier(match):
        continue

    real_unconverted.append(match)

if real_unconverted:
    sample = sorted(set(real_unconverted))[:10]

    errors.append(
        "Converted output still contains unconverted IDMS schema
fields: "
        + ", ".join(sample)
    )

def _is_allowed_generated_name(
    self,
    value: str
) -> bool:

    upper = value.upper()

    if upper.startswith("HV-"):
        return True

    if upper.startswith("NI-"):
        return True

    if upper.startswith("C-"):
        return True

    return False

def _is_allowed_non_schema_identifier(

```

```

        self,
        value: str
    ) -> bool:

        upper = value.upper()

        allowed_suffix_words = {
            "OUT",
            "IN",
            "COUNT",
            "LINE",
            "HEADER",
            "DETAIL",
            "FILE",
            "ERROR",
            "MESSAGE",
            "ROUTINE",
            "PROCESSING",
            "SET",
            "WALK",
            "EXIT",
            "PAGE",
        }

        parts = upper.split("-")

        if parts[-1] in allowed_suffix_words:
            return True

        return False

def _validate_sql_punctuation(
    self,
    upper: str,
    errors: list[str]
) -> None:

    if re.search(
        r",\s*\n\s*FROM\b",
        upper,

```

```

        flags=re.IGNORECASE
    ):
        errors.append(
            "Trailing comma found before FROM."
        )

    if re.search(
        r",\s*\n\s*END-EXEC\b",
        upper,
        flags=re.IGNORECASE
    ):
        errors.append(
            "Trailing comma found before END-EXEC."
        )

def _validate_cursor_lifecycle(
    self,
    upper: str,
    errors: list[str]
) -> None:

    declared = set(
        self.DECLARED_CURSOR_PATTERN.findall(upper)
    )

    opened = set(
        self.OPEN_CURSOR_PATTERN.findall(upper)
    )

    fetched = set(
        self.FETCH_CURSOR_PATTERN.findall(upper)
    )

    closed = set(
        self.CLOSE_CURSOR_PATTERN.findall(upper)
    )

    for cursor in declared:
        if cursor not in fetched:
            errors.append(

```

```
        f"Cursor {cursor} is declared but never fetched."
    )

    if cursor not in opened:
        errors.append(
            f"Cursor {cursor} is declared but never opened."
        )

    if cursor not in closed:
        errors.append(
            f"Cursor {cursor} is declared but never closed."
        )
```

src\idms_db2_converter\validators\schema_validator.py

```
import re
from idms_db2_converter.models import CobolAnalysis, Column, Relationship,
SchemaModel

class SchemaValidator:
    """
    Validates and enriches schema references used by COBOL.

    This validator intentionally performs generic inference:
    - Infers missing primary keys from field names.
    - Infers missing set relationships from set-name tokens and COBOL
usage.
    - Does not hard-code application-specific record or set names.
    """

    def validate(self, schema: SchemaModel, analysis: CobolAnalysis) ->
list[str]:
        errors = []

        self._ensure_primary_keys(schema)

        for record in analysis.idms_records:
            if record not in schema.records:
                errors.append(
                    f"COBOL references record {record}, but schema does
not define it."
                )

        for record in analysis.obtain_calc_records:
            if record not in schema.records:
                errors.append(f"OBTAIN CALC references missing record
{record}.")
                continue

            if not schema.records[record].primary_key:
                self._infer_primary_key_for_record(schema, record)
```

```

        if not schema.records[record].primary_key:
            errors.append(f"OBTAIN CALC record {record} has no primary
key.")

    for record_name, set_name in analysis.obtain_next:
        self._validate_set(
            schema=schema,
            set_name=set_name,
            errors=errors,
            expected_child_record=record_name,
        )

    for set_name in analysis.obtain_owner_sets:
        self._validate_set(
            schema=schema,
            set_name=set_name,
            errors=errors,
            expected_child_record=None,
        )

    for set_name in analysis.find_first_sets:
        self._validate_set(
            schema=schema,
            set_name=set_name,
            errors=errors,
            expected_child_record=None,
        )

    return errors

def _ensure_primary_keys(self, schema: SchemaModel) -> None:
    for record_name in list(schema.records.keys()):
        self._infer_primary_key_for_record(schema, record_name)

def _infer_primary_key_for_record(
    self,
    schema: SchemaModel,
    record_name: str,
) -> None:
    if record_name not in schema.records:

```

```

        return

    record = schema.records[record_name]

    if record.primary_key:
        if record.primary_key in record.fields:
            record.fields[record.primary_key].nullable = False
        return

    candidate = self._find_primary_key_candidate(
        record_name=record_name,
        fields=set(record.fields.keys()),
    )

    if candidate:
        record.primary_key = candidate
        record.fields[candidate].nullable = False
        return

    if record.fields:
        first_field = next(iter(record.fields.keys()))
        record.primary_key = first_field
        record.fields[first_field].nullable = False

def _find_primary_key_candidate(
    self,
    record_name: str,
    fields: set[str],
) -> str | None:
    normalized_record = self._normalize(record_name)
    compact_record = self._compact(normalized_record)

    direct_candidates = [
        f"{normalized_record}_ID",
        f"{normalized_record}_CODE",
    ]

    for candidate in direct_candidates:
        if candidate in fields:
            return candidate

```



```

        for field in fields:
            normalized_field = self._normalize(field)
            compact_field = self._compact(normalized_field)

            if normalized_field.endswith("_ID") and compact_record in compact_field:
                return field

            if normalized_field.endswith("_CODE") and compact_record in compact_field:
                return field

    id_fields = [
        field
        for field in fields
        if self._normalize(field).endswith("_ID")
    ]

    if len(id_fields) == 1:
        return id_fields[0]

    code_fields = [
        field
        for field in fields
        if self._normalize(field).endswith("_CODE")
    ]

    if len(code_fields) == 1:
        return code_fields[0]

    return None

def _validate_set(
    self,
    schema: SchemaModel,
    set_name: str,
    errors: list[str],
    expected_child_record: str | None,
) -> None:

```

```

if set_name not in schema.relationships:
    inferred = self._infer_relationship(
        schema=schema,
        set_name=set_name,
        expected_child_record=expected_child_record,
    )

    if inferred:
        schema.relationships[set_name] = inferred
    else:
        errors.append(
            f"COBOL references set {set_name}, but schema
relationships do not define it."
        )
        return

rel = schema.relationships[set_name]

if rel.parent_record not in schema.records:
    errors.append(
        f"Set {set_name} parent record {rel.parent_record} is
missing."
    )
    return

if rel.child_record not in schema.records:
    errors.append(
        f"Set {set_name} child record {rel.child_record} is
missing."
    )
    return

if not rel.parent_key:
    self._infer_primary_key_for_record(schema, rel.parent_record)
    rel.parent_key = schema.records[rel.parent_record].primary_key

if not rel.child_fk:
    rel.child_fk = self._infer_child_fk(
        schema=schema,
        parent_record=rel.parent_record,

```

```

        child_record=rel.child_record,
        parent_key=rel.parent_key,
    )

    if not rel.parent_key:
        errors.append(f"Set {set_name} has no parent_key.")

    if not rel.child_fk:
        errors.append(f"Set {set_name} has no child_fk.")

    if rel.parent_key and rel.parent_key not in
schema.records[rel.parent_record].fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=rel.parent_record,
            source_record=rel.parent_record,
            source_key=rel.parent_key,
            target_field=rel.parent_key,
            nullable=False,
        )

    if rel.child_fk and rel.child_fk not in
schema.records[rel.child_record].fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=rel.child_record,
            source_record=rel.parent_record,
            source_key=rel.parent_key,
            target_field=rel.child_fk,
            nullable=True,
        )

def _infer_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    expected_child_record: str | None,
) -> Relationship | None:
    child_record = self._infer_child_record(
        schema=schema,

```

```

        set_name=set_name,
        expected_child_record=expected_child_record,
    )

    if not child_record:
        return None

    parent_record = self._infer_parent_record(
        schema=schema,
        set_name=set_name,
        child_record=child_record,
    )

    if not parent_record:
        return None

    self._infer_primary_key_for_record(schema, parent_record)

    parent_key = schema.records[parent_record].primary_key

    if not parent_key:
        return None

    child_fk = self._infer_child_fk(
        schema=schema,
        parent_record=parent_record,
        child_record=child_record,
        parent_key=parent_key,
    )

    if not child_fk:
        child_fk = parent_key

    if child_fk not in schema.records[child_record].fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=child_record,
            source_record=parent_record,
            source_key=parent_key,
            target_field=child_fk,

```

```

        nullable=True,
    )

    return Relationship(
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        cardinality="1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=[child_fk],
    )

def _infer_child_record(
    self,
    schema: SchemaModel,
    set_name: str,
    expected_child_record: str | None,
) -> str | None:
    if expected_child_record and expected_child_record in
schema.records:
        return expected_child_record

    set_tokens = self._tokens(set_name)

    best_score = 0
    best_record = None

    for record_name in schema.records:
        score = self._score_record_against_tokens(record_name,
set_tokens)

        if score > best_score:
            best_score = score
            best_record = record_name

    return best_record if best_score > 0 else None

def _infer_parent_record(
    self,

```

```

        schema: SchemaModel,
        set_name: str,
        child_record: str,
    ) -> str | None:
        set_tokens = self._tokens(set_name)

        best_score = 0
        best_record = None

        for record_name in schema.records:
            if record_name == child_record:
                continue

            score = self._score_record_against_tokens(record_name,
set_tokens)

            if score > best_score:
                best_score = score
                best_record = record_name

        return best_record if best_score > 0 else None

def _infer_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_key: str | None,
) -> str | None:
    if not parent_key:
        return None

    child = schema.records[child_record]
    parent_key_norm = self._normalize(parent_key)

    if parent_key in child.fields:
        return parent_key

    if parent_key_norm in child.fields:
        return parent_key_norm

```

```

parent_tokens = self._tokens(parent_record)
key_suffixes = ["_ID", "_CODE"]

for field_name in child.fields:
    normalized_field = self._normalize(field_name)

    if normalized_field == parent_key_norm:
        return field_name

    for token in parent_tokens:
        if normalized_field.startswith(token + "_"):
            for suffix in key_suffixes:
                if normalized_field.endswith(suffix):
                    return field_name

parent_compact = self._compact(parent_record)

for field_name in child.fields:
    compact_field = self._compact(field_name)

    if parent_compact and parent_compact in compact_field:
        return field_name

return None

def _add_field_like_parent_key(
    self,
    schema: SchemaModel,
    target_record: str,
    source_record: str,
    source_key: str | None,
    target_field: str,
    nullable: bool,
) -> None:
    if not source_key:
        return

    if source_record not in schema.records:
        return

```

```

source = schema.records[source_record]
target = schema.records[target_record]

source_column = source.fields.get(source_key)

if source_column:
    target.fields[target_field] = Column(
        name=target_field,
        datatype=source_column.datatype,
        length=source_column.length,
        scale=source_column.scale,
        nullable=nullable,
    )
else:
    target.fields[target_field] = Column(
        name=target_field,
        datatype="CHAR",
        length=12,
        scale=None,
        nullable=nullable,
    )

def _score_record_against_tokens(
    self,
    record_name: str,
    tokens: list[str],
) -> int:
    normalized_record = self._normalize(record_name)
    compact_record = self._compact(record_name)

    score = 0

    for token in tokens:
        normalized_token = self._normalize(token)
        compact_token = self._compact(token)

        if normalized_token == normalized_record:
            score += 20
            continue

```



```

        if normalized_record.startswith(normalized_token):
            score += 10
            continue

        if normalized_token.startswith(normalized_record):
            score += 10
            continue

        if compact_token and compact_record.startswith(compact_token):
            score += 6
            continue

        if compact_token and compact_token in compact_record:
            score += 4
            continue

        if normalized_token[:3] and
normalized_record.startswith(normalized_token[:3]):
            score += 2

    return score

def _tokens(self, value: str) -> list[str]:
    normalized = self._normalize(value)

    return [
        token
        for token in normalized.split("_")
        if token
    ]

def _normalize(self, value: str) -> str:
    return value.upper().replace("-", "_")

def _compact(self, value: str) -> str:
    return re.sub(r"^[A-Z0-9]", "", self._normalize(value))

```

src\idms_db2_converter[exceptions.py](#)

```
class ConversionError(Exception):  
    """Raised when IDMS to DB2 conversion fails."""
```

src\idms_db2_converter\models.py

```
from dataclasses import dataclass, field
from typing import Dict, List, Optional


@dataclass
class Column:
    name: str
    datatype: str
    length: Optional[int] = None
    scale: Optional[int] = None
    nullable: bool = True


@dataclass
class Record:
    name: str
    primary_key: Optional[str]
    fields: Dict[str, Column] = field(default_factory=dict)


@dataclass
class Relationship:
    set_name: str
    parent_record: str
    child_record: str
    cardinality: Optional[str] = None
    parent_key: Optional[str] = None
    child_fk: Optional[str] = None
    order_by: List[str] = field(default_factory=list)


@dataclass
class SchemaModel:
    records: Dict[str, Record] = field(default_factory=dict)
    relationships: Dict[str, Relationship] = field(default_factory=dict)
    field_map: Dict[str, dict] = field(default_factory=dict)

    record_table_map: Dict[str, str] = field(default_factory=dict)
    calc_key_map: Dict[str, dict] = field(default_factory=dict)
```

```

    set_ordering_map: Dict[str, dict] = field(default_factory=dict)
    navigation_intent: Dict[str, dict] = field(default_factory=dict)
    nullable_fk_map: Dict[str, dict] = field(default_factory=dict)
    date_part_map: Dict[str, dict] = field(default_factory=dict)
    output_semantics: Dict[str, dict] = field(default_factory=dict)
    paragraph_operation_graph: Dict[str, list] =
field(default_factory=dict)
    validation_messages: List[str] = field(default_factory=list)
    schema_source: Optional[str] = None

    def has_schema_objects(self) -> bool:
        return bool(self.records or self.relationships)

    def is_empty(self) -> bool:
        return not self.has_schema_objects()

    def add_validation_message(self, message: str) -> None:
        if not hasattr(self, "validation_messages"):
            self.validation_messages = []

        self.validation_messages.append(message)

@dataclass
class CobolAnalysis:
    program_id: Optional[str]
    idms_records: List[str]
    obtain_calc_records: List[str]
    obtain_next: List[tuple[str, str]]
    obtain_owner_sets: List[str]
    find_first_sets: List[str]

@dataclass
class Paragraph:
    name: str
    text: str

```

